

Multiparty Session Types with a Bang!

MATTHEW ALAN LE BRUN, SIMON FOWLER, and ORNELA DARDHA, University of Glasgow, UK

Multiparty Session Types (MPST) statically verify properties of message-passing programs, however, their adoption in practice is limited. Practical distributed systems nowadays are built using the client-server paradigm, promoting modular design of scalable components. This paper integrates for the first time into MPST: *replication*, a construct modelling infinitely available servers; and *first-class roles*, allowing names of participants in the protocol to be bound and communicated. The type system developed models, and can verify properties of, modular client-server systems. Expressivity results are uncovered: replication increases expressivity w.r.t. the Chomsky Hierarchy, thus this work serves as the first account of context-free MPST; replication and recursion together can encode locks and mutual exclusion; and replication with first-class roles can model communication races, integral to representing ticketing protocols. This expressivity is demonstrated by formalising types for the dining philosophers problem, an online auction, and binary tree serialisation.

CCS Concepts: • **Theory of computation** → **Process calculi**; *Type theory*; Grammars and context-free languages.

Additional Key Words and Phrases: Multiparty session types, Replication, Free role names

ACM Reference Format:

Matthew Alan Le Brun, Simon Fowler, and Ornela Dardha. 2025. Multiparty Session Types with a Bang!. *ACM Trans. Program. Lang. Syst.* 0, 0, Article 0 (November 2025), 51 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

A common design pattern in distributed computing is that of client-server systems [29, 33], where a server is designed to be infinitely available to respond to client requests on demand. In such a paradigm, servers are agnostic of the number of clients in the system, and respond to requests as they are made; i.e., without a bias on ordering. This results in highly modular distributed systems, since the implementations of clients and servers are decoupled. Hence, systems become easier to develop and maintain, and can be easily integrated (since servers can be clients to other servers).

The world today is powered by a multitude of client-server systems working together by *communicating*, i.e., sending and receiving messages according to some *protocol*. It is therefore vital to *verify* the correctness of these communication protocols and their implementations, to ensure that programs behave according to their specifications, and to guarantee that the specifications are *safe*.

Session types [11, 13, 14] provide a lightweight method by which a developer can ensure safety of, and conformance to, communication protocols. They are *behavioural* type systems encoding, not only *what* data is, but also *how* it should be sent and received—ensuring that implemented code observes its prescribed communication pattern. Thereby, session types bridge the gap between the verification of protocols and their implementation, whilst providing guarantees of properties which are proven to be upheld during a system’s runtime. *Multiparty session types* (MPST) [5, 15, 26]

Authors’ Contact Information: [Matthew Alan Le Brun](mailto:matthewalan.lebrun@glasgow.ac.uk), matthewalan.lebrun@glasgow.ac.uk; [Simon Fowler](mailto:simon.fowler@glasgow.ac.uk), simon.fowler@glasgow.ac.uk; [Ornela Dardha](mailto:ornela.dardha@glasgow.ac.uk), ornela.dardha@glasgow.ac.uk, University of Glasgow, Glasgow, UK.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1558-4593/2025/11-ART0

<https://doi.org/XXXXXXX.XXXXXXX>

generalise binary session types to allow for reasoning about systems involving two or more named participants, aiming to provide communication-centric guarantees for interleaved communication patterns.

Example 1.1 (Client-Server-Worker). MPST may be used to describe communication protocols for client-server systems. Consider the following basic interactions between a *client*, *server* and *worker*:

$$S_c := \oplus s:\text{req}(\text{Int}) . \& w:\text{ans}(\text{String})$$

$$S_s := \& c:\text{req}(\text{Int}) . \oplus w:\text{fw}(\text{Int}) \quad S_w := \& s:\text{fw}(\text{Int}) . \oplus c:\text{ans}(\text{String})$$

The above describes types for a *client*, *server*, and *worker* respectively. The *client*, having type S_c , sends ($\oplus$) to the *server* a *request* with payload type *Int*, then ($\cdot$) waits to receive ($\&$) from the *worker* an *answer* with payload type *String*. The *server*, upon receiving the *request* from the *client*, *forwards* it to the *worker*. Lastly the *worker*, after receiving the *forwarded request* from the *server*, sends the *answer* to the *client*. A MPST system verifies that any written program code conforms to this specification (known as *session fidelity*), and that this protocol is *safe*—i.e., that processes never receive unexpected messages.

Despite the potential of MPST for safe client-server programming, there remain limitations to the theory that impede their adoption in practical systems. For instance, generalising Example 1.1 to multiple workers in the style of a load-balancer is non-trivial and has inspired a series of work on the generalisation of *direction of choice* [30]. Further, generalising the number of *clients* is also non-trivial—typically, MPST theories assume a global view of *all* participants in a session. Lastly, the *targets* of actions in MPST (e.g., the recipient of a sent message) are *hard coded* because role names are *constants*. The following demonstrates impracticalities that result from these qualities.

Example 1.2 (Motivation: Load Balancer). Consider a protocol, similar to that of Example 1.1, but now involving two workers. The protocol is defined by associating role names (participants) to their respective session types:

$$\Psi_{lb} = \{srv : S_{srv}, w_1 : S_w, w_2 : S_w, c : S_c\}$$

The intention with this protocol is to generalise Example 1.1 in the style of a load balancer, by involving an additional worker; doing this *safely* whilst maintaining a system that is *deadlock-free*, however, is not so trivial.

$$\begin{aligned} S_{srv} &= \& c:\text{req}(\text{Int}) . \oplus \begin{cases} w_1:\text{fw}(\text{Int}) . \oplus w_2:\text{stop}() . \oplus c:w1() \\ w_2:\text{fw}(\text{Int}) . \oplus w_1:\text{stop}() . \oplus c:w2() \end{cases} \\ S_w &= \& srv : \begin{cases} \text{fw}(\text{Int}) . \oplus c:\text{ans}(\text{String}) \\ \text{stop}() \end{cases} \\ S_c &= \oplus srv:\text{req}(\text{Int}) . \& srv : \begin{cases} w1() . \& w_1:\text{ans}(\text{String}) \\ w2() . \& w_2:\text{ans}(\text{String}) \end{cases} \end{aligned}$$

Starting from the type of the server *srv*, after receiving the client request, it now *selects* to send to one of the two workers (w_1 or w_2), continuing its protocol according to the (internally) chosen path. It then proceeds by informing the other worker that it was not chosen, by sending a *stop* message; and telling the client which worker is processing the request, via the *w1* or *w2* messages.

Workers with the S_w type wait to receive one of two possible messages from the server: either the *fw(Int)* message, then proceed by handling the client request; or the *stop* message, in which case it terminates. This external choice is referred to as a *branch*.

Lastly, the client issues its request to the server, branches from the server to receive a label identifying the chosen worker, and waits for the final answer.

Example 1.2 uncovers a number of issues with the *practicality* of using MPST for client-server systems. Firstly, the server- and client-side protocols are tightly coupled. For instance, adding a new worker to the system would require changes to both the S_{srv} and S_c types; and generalising the server-side to handle multiple clients is non-trivial—handling n clients where n is known requires the server to keep track of how many requests it services so as to only send `stop` labels when all clients are serviced. Handling an *unknown* number of clients and requests is not possible using standard MPST formalisms since: (i) MPSTs assume a statically known set of roles; (ii) the *targets* of actions (e.g., the recipient of a sent message) are *hard coded* (because role names are *constants*); and (iii) servers would not know when it is safe to terminate, and thus may result in deadlocks.

The design of client-server systems in practice relies on the *modularisation* of server-side and client-side components, and on making servers *reactive*—i.e., observed as infinitely available components that handle requests when prompted. This work addresses exactly this issue, by introducing—for the first time in MPST—*replication* and *first-class roles*.

Example 1.3 (Modular Load Balancer). By extending MPST with *replication* and *first-class roles*, Example 1.2 can be generalised to support *any number* of clients.

$$S_{srv} = !\&\alpha:\text{req}(\text{Int}) . \oplus \left\{ \begin{array}{l} w_1:\text{fw}(\text{Int}, \alpha) . \oplus \alpha:\text{wrk}(w_1) \\ w_2:\text{fw}(\text{Int}, \alpha) . \oplus \alpha:\text{wrk}(w_2) \end{array} \right.$$

$$S_w = !\&srv:\text{fw}(\text{Int}, \beta) . \oplus \beta:\text{ans}(\text{String})$$

The first difference is the use of the *bang* (!) operator, denoting a *replicated* receive, i.e., one which may occur *any number* of times. This makes the server agnostic to the *number* of requests. Second, a server now waits for requests—not from a specific client—but from *any* participant, binding the name of the sender to role variable α . This makes the server agnostic to the *source* of requests.

After receiving a request, the server makes a *choice* to forward it to one of two workers, notably whilst passing the name of the client as one of the payloads. Finally, the server informs the client of the choice it made by sending the name of the worker as a *payload* rather than a label.

The type of a worker is also updated to be replicated, as it is dependent on the number of requests forwarded by the server. Notably, it receives the name of the client in the forward message, binding it to β , and uses it to send the final answer. It is key to observe that the use of a control signal (`stop`) is no longer needed, since workers are modelling *infinitely available servers*, and thus are not expected to terminate.

A client may now be defined as:

$$S_c = \oplus srv:\text{req}(\text{Int}) . \&srv:\text{wrk}(\gamma) . \&\gamma:\text{ans}(\text{String})$$

As a result of the *replicated types* and *first-class roles*, *any number of clients* may now be instantiated with type S_c —without needing to make changes to the protocol on the server-side. Furthermore, clients may also be updated to make different numbers of requests, and the server-side protocol could still remain the same. Conversely, if the number of workers in the load balancer were to change, the client type remains safe. Therefore, protocol design for server- and client-side components has been *decoupled*.

It so turns out that the use of replication and first-class roles in MPST has several surprising consequences. As will be discussed, and demonstrated by example, the following observations are made:

- (1) Replication and (tail) recursion in MPST are mutually non-inclusive, and using both constructs together allows for modelling of protocols that rely on *locks* and *mutual exclusion*;
- (2) Replication increases the expressivity of the type system to be capable of describing *context-free* protocols (e.g. binary tree serialisation);

- (3) First-class roles, when mixed with replication, act as a fair way of introducing and handling communication *races*.

Contributions. The overarching contribution of this work is the first integration of replication and first-class roles into a generalised MPST calculus, with an exploration of the impacts of these extensions on expressivity and decidability. The specific contributions are as follows:

- (1) Section 2 presents MPST!, the first multiparty session-typed language with *replication* and *first-class roles*, and its *metatheory* is proven in Section 3.
- (2) Several expressiveness results are shown through a series of representative examples (Section 4.1). In particular, replication lifts the expressive power of types, thus MPST! serves as the first account of *context-free* MPST. Furthermore, in MPST, replication and recursion are mutually non-inclusive—a type system combining both constructs allows for modelling *races* and *mutual exclusion*. These results are demonstrated through non-trivial examples including *binary tree serialisation*, the *dining philosophers problem*, and an *auction service*.
- (3) In Section 4.2, it is shown that the decidability of typechecking is contingent on the decidability of a given safety property; and sublanguages for which typechecking is decidable are identified. (Importantly, these sublanguages are still expressive enough to capture the examples demonstrated in the paper.)

Section 5 gives an account of related work and Section 6 concludes the chapter. This journal is an extended version of a proceedings paper [20].

2 Multiparty Session Types with a Bang!

This section introduces MPST!, an extension of standard multiparty session calculi [15, 25, 28], with support for *replication* and *first-class roles*.

2.1 Language

Figure 1 shows the syntax of MPST!.

Names, values, and binders. A *session name*, ranged over by $s, s', \dots$, represents a collection of interconnected participants. A *role* is a participant in a multiparty communication protocol, and each *communication endpoint* $s[q]$ is obtained by indexing a session name with a role. In contrast to existing MPST calculi, MPST! supports *first-class* roles, meaning that a role may be communicated as part of a message. To this end, a role ρ may either be a concrete role q (e.g., srv, w_1, w_2 , in the load balancer Example 1.2) or a *role variable* α .

A name a is either an endpoint, a variable, or a role variable, whereas a value V is either a channel, a basic value, or a role. Binders b are used when receiving a message and can either be a variable binder or a role variable binder. Concrete values d are used when sending a message (at runtime) and are either an endpoint, a basic value, or a concrete role.

Processes. Processes are ranged over by $P, Q, R, \dots$: *session restriction* $(\nu s)P$ binds session name s in process P ; and process $P|Q$ denotes P and Q running in parallel.

MPST! supports *choice-guarded output* $\sum_{i \in I} c_i[\rho_i] \oplus m_i(\vec{V}_i) . P_i$, allowing a nondeterministic send along any c_i to role ρ_i with label m_i and payload $\vec{V}_i$, with the process continuing as P_i . (It is assumed that all channels c_i are the same.) *Branching receive* $c[\rho] \&_{i \in I} m_i(\vec{b}_i) . P_i$ denotes a process waiting on channel c for one of a set of messages from role ρ with label m_i , binding the received data to binders $\vec{b}_i$ before continuing according to P_i . It is key to note that the subject of a communication action is indicated via ρ (for both sending and receiving), which can either be a hard-coded value, or a role variable.

<i>Concretes</i>	$d ::= s[q] \mid v \mid q$
<i>Channels</i>	$c ::= x \mid s[q]$
<i>Binders</i>	$b ::= x \mid \alpha$
<i>Names</i>	$a ::= c \mid \alpha$
<i>Values</i>	$V ::= c \mid v \mid \rho$
<i>Roles</i>	$\rho ::= q \mid \alpha$
<i>Processes</i>	$P, Q ::= (vs)P \mid P Q \mid \sum_{i \in I} P_i^\oplus \mid P^\& \mid !P^\& \mid X\langle \vec{V} \rangle \mid \mathbf{0}$
<i>Send Process</i>	$P^\oplus ::= c[\rho] \oplus m\langle \vec{V} \rangle . P$
<i>Receive Process</i>	$P^\& ::= c[\rho] \&_{i \in I} m_i(\vec{b}_i) . P_i$
<i>Programs</i>	$\mathcal{P} ::= (P, D)$
<i>Definitions</i>	$D ::= \emptyset \mid D, X \mapsto (\vec{b}, \vec{T}, P)$
<i>Protocols</i>	$\Psi ::= \{\rho : S_\rho\}_{\rho \in I}$

Fig. 1. Syntax of MPST!

Another key difference to standard formalisms is that receiving processes $P^\&$ may be optionally prefixed with the *bang* $!$ operator, identifying it as *replicated*—i.e., it may be used 0 or more times, modelling *infinitely available servers*.

Lastly, *process call* $X\langle \vec{V} \rangle$ models recursion and $\mathbf{0}$ denotes the *empty process*.

Programs, Definitions, and Protocols. A *program* is what is written by an end-user of the language; it is a tuple (P, D) of a “main” process P and process *definitions* D . Definitions map process names (ranged over by $X, Y, \dots$) to process bodies $(\vec{b}, \vec{T}, P)$ to be used for recursion, where $\vec{b}$ is a list of accepted arguments, $\vec{T}$ is a list of types for said arguments, and P is the process in which the arguments are used. Finally, *protocols* are *session type* specifications of roles in a given session. Formally, Ψ is a set of role-session type pairs, $\{\rho : S_\rho\}_{\rho \in I}$ for a non-empty I , defining the communication patterns that each participant of a session should follow.

2.2 Process Semantics

MPST! is enriched with operational semantics to model reduction via communication.

Definition 2.1 (Process reduction). A process P reduces parametric of some process definitions D , written $P \rightarrow_D P'$, via the operational semantics defined in Figure 2 (bottom). Multistep reductions are expressed using the reflexive and transitive closure, i.e., $P \rightarrow_D^* P'$.

Definition 2.1 defines process reduction $(\rightarrow_D)$ up-to structural congruence $(\equiv)$, both given in Figure 2. Structural congruence for MPST! is similar to what is standard for the π -calculus. Parallel composition is commutative and associative with identity element $\mathbf{0}$; session restriction is commutative and has absorbing element $\mathbf{0}$; and scope extrusion allows restrictions to move in/out of parallel compositions provided there are no naming collision.

Structural Congruence

$$P \equiv Q$$

$$P \mid Q \equiv Q \mid P \quad P \mid (Q \mid R) \equiv (P \mid Q) \mid R \quad P \mid \mathbf{0} \equiv P \quad (vs)\mathbf{0} \equiv \mathbf{0}$$

$$(vs_1)(vs_2)P \equiv (vs_2)(vs_1)P \quad \frac{s \notin fs(Q)}{(vs)(P \mid Q) \equiv (vs)P \mid Q}$$

Process Reduction

$$P_1 \rightarrow_D P_2$$

R-C

$$s[\mathbf{p}][\mathbf{q}] \oplus m_k \langle \vec{d} \rangle . P \mid s[\mathbf{q}][\mathbf{p}] \&_{i \in I} m_i(\vec{b}_i) . Q_i \rightarrow_D P \mid Q_k \{ \vec{d} / \vec{b}_k \} \quad \text{if } k \in I$$

R-!C₁

$$\frac{R = !s[\mathbf{q}][\mathbf{p}] \&_{i \in I} m_i(\vec{b}_i) . Q_i}{s[\mathbf{p}][\mathbf{q}] \oplus m_k \langle \vec{d} \rangle . P \mid R \rightarrow P \mid R \mid Q_k \{ \vec{d} / \vec{b}_k \}} \quad \text{if } k \in I$$

R-+

$$\frac{j \in I}{\sum_{i \in I} P_i^\oplus \rightarrow_D P_j^\oplus}$$

R-!C₂

$$\frac{R = !s[\mathbf{q}][\alpha] \&_{i \in I} m_i(\vec{b}_i) . Q_i}{s[\mathbf{p}][\mathbf{q}] \oplus m_k \langle \vec{d} \rangle . P \mid R \rightarrow P \mid R \mid Q_k \{ \vec{d} / \vec{b}_k \} \{ \mathbf{p} / \alpha \}} \quad \text{if } k \in I$$

R-X

$$\frac{D(X) = (\vec{x}, \vec{T}, P)}{X \langle \vec{d} \rangle \rightarrow_D P \{ \vec{d} / \vec{x} \}}$$

R-|

$$\frac{P \rightarrow_D P'}{P \mid Q \rightarrow_D P' \mid Q}$$

R-ν

$$\frac{P \rightarrow_D P'}{(vs)P \rightarrow_D (vs)P'}$$

R-≡

$$\frac{P \equiv P' \quad P \rightarrow_D Q \quad Q \equiv Q'}{P' \rightarrow_D Q'}$$

Fig. 2. Operational semantics for MPST!

Rule R-C shows synchronous communication between two processes in session s . The first process, playing role $\mathbf{p}$, is sending a message with label m_k and payloads $\vec{d}$. The second process, playing role $\mathbf{q}$, offers role $\mathbf{p}$ a choice of message labels and has associated process continuations. The processes reduce to the selected continuation with the transmitted payloads substituted for the binders in the selected branch.

Rules **R-!C₁** and **R-!C₂** describe communication with a *replicated* process R . **Rule R-!C₁** is similar to R-C but the replicated process remains unchanged and the continuation Q_k is evaluated in parallel. **Rule R-!C₂** handles the case where the replicated process does not need to receive from a specific role, but instead allows communication with an *arbitrary* role: the rule binds the sending role to α in the replicated continuation. This is referred to as *universal receive*.

Rule R-+ evaluates a branching output by nondeterministically evaluating to one of the sending branches; and **Rule R-X** handles a recursive call. Rules **R-|** and **R-ν** allow reduction under parallel composition and session restriction respectively. Finally, **Rule R-≡** allows for reduction modulo structural congruence.

Example 2.2 (Load Balancer: Process Reduction). This example presents processes, and their reductions, for the protocol given in Example 1.3. The program in question uses process definitions D , containing the definitions given in Figure 3. Consider a single client in parallel with three server-side processes:

$$\begin{aligned} & \text{CInt}\langle s[\mathbf{c}], 42 \rangle \quad | \quad \text{Srv}\langle s[\mathbf{srv}] \rangle \quad | \quad \text{Wrk}\langle s[\mathbf{w}_1] \rangle \quad | \quad \text{Wrk}\langle s[\mathbf{w}_2] \rangle \\ \rightarrow_D^* & s[\mathbf{c}][\mathbf{srv}] \oplus \text{req}\langle 42 \rangle . P_c \quad | \quad !s[\mathbf{srv}][\alpha] \& \text{req}(x) . P_{\text{srv}} \quad | \quad \text{Wrk}\langle s[\mathbf{w}_1] \rangle \quad | \quad \text{Wrk}\langle s[\mathbf{w}_2] \rangle \end{aligned}$$

$$\begin{aligned}
 \text{Srv}(r : S_{\text{srv}}) &\mapsto !r[\alpha] \ \& \ \text{req}(x) \cdot \left(\begin{array}{c} r[w_1] \oplus \text{fw}\langle x, \alpha \rangle \cdot r[\alpha] \oplus \text{wrk}\langle w_1 \rangle \cdot 0 \\ + \\ r[w_2] \oplus \text{fw}\langle x, \alpha \rangle \cdot r[\alpha] \oplus \text{wrk}\langle w_2 \rangle \cdot 0 \end{array} \right) \\
 \text{Wrk}(w : S_w) &\mapsto !w[\text{srv}] \ \& \ \text{fw}(y, \beta) \cdot w[\beta] \oplus \text{ans}\langle f(y) \rangle \cdot 0 \\
 \text{Clnt}(c : S_c, n : \text{Int}) &\mapsto c[\text{srv}] \oplus \text{req}\langle n \rangle \cdot c[\text{srv}] \ \& \ \text{wrk}(y) \cdot c[y] \ \& \ \text{ans}(z) \cdot 0
 \end{aligned}$$

Fig. 3. Process definitions for load balancer example

Using **Rule R-!C₂**, the client and server reduce. The reduction advances the client to its continuation P_c , and pulls out a copy of the server's continuation as a new process. It is key to note that α is acting as a binder in the server process, therefore, in the continuation *role variable substitution* is observed:

$$\rightarrow_D P_c \mid !s[\text{srv}][\alpha] \ \& \ \text{req}(x) \cdot P_{\text{srv}} \mid P_{\text{srv}}\{42/x\}\{c/\alpha\} \mid \text{Wrk}\langle s[w_1] \rangle \mid \text{Wrk}\langle s[w_2] \rangle$$

$$\begin{aligned}
 \text{where } P_{\text{srv}}\{42/x\}\{c/\alpha\} &= \left(\sum_{i=1}^2 s[\text{srv}][w_i] \oplus \text{fw}\langle x, \alpha \rangle \cdot P'_{\text{srv}_i} \right) \{42/x\}\{c/\alpha\} \\
 &= \sum_{i=1}^2 s[\text{srv}][w_i] \oplus \text{fw}\langle 42, c \rangle \cdot (P'_{\text{srv}_i} \{42/x\}\{c/\alpha\})
 \end{aligned}$$

The spawned server process will then non-deterministically choose a worker to send to via **Rule R-+**; suppose w_1 is picked.

$$\begin{aligned}
 \rightarrow_D^* P_c \mid !s[\text{srv}][\alpha] \ \& \ \text{req}(x) \cdot P_{\text{srv}} \mid s[\text{srv}][w_1] \oplus \text{fw}\langle 42, c \rangle \cdot P'_{\text{srv}_1} \\
 \mid !s[w_1][\text{srv}] \ \& \ \text{fw}(y, \beta) \cdot P'_{w_1} \mid !s[w_2][\text{srv}] \ \& \ \text{fw}(y, \beta) \cdot P'_{w_2}
 \end{aligned}$$

Communication is now possible between the spawned server process and worker w_1 , using **Rule R-!C₁**. As before, this communication advances the sender process and pulls out a copy of the worker's continuation:

$$\begin{aligned}
 \rightarrow_D P_c \mid !s[\text{srv}][\alpha] \ \& \ \text{req}(x) \cdot P_{\text{srv}} \mid s[\text{srv}][c] \oplus \text{wrk}\langle w_1 \rangle \cdot 0 \\
 \mid !s[w_1][\text{srv}] \ \& \ \text{fw}(y, \beta) \cdot P'_{w_1} \mid s[w_1][\beta] \oplus \text{ans}\langle f(y) \rangle \cdot 0 \{42/y\}\{c/\beta\} \\
 \mid !s[w_2][\text{srv}] \ \& \ \text{fw}(y, \beta) \cdot P'_{w_2} \\
 = P_c \mid !s[\text{srv}][\alpha] \ \& \ \text{req}(x) \cdot P_{\text{srv}} \mid s[\text{srv}][c] \oplus \text{wrk}\langle w_1 \rangle \cdot 0 \\
 \mid !s[w_1][\text{srv}] \ \& \ \text{fw}(y, \beta) \cdot P'_{w_1} \mid s[w_1][c] \oplus \text{ans}\langle f(42) \rangle \cdot 0 \\
 \mid !s[w_2][\text{srv}] \ \& \ \text{fw}(y, \beta) \cdot P'_{w_2}
 \end{aligned}$$

The client can now learn which worker was chosen by the server, terminating the spawned server process, then the answer is exchanged between the worker and client:

$$\begin{aligned}
 \rightarrow_D s[c][w_1] \ \& \ \text{ans}(z) \cdot 0 \mid !s[\text{srv}][\alpha] \ \& \ \text{req}(x) \cdot P_{\text{srv}} \mid 0 \\
 \mid !s[w_1][\text{srv}] \ \& \ \text{fw}(y, \beta) \cdot P'_{w_1} \mid s[w_1][c] \oplus \text{ans}\langle f(42) \rangle \cdot 0 \\
 \mid !s[w_2][\text{srv}] \ \& \ \text{fw}(y, \beta) \cdot P'_{w_2} \\
 \rightarrow_D !s[\text{srv}][\alpha] \ \& \ \text{req}(x) \cdot P_{\text{srv}} \mid !s[w_1][\text{srv}] \ \& \ \text{fw}(y, \beta) \cdot P'_{w_1} \mid !s[w_2][\text{srv}] \ \& \ \text{fw}(y, \beta) \cdot P'_{w_2}
 \end{aligned}$$

Session types	$S ::= S^\& \mid !(S^\&) \mid S^\oplus \mid \mu t.S \mid t \mid \mathbf{end}$
Branching/Selection types	$S^\& ::= \&_{i \in I} \rho_i : \vec{m}_i(\vec{T}_i).S_i \quad S^\oplus ::= \oplus_{i \in I} \rho_i : \vec{m}_i(\vec{T}_i).S_i$
Role singletons	$\rho ::= q \mid \alpha$
Static types	$T ::= S \mid B \mid \rho$
Basic types	$B ::= \mathbf{Nat} \mid \mathbf{Real} \mid \mathbf{String} \mid \dots$
Runtime types	$U ::= S \mid (U_1 U_2)$

Fig. 4. Syntax of Types

2.3 Types

Figure 4 shows the syntax of MPST! types.

Syntax of types. Session types S type communication endpoints. They consist of branching types $S^\&$, replicated branching types $!(S^\&)$, selection types $S^\oplus$, recursive types $\mu t.S$ and variables t , and the **end** type.¹

Branching session types have the form $\&_{i \in I} \rho_i : \vec{m}_i(\vec{T}_i).S_i$ indicating that role ρ offers a choice of message labels $\vec{m}_i$ with payload types $\vec{T}_i$ and continuations S_i . Similarly, selection session types $\oplus_{i \in I} \rho_i : \vec{m}_i(\vec{T}_i).S_i$ indicate an internal choice towards one of a set of roles ρ_i , with a message label, given a list of payload types and then continues as the corresponding continuation type.

A replicated branching type $!\&_{i \in I} \rho_i : \vec{m}_i(\vec{T}_i).S_i$ types a replicated channel. Role ρ can be a concrete role, or a role variable in *binding* position: as will be seen later, the type semantics (cf. Figure 7) are responsible for pulling out copies of the type continuation and performing type-level substitution on roles.

The type system supports *singleton* types for roles: specifically a role ρ has singleton type ρ , and these are used to require that a specific role is passed as a payload. *Static* types T are those written as part of a protocol specification, whereas *runtime* types U are introduced by the type semantics to include a notion of *parallel composition* at the level of types: a type $U_1|U_2$ allows an endpoint to be associated with multiple “active” session types.

Branching and selection types are assumed to include a non-empty set of messages with pairwise distinct message names $\vec{m}_i$ (per role for $\oplus$). It is also required that recursion variables are guarded.

Definition 2.3 (Subtyping). The **subtyping relation** $\leq$ is co-inductively defined on types by the following inference rules:

$$\begin{array}{c}
\frac{(\vec{T}_i \leq \vec{T}'_i)_{i \in I} \quad (S_i \leq S'_i)_{i \in I}}{\&_{i \in I} \rho_i : \vec{m}_i(\vec{T}_i).S_i \leq \&_{i \in I} \rho_i : \vec{m}_i(\vec{T}'_i).S'_i} \quad \frac{(\vec{T}_i \geq \vec{T}'_i)_{i \in I} \quad (S_i \leq S'_i)_{i \in I}}{\oplus_{i \in I} \rho_i : \vec{m}_i(\vec{T}_i).S_i \leq \oplus_{i \in I} \rho_i : \vec{m}_i(\vec{T}'_i).S'_i} \quad \frac{S_1^\& \leq S_2^\&}{!(S_1^\&) \leq !(S_2^\&)} \\
\\
\frac{U \leq U' \quad S \leq S'}{U | S \leq U' | S'} \quad \frac{S \{\mu t.S/t\} \leq S'}{\mu t.S \leq S'} \quad \frac{S \leq S' \{\mu t.S'/t\}}{S \leq \mu t.S'} \quad \frac{}{T \leq T}
\end{array}$$

The definition of subtyping (Definition 2.3) is mostly standard. The adopted approach is the convention of smaller types being ones with less external choice and more internal choice (*à la*

¹For selection and (replicated) branching types with a single path, the shorthand notations $\rho \oplus \vec{m}(\vec{T}).S$, $\rho \& \vec{m}(\vec{T}).S$ and $!\rho \& \vec{m}(\vec{T}).S$ are sometimes used. Trailing **end** types are occasionally omitted.

Context splitting

$$\boxed{\Gamma = \Gamma_1 \cdot \Gamma_2}$$

$$\begin{array}{c} \frac{\Gamma = \Gamma_1 \cdot \Gamma_2}{\Gamma, c : U = \Gamma_1, c : U \cdot \Gamma_2} \quad \frac{\Gamma = \Gamma_1 \cdot \Gamma_2}{\Gamma, c : U = \Gamma_1 \cdot \Gamma_2, c : U} \quad \frac{\Gamma = \Gamma_1 \cdot \Gamma_2}{\Gamma, x : B = \Gamma_1, x : B \cdot \Gamma_2, x : B} \\ \\ \frac{}{\emptyset = \emptyset \cdot \emptyset} \quad \frac{\Gamma = \Gamma_1 \cdot \Gamma_2}{\Gamma, \alpha : \alpha = \Gamma_1, \alpha : \alpha \cdot \Gamma_2, \alpha : \alpha} \quad \frac{\Gamma = \Gamma_1 \cdot \Gamma_2}{\Gamma, c : U_1 | U_2 = \Gamma_1, c : U_1 \cdot \Gamma_2, c : U_2} \end{array}$$

Context addition

$$\boxed{\Gamma_1 + \Gamma_2 = \Gamma}$$

$$\begin{array}{c} \frac{\Gamma_1 + \Gamma_2 = \Gamma \quad a \notin \text{dom}(\Gamma_2)}{\Gamma_1, a : T + \Gamma_2 = \Gamma, a : T} \quad \frac{\Gamma_1 + \Gamma_2 = \Gamma \quad a \notin \text{dom}(\Gamma_1)}{\Gamma_1 + \Gamma_2, a : T = \Gamma, a : T} \quad \frac{\Gamma_1 + \Gamma_2 = \Gamma}{\Gamma_1, x : B + \Gamma_2, x : B = \Gamma, x : B} \\ \\ \frac{}{\Gamma + \emptyset = \Gamma} \quad \frac{\Gamma_1 + \Gamma_2 = \Gamma}{\Gamma_1, \alpha : \alpha + \Gamma_2, \alpha : \alpha = \Gamma, \alpha : \alpha} \quad \frac{\Gamma_1 + \Gamma_2 = \Gamma}{\Gamma_1, c : U_1 + \Gamma_2, c : U_2 = \Gamma, c : U_1 | U_2} \end{array}$$

Context role insertion

$$\boxed{\Gamma \leftarrow \rho}$$

$$\Gamma \leftarrow q = \Gamma \quad \Gamma \leftarrow \alpha = \Gamma + \alpha : \alpha$$

Fig. 5. Typing context operations.

Gay and Hole [10]). Subtyping of replication is based on regular branching; and parallel types are related iff their session types are subtypes. Subtypes are related up-to their recursive unfolding, and subtyping is *reflexive*. Subtyping is extended pointwise to lists of types $\vec{T}_1 \leq \vec{T}_2$, assuming the lists are of equal lengths.

Definition 2.4 (Type Congruence). *Type congruence* treats parallel runtime types as commutative and associative with identity element **end**.

$$U_1 | U_2 \equiv U_2 | U_1 \quad (U_1 | U_2) | U_3 \equiv U_1 | (U_2 | U_3) \quad U | \text{end} \equiv U$$

Contexts. For types to be useful, they must be associated with an entity within a program. Type contexts do exactly this. Definition 2.5 defines the type context for MPST!

Definition 2.5 (Type Contexts). The type context Γ maps channels to session types and variables to static types, and is defined as:

$$\Gamma ::= \emptyset \mid \Gamma, s[\rho] : U \mid \Gamma, x : T \mid \Gamma, \alpha : \alpha$$

Context *composition* Γ, Γ' is defined iff Γ and Γ' have disjoint domains.

Contexts are subtypes $\Gamma \leq \Gamma'$ iff $\text{dom}(\Gamma) = \text{dom}(\Gamma')$ and $\forall c \in \text{dom}(\Gamma) : \Gamma(c) \leq \Gamma'(c)$.

Context Γ maps channels to runtime types, variable names to static types, and role variable names to their associated singleton type. Context composition Γ, Γ' is defined only if both Γ and Γ' have disjoint domains. Subtyping and type congruence are lifted to contexts in the expected way.

Linearity plays a key part in session type theory. Since type context Γ holds information on both linear session types and unrestricted basic (and role) types, Figure 5 defines additional operations to help enforce linearity only where it is needed (as inspired by, e.g., [34]).

Linearity is enforced through the use of a *context split* operation $\Gamma = \Gamma_1 \cdot \Gamma_2$ that splits a context Γ into two environments Γ_1 and Γ_2 . These environments may share variables with non-linear types (basic types and role singletons). Additionally, a channel c with runtime type $U_1 | U_2$ may be split such that Γ_1 contains $c : U_1$ and Γ_2 contains $c : U_2$; this allows for typing of processes pulled out in parallel at runtime via Rules $R\text{-}!C_1$ and $R\text{-}!C_2$.

Typing Rules for Values

$$\boxed{\Gamma \vdash V : T}$$

$$\begin{array}{c} \text{T-SUB} \\ \frac{T \leq T'}{c : T \vdash c : T'} \\ \text{T-WKN} \\ \frac{\Gamma_1 + \Gamma_2 = \Gamma \quad \Gamma_1 \vdash V : T \quad \text{end}(\Gamma_2)}{\Gamma \vdash V : T} \\ \text{T-B} \\ \frac{v \in B}{\emptyset \vdash v : B} \\ \text{T-q} \\ \frac{}{\emptyset \vdash q : q} \\ \text{T-}\alpha \\ \frac{}{\alpha : \alpha \vdash \alpha : \alpha} \end{array}$$

Typing Rules for Programs and Processes

$$\boxed{\vdash \mathcal{P}} \quad \boxed{\Gamma \vdash_D P}$$

$$\begin{array}{c} \text{T-P} \\ \frac{\emptyset \vdash_D P \quad \forall i \in I : \{b_i : \vec{T}_i\} \vdash_D Q_i}{\vdash (P, D = \{X_i \mapsto (\vec{b}_i, \vec{T}_i, Q_i)\}_{i \in I})} \\ \text{T-X} \\ \frac{D(X) = (\vec{b}, \vec{T}, P) \quad \forall i \in 1..n : \Gamma_i \vdash V_i : \vec{T}_i \quad \text{end}(\Gamma_0)}{\Gamma_0(\cdot \Gamma_i)_{i \in 1..n} \vdash_D X((V_i)_{i \in 1..n})} \\ \text{T-!} \\ \frac{\Gamma_1 \vdash c : !\&_{i \in I} \rho : m_i(\vec{T}_i).S_i \quad \text{end}(\Gamma) \quad \varphi(\text{assoc}_s(\Psi)) \quad \varphi \text{ is a !-safety property} \quad s \notin \Gamma \quad \Gamma + \text{assoc}_s(\Psi) \vdash_D P}{\Gamma \cdot \Gamma_1 \vdash_D !c[\rho] \&_{i \in I} m_i(\vec{b}_i) . P_i} \\ \text{T-+} \\ \frac{(\Gamma \vdash_D P_i^{\oplus})_{i \in I}}{\Gamma \vdash_D \sum_{i \in I} P_i^{\oplus}} \\ \text{T-|} \\ \frac{\Gamma_1 \vdash_D P_1 \quad \Gamma_2 \vdash_D P_2}{\Gamma_1 \cdot \Gamma_2 \vdash_D P_1 | P_2} \\ \text{T-\&} \\ \frac{\Gamma_{\&} \vdash c : \&_{i \in I} \rho : m_i(\vec{T}_i).S_i \quad \Gamma_r \vdash \rho : \rho \quad (\Gamma + c : S_i + b_i : \vec{T}_i \vdash_D P_i)_{i \in I}}{\Gamma \cdot \Gamma_{\&} \cdot \Gamma_r \vdash_D c[\rho] \&_{i \in I} m_i(\vec{b}_i) . P_i} \\ \text{T-}\oplus \\ \frac{\Gamma_{\oplus} \vdash c : \oplus \rho : m(\vec{T}).S \quad \Gamma_r \vdash \rho : \rho \quad (\Gamma_i \vdash V_i : \vec{T}_i)_{i \in 1..n} \quad \Gamma + c : S \vdash_D P}{\Gamma \cdot \Gamma_{\oplus} \cdot \Gamma_r(\cdot \Gamma_i)_{i \in 1..n} \vdash_D c[\rho] \oplus m((V_i)_{i \in 1..n}) . P} \\ \text{T-0} \\ \frac{\text{end}(\Gamma)}{\Gamma \vdash_D 0} \end{array}$$

Fig. 6. Typing rules.

The (left) inverse operation is *context addition* $\Gamma_1 + \Gamma_2 = \Gamma$ combining Γ_1 and Γ_2 into an environment Γ ; again roles and unrestricted variables may be shared across environments. In the case of $\Gamma_1, c : U_1 + \Gamma_2, c : U_2$ (i.e., adding two session types associated with the same endpoint), context addition results in the channel c having the combined runtime type $U_1|U_2$.

The *role insertion* operation $\Gamma \leftarrow \rho$ is used when typing replicated receives and extends a context with a mapping $\alpha : \alpha$ in the case that ρ is a role variable, and leaves Γ unchanged otherwise.

As shown by the definitions of *session restriction*, users define session types using a protocol Ψ . Type contexts are extracted from a protocol using the *assoc* function (Definition 2.6), to be used for type checking.

Definition 2.6 (Session Association). Contexts can be obtained from protocols by *associating* each prescribed session type to a *session with role*. Formally:

$$\text{assoc}_s(\{\rho : S_{\rho}\}_{\rho \in I}) := \{s[\rho] : S_{\rho}\}_{\rho \in I}$$

Definition 2.7 (End-typed environment). A context is *end-typed*, written $\text{end}(\Gamma)$, iff its channels have type **end**: i.e., $\forall c \in \text{dom}(\Gamma) : \Gamma(c) \leq \text{end}$.

The final ingredient needed for type checking is Definition 2.7. Due to the *linearity* restriction on session types, the end predicate is needed to identify contexts that can be considered as “entirely used”, i.e., all the session types in that context have reached **end**.

2.3.1 Type Checking. Typing rules for MPST!, given in Figure 6, use three typing judgments: (i) the *value judgement* $\Gamma \vdash V : T$ assigns type T to value V under context Γ ; (ii) the *program judgement*

$\vdash \mathcal{P}$ checks whether a program is self-enclosed and *type safe*; and (iii) the *process judgement* $\Gamma \vdash_D P$ checks whether a process P is well-typed under a context Γ and process definitions D .

The value judgement has five rules. **Rule T-Sub** types variables and endpoints, allowing for subtyping. **Rule T-Wkn** allows weakening, i.e., if value V has type T under some environment Γ_1 , and there exists an end-typed environment Γ_2 such that $\Gamma_1 + \Gamma_2 = \Gamma$, then the rule allows the value to be typed under the larger context Γ . **Rule T-B** types a basic value v to type B iff v belongs to B . New to MPST! are **Rule T- q** , which types concrete roles as singleton types under the empty environment; and **Rule T- α** , which types a role variable provided that it is contained within the type environment.

The program judgement is only used by **Rule T- $\mathcal{P}$** to type a program (P, D) . This judgement does not use a type environment, and ensures that programs are *closed*. This is achieved by requiring the main process P to be well-typed under an empty context. Furthermore, process definitions are also required to be closed by typing them under a context containing only the arguments defined for that process name.

The final judgement types processes under type context Γ .

Rule T- ν makes the type system *generalised* [28]. Instead of forcing a specific property on types, session restriction requires that the defined protocol adheres to the minimum requirements for subject reduction. Specifically, the rule requires a proof of $\varphi(\text{assoc}_s(\Psi))$, where φ is any *safety* property—the *largest* safety property will be defined later. If the session protocol adheres to these requirements, and the session has not been previously established, then the restricted process can be typed using the newly added context. **Rule T-X** checks that the type of every value used in a process call match the expected types defined in the process definition D .

Rule T-! and **Rule T- $\&$** type replicated and non-replicated receives respectively. In both cases the rules check that the channel has a receiving session type. Both rules check that each branch is typable by extending a common typing context with the variables bound by the receives, along with the continuation type for the session channel. In the case of a replicated receive there are two differences: first, the context used to type each branch must be end-typed (in order to avoid duplicating linear resources). Second, since the role ρ may be a *binding* occurrence of a role variable α , the context used to type each branch must be extended with the role variable (if applicable) using the insertion operator.

Rule T- $\oplus$ types an output by checking that the sending channel can be mapped to a selection type with payload types that match the values being sent. The rule ensures that role ρ is either a value, or it exists in the type context—i.e., messages cannot be sent to unbound role variables. The selection type continuation should be used, along with the common context, to type the continuation process.

Rule T- $+$ types an output-directed choice; since this operation uses branching control flow, each choice must be typable under the same environment. Parallel composition **Rule T-|** types two parallel processes under a split environment (note that by the definition of context splitting in Figure 5, this rule automatically caters for processes pulled out of replicated receives at runtime). Lastly, **Rule T-0** types the inactive process under an end-typed environment.

2.4 Type Semantics

Integral to the generalised approach to MPST, type context Γ is enriched with a labelled transition system (LTS) aiming to simulate the communication protocol of a program. This LTS is what will be used to define type-level properties and allow for protocol verification.

Definition 2.8 (Context reduction). Labels for the type LTS are given by actions A below, representing *output*, *input*, and *communication*.

$$\text{Actions } A ::= s:q\oplus r:m(\vec{T}) \mid s:q\& \rho:m(\vec{T}) \mid s:q,r:m$$

Context Reduction

$$\boxed{\Gamma \xrightarrow{A} \Gamma'}$$

$$\begin{array}{c}
\text{540} \\
\text{541} \\
\text{542} \quad \Gamma\text{-}\& \quad \Gamma\text{-}\oplus \quad \Gamma\text{-}\mu \\
\text{543} \quad \frac{S = \&_{i \in I} \mathbf{q} : \mathbf{m}_i(\vec{T}_i).S'_i \quad k \in I}{s[\mathbf{p}] : S \xrightarrow{s:\mathbf{p}\&\mathbf{q}:\mathbf{m}_k(\vec{T}_k)} s[\mathbf{p}] : S'_k} \quad \frac{S = \oplus_{i \in I} \mathbf{q} \mathbf{i} : \mathbf{m}_i(\vec{T}_i).S'_i \quad k \in I}{s[\mathbf{p}] : S \xrightarrow{s:\mathbf{p}\oplus\mathbf{q}_k:\mathbf{m}_k(\vec{T}_k)} s[\mathbf{p}] : S'_k} \quad \frac{\Gamma \cdot \mu}{\Gamma \cdot c : S\{\mu t.S/t\} \xrightarrow{A} \Gamma'} \\
\text{544} \\
\text{545} \\
\text{546} \\
\text{547} \quad \Gamma\text{-}! \quad \Gamma\text{-CONG}_1 \quad \Gamma\text{-CONG}_2 \\
\text{548} \quad \frac{R = !\&_{i \in I} \mathbf{p} : \mathbf{m}_i(\vec{T}_i).S_i \quad k \in I}{s[\mathbf{q}] : R \xrightarrow{s:\mathbf{q}\&\mathbf{p}:\mathbf{m}_k(\vec{T}_k)} s[\mathbf{q}] : R | S_k} \quad \frac{\Gamma \xrightarrow{A} \Gamma'}{\Gamma, \Gamma'' \xrightarrow{A} \Gamma' + \Gamma''} \quad \frac{\Gamma \xrightarrow{A} \Gamma'}{\Gamma \cdot \Gamma'' \xrightarrow{A} \Gamma' + \Gamma''} \\
\text{549} \\
\text{550} \\
\text{551} \\
\text{552} \quad \Gamma\text{-COM}_1 \quad \Gamma\text{-COM}_2 \\
\text{553} \quad \frac{\Gamma = \Gamma_1 \cdot \Gamma_2 \quad \vec{T}'_1 \leq \vec{T}'_2 \quad \vec{T}_1 = (\vec{r}, \vec{T}'_1) \quad \vec{T}_2 = (\vec{\alpha}, \vec{T}'_2)}{\Gamma_1 \xrightarrow{s:\mathbf{p}\&\mathbf{q}:\mathbf{m}(\vec{T}_1)} \Gamma'_1 \quad \Gamma_2 \xrightarrow{s:\mathbf{q}\&\mathbf{p}:\mathbf{m}(\vec{T}_2)} \Gamma'_2} \quad \frac{\Gamma = \Gamma_1 \cdot \Gamma_2 \quad \vec{T}'_1 \leq \vec{T}'_2 \quad \vec{T}_1 = (\vec{r}, \vec{T}'_1) \quad \vec{T}_2 = (\vec{\alpha}, \vec{T}'_2)}{\Gamma_1 \xrightarrow{s:\mathbf{p}\oplus\mathbf{q}:\mathbf{m}(\vec{T}_1)} \Gamma'_1 \quad \Gamma_2 \xrightarrow{s:\mathbf{q}\&\mathbf{\alpha}':\mathbf{m}(\vec{T}_2)} \Gamma'_2} \\
\text{554} \\
\text{555} \\
\text{556} \quad \frac{\Gamma_1 \xrightarrow{s:\mathbf{p}\&\mathbf{q}:\mathbf{m}(\vec{T}_1)} \Gamma'_1 \quad \Gamma_2 \xrightarrow{s:\mathbf{q}\&\mathbf{p}:\mathbf{m}(\vec{T}_2)} \Gamma'_2}{\Gamma \xrightarrow{s:\mathbf{p},\mathbf{q}:\mathbf{m}} \Gamma'_1 + \Gamma'_2\{\vec{r}/\vec{\alpha}\}} \quad \frac{\Gamma_1 \xrightarrow{s:\mathbf{p}\oplus\mathbf{q}:\mathbf{m}(\vec{T}_1)} \Gamma'_1 \quad \Gamma_2 \xrightarrow{s:\mathbf{q}\&\mathbf{\alpha}':\mathbf{m}(\vec{T}_2)} \Gamma'_2}{\Gamma \xrightarrow{s:\mathbf{p},\mathbf{q}:\mathbf{m}} \Gamma'_1 + \Gamma'_2\{\vec{r}/\vec{\alpha}\}\{\mathbf{p}/\mathbf{\alpha}'\}} \\
\text{557} \\
\text{558} \\
\text{559} \\
\text{560} \\
\text{561}
\end{array}$$

Fig. 7. Type semantics.

The roles involved in an action are referred to by the (partial) function $\text{roles}(A)$, defined as:

$$\text{roles}(s:\mathbf{p}\oplus\mathbf{q}:\mathbf{m}(\vec{T})) = \text{roles}(s:\mathbf{p}\&\mathbf{q}:\mathbf{m}(\vec{T})) = \text{roles}(s:\mathbf{p},\mathbf{q}:\mathbf{m}) = \{\mathbf{p}, \mathbf{q}\}$$

The type LTS is defined by the transitions listed in Figure 7. Context *reduction*, $\Gamma \rightarrow \Gamma'$, is defined iff $\Gamma \xrightarrow{A} \Gamma'$ where $A = s:\mathbf{q},\mathbf{r}:\mathbf{m}$ for any $s, \mathbf{q}, \mathbf{r}, \mathbf{m}$ —i.e., a context reduces iff it can transition via a communication action. The transitive and reflexive closure of context reduction is written as $\rightarrow^*$; and $\Gamma \rightarrow$ represents the existence of some transition from Γ .

Transitions $\Gamma\text{-}\&$ and $\Gamma\text{-}\oplus$ are standard: a context can fire an input label (resp. output label) matching any of the branches in the top-level branch type (resp. selection type). Doing so transitions the context entry to the continuation type of the selected branch.

Transition $\Gamma\text{-}!$ models the receipt of a message by a replicated input. The two main differences to the linear receive are: (i) the role $\mathbf{p}$ used in the transition label is allowed to be a role variable name; and (ii) firing an input does not advance the type, but instead pulls out a copy of the continuation and places it in parallel. Role $\mathbf{p}$ is considered bound in R and its continuations, but is *free* in pulled out copies of the continuations composed in parallel (S_k).

Transition rules $\Gamma\text{-CONG}_1$, $\Gamma\text{-CONG}_2$, $\Gamma\text{-}\mu$ allow contexts to reduce under a larger context, or when types are guarded by recursive binders.

Transitions $\Gamma\text{-COM}_1$ and $\Gamma\text{-COM}_2$ model type-level communication; for simplicity and without loss of generality, assume a convention wherein role-typed payloads are at the head. Both rules state that if a context can be split such that one part fires an output label, and the other fires an input with matching roles and message label, then the entire context can transition on a *communication* action provided the compatibility of the payload types (allowing for subtyping). Sender payloads must be subtypes of the receiver payloads, provided they are not role types, for which *role substitution* occurs after communication. Rule $\Gamma\text{-COM}_2$ additionally caters for universal receives, where the input label consists of a role variable in binding position—this is reflected in the role substitution.

Example 2.9 (Load Balancer: Context Reduction). Recalling the load balancer protocol from Example 1.3, context reduction is now demonstrated for a system with one server, two workers, and one client. Initially, the only communication action possible is between the client and server, via Rule $\Gamma\text{-Com}_2$.

$$\begin{aligned}
 & s[c] : S_c, s[srv] : S_{srv}, s[w_1] : S_w, s[w_2] : S_w \\
 & = \\
 & (s[c] : S_c \cdot s[srv] : S_{srv}) \cdot (s[w_1] : S_w, s[w_2] : S_w) \\
 & \rightarrow \\
 & s[c] : srv \& wrk(\gamma). \gamma \& \text{ans}(\text{String}). \text{end} \\
 & + \left(s[srv] : S_{srv} \left| \oplus \begin{cases} w_1.\text{fw}(\text{Int}, \alpha). \alpha \oplus wrk(w_1). \text{end} \\ w_2.\text{fw}(\text{Int}, \alpha). \alpha \oplus wrk(w_2). \text{end} \end{cases} \right\} c/\alpha \right) \\
 & + s[w_1] : S_w, s[w_2] : S_w \\
 & = \\
 & s[c] : srv \& wrk(\gamma). \gamma \& \text{ans}(\text{String}). \text{end}, \\
 & s[srv] : S_{srv} \left| \oplus \begin{cases} w_1.\text{fw}(\text{Int}, c). c \oplus wrk(w_1). \text{end} \\ w_2.\text{fw}(\text{Int}, c). c \oplus wrk(w_2). \text{end} \end{cases} \right., s[w_1] : S_{w_1}, s[w_2] : S_{w_2} \\
 & \text{It is key to note the role substitution for the type of } s[srv] \text{ above; specifically, how the substitution} \\
 & \text{affects the parallel type extracted through communication, but does not affect the replicated type} \\
 & S_{srv}. \text{ This is because only the role variables in the pulled out continuation are considered free. From} \\
 & \text{here, there are multiple reduction paths. Consider the path in which } srv \text{ communicates with } w_1. \\
 & = \\
 & s[c] : srv \& wrk(\gamma). \gamma \& \text{ans}(\text{String}). \text{end}, s[w_2] : S_w, s[srv] : S_{srv} \\
 & \cdot \left(s[srv] : \oplus \begin{cases} w_1.\text{fw}(\text{Int}, c). c \oplus wrk(w_1). \text{end} \\ w_2.\text{fw}(\text{Int}, c). c \oplus wrk(w_2). \text{end} \end{cases} \cdot s[w_1] : S_w \right) \\
 & \rightarrow \\
 & s[c] : srv \& wrk(\gamma). \gamma \& \text{ans}(\text{String}). \text{end}, s[w_2] : S_w, s[srv] : S_{srv} \\
 & + (s[srv] : c \oplus wrk(w_1). \text{end} + (s[w_1] : S_w \mid \gamma \& \text{ans}(\text{String}). \text{end}) \{c/\gamma\}) \\
 & = \\
 & s[c] : srv \& wrk(\gamma). \gamma \& \text{ans}(\text{String}). \text{end}, s[w_2] : S_w, \\
 & s[srv] : S_{srv} \mid c \oplus wrk(w_1). \text{end}, s[w_1] : S_w \mid \gamma \& \text{ans}(\text{String}). \text{end}
 \end{aligned}$$

Note how reduction is possible because the context split allows the server's parallel type to be extracted into its own context. Then, reduction occurs via Rule $\Gamma\text{-Cong}_1$ and Rule $\Gamma\text{-Com}_1$. From here, the context reduces in a similar fashion: first the server communicates a role with the client; followed by the final communication between the client and worker.

$$\begin{aligned}
 & \rightarrow \\
 & s[c] : w_1 \& \text{ans}(\text{String}). \text{end}, s[srv] : S_{srv} \mid \text{end}, \\
 & s[w_1] : S_w \mid c \oplus \text{ans}(\text{String}). \text{end}, s[w_2] : S_w \\
 & \rightarrow \\
 & s[c] : \text{end}, s[srv] : S_{srv} \mid \text{end}, s[w_1] : S_w \mid \text{end}, s[w_2] : S_w \\
 & \equiv \\
 & s[c] : \text{end}, s[srv] : S_{srv}, s[w_1] : S_w, s[w_2] : S_w
 \end{aligned}$$

A formal definition of free role variables (frv) is given in Figure 8 (top). In selection types, the free role variables are the ones in the target position and in payloads. For branching, payloads are

Free role variables in types

$$\text{frv}(T) = \{\alpha_i\}_{i \in I}$$

$$\text{frv}(B) = \text{frv}(q) = \text{frv}(t) = \text{frv}(\text{end}) = \emptyset$$

$$\text{frv}(\alpha) = \{\alpha\}$$

$$\text{frv}(\mu t.S) = \text{frv}(S)$$

$$\text{frv}(\oplus_{i \in I} p_i : m_i(\vec{T}_i).S_i) = \bigcup_{i \in I} \text{frv}(p_i) \cup \text{frv}(\vec{T}_i) \cup \text{frv}(S_i)$$

$$\text{frv}(\&_{i \in I} p_i : m_i(\vec{T}_i).S_i) = \text{frv}(p) \bigcup_{i \in I} (\text{frv}(S_i) \setminus \text{frv}(\vec{T}_i))$$

$$\text{frv}(!\&_{i \in I} p_i : m_i(\vec{T}_i).S_i) = \bigcup_{i \in I} (\text{frv}(S_i) \setminus (\text{frv}(p) \cup \text{frv}(\vec{T}_i)))$$

Substitution in contexts

$$\Gamma\{q/\alpha\} = \Gamma$$

$$\frac{}{(c : T)\{q/\alpha\} = c : (T\{q/\alpha\})} \quad \frac{\text{end}(\Gamma)}{\Gamma\{q/\alpha\} = \Gamma} \quad \frac{}{(\Gamma_1 \cdot \Gamma_2)\{q/\alpha\} = (\Gamma_1\{q/\alpha\}) \cdot (\Gamma_2\{q/\alpha\})}$$

$$\frac{}{(\Gamma_1, \Gamma_2)\{q/\alpha\} = (\Gamma_1\{q/\alpha\}), (\Gamma_2\{q/\alpha\})}$$

Fig. 8. Free role variables and substitution in contexts

in binding position, thus only a role variable in the target is free. Lastly, both the target role and payloads are in binding position for replicated receives.

Substitution on types $T\{q/\alpha\}$ then refers to replacing all free occurrences of α in T with q . The operation is lifted to type contexts in Figure 8 (bottom). Notably, substitution should only affect session-typed entities in contexts, thus it ignores end-typed environments and is propagated across context composition and context splits (to handle parallel types).

3 Metatheory

This section now proves the metatheory for MPST!, following the *generalised* approach to MPST [28]. Unlike traditional MPST theory, the *generalised* approach does not syntactically guarantee any properties on typed processes. Rather, it serves as a framework for verifying customisable runtime properties. This is accomplished by verifying properties on the type LTS for a given protocol, from which process-level properties may be inferred. For this to be possible, the metatheory of a generalised MPST system should provide two guarantees—*subject reduction* and *session fidelity*. The former guarantees that process reduction preserves typing, whilst the latter guarantees that processes follow at least one path of the type LTS. Both theorems together allow for runtime properties to be inferred from contexts, since no communication will occur at runtime that is not modelled by the type semantics. Furthermore, the theorems should hold for some largest set of contexts which can be described with a property φ . (This is the requirement made for typechecking in Rule T-v.) Essentially, φ should describe the minimum (and most general) requirements for *subject reduction* or *session fidelity* to hold. By proving these theorems parametric of some largest φ , the property can be re-instantiated with more specific properties without having to reprove any of the base metatheory. (In MPST!, φ should be instantiated with a *safety* property to guarantee subject reduction, and a *fidelity* property for session fidelity.) This is what makes the type system

generalised, allowing for verification of redefinable properties as long as they conform to the minimum requirements.

Given the calculus has been extended to include replication and first-class roles, some definitions of properties need to be updated from their standard formalisms—these are presented in Section 3.1. Section 3.2 presents the necessary lemmata and propositions on which the main results are dependent upon. *Subject reduction* is presented and proved in Section 3.3, and *session fidelity* in Section 3.4. Lastly, Section 3.5 updates process properties to allow for the infinite behaviour of replicated processes, and proves that the updated properties can be inferred from type contexts. Simplified proofs are presented in the main text; detailed proofs can be found in Appendices A–C.

3.1 Context Properties

Before presenting the main theorems of the calculus, some auxiliary definitions are required. The following formalises the notion of a *path* (Definition 3.2)—i.e., a sequence of transitions of a context—and defines properties on *type contexts* (Table 1). The definitions rely on an assumption that is made for all MPST! types henceforth: message labels used in linear communication are unique w.r.t. those used by replicated receives.

Definition 3.1 (Unique Label Sets). Message labels are assumed to be taken from one of two unique and infinitely countable sets L and R , such that $L \cap R = \emptyset$. Sending and receiving types are updated to be constructed using the following:

Linear Branch	$S^{\&} ::= \&_{i \in I} \mathbf{p}:\mathbf{m}_i(\vec{T}_i).S_i$	where $\forall i \in I : \mathbf{m}_i \in L$
Replicated Branch	$!S^{\&} ::= !\&_{i \in I} \mathbf{p}:\mathbf{m}_i(\vec{T}_i).S_i$	where $\forall i \in I : \mathbf{m}_i \in R$
Selection	$S^{\oplus} ::= \oplus_{i \in I} \mathbf{p}:\mathbf{m}_i(\vec{T}_i).S_i$	where $\forall i \in I : \mathbf{m}_i \in L \cup R$

Definition 3.2 (Path). A type context *path* is a possibly infinite sequence of type contexts, $\vec{\Gamma}$, s.t. $\forall \Gamma_i \in \vec{\Gamma} : \Gamma_i \rightarrow \Gamma_{i+1}$. A path $\vec{\Gamma}$ is said to be:

- (1) *terminating* iff $\vec{\Gamma}$ is finite;
- (2) *fair* iff $\forall \Gamma_i \in \vec{\Gamma} : \Gamma_i \xrightarrow{s:\mathbf{p},\mathbf{q}:\mathbf{m}} \Rightarrow \exists \Gamma_k \in \vec{\Gamma}, \mathbf{r}, \mathbf{m}' : k \geq i \wedge \Gamma_k \xrightarrow{s:\mathbf{p},\mathbf{r}:\mathbf{m}'} \Gamma_{k+1}$
- (3) *live* iff $\forall \Gamma_i \in \vec{\Gamma} :$
 - (a) $\Gamma_i \xrightarrow{s:\mathbf{p} \oplus \mathbf{q}:\mathbf{m}(\vec{T})} \Rightarrow \exists \Gamma_k \in \vec{\Gamma}, \mathbf{r}, \mathbf{m}' : k \geq i \wedge \Gamma_k \xrightarrow{s:\mathbf{p},\mathbf{r}:\mathbf{m}'} \Gamma_{k+1}$; and
 - (b) $\Gamma_i \xrightarrow{s:\mathbf{p} \& \mathbf{q}:\mathbf{m}(\vec{T})} \wedge \mathbf{m} \in L \Rightarrow \exists \Gamma_k \in \vec{\Gamma}, \mathbf{m}' \in L : k \geq i \wedge \Gamma_k \xrightarrow{s:\mathbf{q},\mathbf{p}:\mathbf{m}'} \Gamma_{k+1}$.

A context *path* is a possibly infinite sequence of context reductions. A path is said to be (i) *terminating* iff it is finite; (ii) *fair* iff at any point on the path, if a communication action is possible for some sender role, then there exists a future on that path where the context reduces via communication stemming from the same sender; and (iii) *live* iff at any point on the path, if a sender (resp. *non-replicated* receiver) is blocked, then there exists a future on that path where that role becomes unblocked. Given the semantic approach to replication at the type level, the definition of a live path differs to the standard (e.g. as defined in [36]). Using that version, protocols involving finite clients communicating with infinitely available servers would be incapable of being live, since the context would eventually reach a state where clients have type **end** and servers are capable of receiving (through a replicated branch). Therefore, the liveness condition is updated to only apply for *linear* branching types, and does not apply for replicated types (since they are *always* capable of firing an input action).

safe	$\left(\Gamma \xrightarrow{s:p \oplus q:m(\vec{T})} \wedge \Gamma \xrightarrow{s:q \& p:m'(\vec{T}')} \wedge m, m' \in L \implies \text{all} \left(\Gamma \xrightarrow{s:p,q:m} \right) \right)$ and $\Gamma \xrightarrow{s:p \oplus q:m(\vec{T})} \wedge \Gamma \xrightarrow{s:q \& p:m'(\vec{T}')} \wedge m, m' \in R \wedge p \in \{p, \alpha\} \implies \text{all} \left(\Gamma \xrightarrow{s:p,q:m} \right)$
fidelitous	$\Gamma \xrightarrow{s:p \oplus q:m(\vec{T})} \wedge \Gamma \xrightarrow{s:p,r:m'} \implies \forall \vec{A} : \Gamma \xrightarrow{\vec{A}} * \Gamma' \xrightarrow{s:p,q:m} \wedge p \notin \text{roles}(\vec{A})$
deadlock-free	$\Gamma \rightarrow^* \Gamma' \not\rightarrow \implies \text{end}!(\Gamma')$
terminating	Γ is deadlock free and all paths from Γ are terminating
never-terminating	$\Gamma \rightarrow^* \Gamma' \implies \Gamma' \rightarrow$
lock-free	$\Gamma \rightarrow^* \Gamma' \implies$ all fair paths from Γ' are also live
live	$\Gamma \rightarrow^* \Gamma' \implies$ all paths from Γ' are live

Table 1. Properties of a type context Γ , in MPST!

Definition 3.3 (All Enabled). A context with transition $\Gamma \xrightarrow{s:p,q:m}$ is said to be enabled for *all* labels m iff a context can perform the transition for every path labelled by m . Formally:

$$\text{all} \left(\Gamma \xrightarrow{s:p,q:m} \right) ::= \Gamma \xrightarrow{s:p,q:m} \wedge \Gamma = \Gamma_0(\cdot \Gamma_i^{\&})_{i \in I} \wedge \Gamma_0 \xrightarrow{s:q \& p:m(\vec{T}_i)} \\ \wedge \forall i \in I : \Gamma_i^{\&} = \Gamma_i^e, c_i : S_i \wedge \text{end}(\Gamma_i^e) \wedge \Gamma_i^{\&} \xrightarrow{s:q \& p:m(\vec{T}_i)} \implies \forall i \in I : (\Gamma_0 + \Gamma_i^{\&}) \xrightarrow{s:p,q:m}$$

Example 3.4. Consider the following contexts, Γ and Γ' .

$$\Gamma = \left\{ \begin{array}{l} s[p] : !S \mid q \& m(\text{Int}) \mid q \& m(\text{String}) \\ s[q] : p \oplus m(\text{Int}).S' \end{array} \right\} \quad \Gamma' = \left\{ \begin{array}{l} s[p] : !S \mid q \& m(\text{Int}) \mid q \& m(\text{Int}) \\ s[q] : p \oplus m(\text{Int}).S' \end{array} \right\}$$

Only context Γ' is *all-enabled* for transition $\xrightarrow{s:p,q:m}$. This is because Γ can be split in a way such that some splits can communicate via m , where others cannot, due to mismatching payloads. Specifically:

$$\Gamma = \{s[p] : !S, s[q] : p \oplus m(\text{Int}).S'\} \cdot \{s[p] : q \& m(\text{Int})\} \cdot \{s[p] : q \& m(\text{String})\}$$

Observe that:

$$\{s[p] : !S, s[q] : p \oplus m(\text{Int}).S'\} + \{s[p] : q \& m(\text{String})\} \xrightarrow{s:p,q:m}$$

Using the definitions for paths, message labels and the *all* predicate, context properties for MPST! types are presented in Table 1. A context is *safe* when: (i) if a linear message can be sent and received between two participants, then communication between them should be possible; and (ii) if a replicated message can be sent and received between two participants, then communication between them should be possible. In both cases, communication must be enabled for all paths prefixed with the label chosen by the sender. (Otherwise, context splits would not preserve safety, cf. Lemma B.1(1).) Splitting the condition between linear and replicated labels prevents protocols from being flagged as unsafe when a linear message can be sent, a replicated receive is possible, but the corresponding linear receive is hidden behind some action.

Example 3.5 (Modality of Labels in a Safe Context). Context Γ' below is safe w.r.t. the definition provided in Table 1, but would not be safe if linear and replicated receives were not treated separately.

$$\Gamma = \left\{ \begin{array}{l} s[p] : !q \& m.r \& m'.q \& m' \\ s[q] : p \oplus m.p \oplus m' \\ s[r] : p \oplus m' \end{array} \right\} \rightarrow \Gamma' = \left\{ \begin{array}{l} s[p] : !q \& m.r \& m'.q \& m' \mid r \& m'.q \& m' \\ s[q] : p \oplus m' \\ s[r] : p \oplus m' \end{array} \right\}$$

This is because Γ' can fire an input label at p from q , but q is not enabled with a corresponding output action towards p . However, if the modality of labels is taken into account, Γ' is said to fire a *replicated* input label at p from q whereas q is enabled with a *linear* output towards p .

A context is *fidelitous*² when: if a message can be output from p , and communication stemming from p is possible, then every message that can be output should be eventually capable of communication (possibly after some reductions not involving p). Fidelity is stronger than safety, as it requires the communication of all messages in a selection type irrespective of the destination role. Essentially, if at least one destination role in a selection can be reached, then all of them should be able to be reached. This property is only needed because of undirected selection.

Example 3.6 (Safe vs. Fidelitous). Note the following *safe* but not *fidelitous* context.

$$\Gamma = \left\{ s[p] : \oplus \left\{ \begin{array}{l} q:m.\text{end} \\ r:m'.\text{end} \end{array} \right. , \quad s[q] : \&p:m.\text{end}, \quad s[r] : \text{end} \right\}$$

Context Γ is safe since there are no *unexpected messages*—role q expects to receive a message m from role p , which does in fact offer this message. The protocol, however, is not fidelitous since only one out of the two choices are eventually capable of communication; the selection towards role r is deadlocked. (Later, safe but non-fidelitous contexts will be shown to type processes that violate session fidelity.)

A context is *deadlock-free* when: if a context cannot reduce, then it must be end-typed. Due to situations in which terminating clients communicate with replicated servers, the definition for *end-typed* contexts must be updated. Essentially, the end predicate used for typechecking is too strict for verifying deadlock-freedom. The property instead relies on a new *end!* predicate presented in Definition 3.7, allowing for leftover replicated types.

Definition 3.7 (End!-typed environment). A context is *end!-typed*, written $\text{end}!(\Gamma)$, iff all of its channels are replicated (up-to $\equiv$) or have the type **end**: i.e., $\forall c \in \text{dom}(\Gamma) : \Gamma(c) \equiv !S^{\&} \vee \Gamma(c) \leq \text{end}$.

The remaining properties can be described as follows. A *terminating* context is deadlock-free and all of its paths are finite. A *never-terminating* context can always take a next reduction step. A context is *lock-free* iff, under a fair scheduler, all outputs and inputs eventually become unblocked. Lastly, a context is *live* iff all outputs and inputs eventually become unblocked, regardless of the scheduler. (For detailed examples of lock-free vs. live contexts, the interested reader may consult [36, Example 9].)

3.2 Role Substitution

The following presents non-standard lemmata for the novel role substitution used in MPST!. Their intention is to provide further insight into the internal workings of first-class roles in the designed

²*Fidelitous* is an old English word—an adjective form of the noun fidelity—describing something to be *faithful*. The word fidelitous is used, rather than faithful, as it is describing a property that is non-standard in this field of research; it should not be confused with other “faithful” properties found in the literature.

calculus, and they are integral to the larger proofs of main results presented later. An exhaustive list of all the lemmata, along with their detailed proofs, can be found in Appendix A.

Lemma 3.8 states that role substitution is preserved through addition, whilst Lemma 3.9 states that it preserves subtyping. These are then used to present the main role substitution lemmas, for the value judgement (Lemma 3.10) and the process judgement (Lemma 3.11).

LEMMA 3.8. *If $\Gamma_1 + \Gamma_2 = \Gamma$, then $\Gamma_1\{q/\alpha\} + \Gamma_2\{q/\alpha\} = \Gamma\{q/\alpha\}$.*

PROOF. By induction on the derivation of $\Gamma_1 + \Gamma_2 = \Gamma$. A sample case is given below.

Case 1:
$$\frac{\Gamma_1 + \Gamma_2 = \Gamma \quad a \notin \text{dom}(\Gamma_2)}{\Gamma_1, a : T + \Gamma_2 = \Gamma, a : T}$$

First, observe that by Figure 8, substitution does not affect context domains.

$(\Gamma_1, a : T)\{q/\alpha\} = (\Gamma_1\{q/\alpha\}), (a : T\{q/\alpha\})$ (by Figure 8) (1)

$\Gamma_1\{q/\alpha\} + \Gamma_2\{q/\alpha\} = \Gamma\{q/\alpha\}$ (ind. hyp.) (2)

$a \notin \text{dom}(\Gamma_2\{q/\alpha\})$ (since $a \notin \text{dom}(\Gamma_2)$) (3)

$(\Gamma_1, a : T)\{q/\alpha\} + \Gamma_2\{q/\alpha\} = (\Gamma\{q/\alpha\}), (a : T\{q/\alpha\})$ (by (1), (2), (3), +) (4)

Case 1 holds. (by (4), Figure 8) (5)

Other cases are either similar or follow directly from the inductive hypothesis. $\square$

LEMMA 3.9. *$T \leq T'$ implies $T\{q/\alpha\} \leq T'\{q/\alpha\}$.*

PROOF. By coinduction on $T \leq T'$ (Definition 2.3).

Case 1:
$$\frac{}{T \leq T}$$

The reflexive case immediately holds since the substitution occurs on the same type on both sides.

Case 2:
$$\frac{(\vec{T}_i \leq \vec{T}'_i)_{i \in I} \quad (S_i \leq S'_i)_{i \in I}}{\&_{i \in I} \rho : m_i(\vec{T}_i).S_i \leq \&_{i \in I \cup J} \rho : m_i(\vec{T}'_i).S'_i}$$

From the coinductive hyp., it follows that $\forall i \in I : \vec{T}_i\{q/\alpha\} \leq \vec{T}'_i\{q/\alpha\}$ and $S_i\{q/\alpha\} \leq S'_i\{q/\alpha\}$. Furthermore, if $\rho = \alpha$, then the substitution occurs on both sides.

Other cases follow the same reasoning and hold directly from the coinductive hypothesis and since any role substitution made on the target role affects both sides equally. $\square$

LEMMA 3.10. *If $\Gamma \vdash V : T$, then $\Gamma\{q/\alpha\} \vdash V\{q/\alpha\} : T\{q/\alpha\}$.*

PROOF. By rule induction on $\Gamma \vdash V : T$. Two sample cases are given below.

Case T-Sub:
$$\frac{T \leq T'}{\Gamma = c : T + c : T'}. \text{ RTP: } (c : T)\{q/\alpha\} \vdash c\{q/\alpha\} : T'\{q/\alpha\}$$

Note that if c is a variable and T is *not* a session type, then role substitution has no effect and thus the case would hold trivially. Consider now $T = S$ and $T' = S'$.

$S \leq S'$ (by assumption) (1)

$S\{q/\alpha\} \leq S'\{q/\alpha\}$ (by (1) and Lemma 3.9) (2)

$(c : S)\{q/\alpha\} \vdash c : S'\{q/\alpha\}$ (by (2) and Rule T-Sub) (3)

$c\{q/\alpha\} = c$ (since $c = x$ or $c = s[p]$, for any x, s, p) (4)

Case T-Sub holds. (by (3), (4), and since $S = T$ and $S' = T'$) (5)

Case T-Wkn:
$$\frac{\Gamma_1 + \Gamma_2 = \Gamma \quad \Gamma_1 \vdash V : T \quad \text{end}(\Gamma_2)}{\Gamma \vdash V : T}. \text{ RTP: } \Gamma\{q/\alpha\} \vdash V\{q/\alpha\} : T\{q/\alpha\}$$

883 $\Gamma_1\{q/\alpha\} \vdash V\{q/\alpha\} : T\{q/\alpha\}$ (ind. hyp.) (1)
 884 $\text{end}(\Gamma_2)$ (by assumption) (2)
 885 $\Gamma_2 = \Gamma_2\{q/\alpha\}$ (by Figure 8) (3)
 886 $\Gamma_1\{q/\alpha\} + \Gamma_2 = \Gamma\{q/\alpha\}$ (by hyp., (3), and Lemma 3.8) (4)
 887 $\Gamma\{q/\alpha\} \vdash V\{q/\alpha\} : T\{q/\alpha\}$ (by (1), (2), (4)) (5)
 888 Case T-Wkn holds. (by (5)) (6)
 889 $\square$

890 LEMMA 3.11. If $\Gamma \vdash_D P$, then $\Gamma\{q/\alpha\} \vdash_D P\{q/\alpha\}$.
 891

892 PROOF. By rule induction on $\Gamma \vdash_D P$.
 893

894 **Case T-0:** $\frac{\text{end}(\Gamma)}{\Gamma \vdash_D 0}$. Since $\text{end}(\Gamma)$, then $\Gamma\{q/\alpha\} = \Gamma$ (by Figure 8).
 895

896 $\Gamma_{\oplus} \vdash c : \oplus p : m(\vec{T}).S \quad \Gamma_r \vdash \rho : \rho$
 897 $(\Gamma_i \vdash V_i : T_i)_{i \in 1..n} \quad \Gamma_0 + c : S \vdash_D P'$
 898 **Case T- $\oplus$:** $\frac{\Gamma_{\oplus} \vdash c : \oplus p : m(\vec{T}).S \quad \Gamma_r \vdash \rho : \rho \quad (\Gamma_i \vdash V_i : T_i)_{i \in 1..n} \quad \Gamma_0 + c : S \vdash_D P'}{\Gamma = \Gamma_0 \cdot \Gamma_{\oplus} \cdot \Gamma_r(\cdot \Gamma_i)_{i \in 1..n} \vdash_D P = c[\rho] \oplus m\langle (V_i)_{i \in 1..n} \rangle \cdot P'}$
 899

900 First observe that by the definition of role substitution on contexts (Figure 8), substitution distributes
 901 over context splits. Then observe that by the ind. hyp.:
 902

$$(\Gamma_0 + c : S)\{q/\alpha\} \vdash_D P'\{q/\alpha\}$$

903 Furthermore, by Lemma 3.10:
 904

$$\begin{aligned} \Gamma_{\oplus}\{q/\alpha\} &\vdash (c\{q/\alpha\}) : (\oplus p : m(\vec{T}).S\{q/\alpha\}) \\ \Gamma_r\{q/\alpha\} &\vdash (\rho\{q/\alpha\}) : (\rho\{q/\alpha\}) \\ \forall i \in 1..n : \Gamma_i\{q/\alpha\} &\vdash (V_i\{q/\alpha\}) : (T_i\{q/\alpha\}) \end{aligned}$$

905 Then, using T- $\oplus$, the case holds.
 906

907 All other cases follow the same steps, relying on the inductive hypothesis and Lemma 3.10. $\square$
 908

909 3.3 Subject Reduction

910 Using the lemmata from the previous section, the main results of the language are presented, starting
 911 with *subject reduction*. Subject reduction (Theorem 3.13) informally states that, if a well-typed
 912 process can reduce, then its reductums remain well-typed. As previously alluded to, the property is
 913 proven parametric of a largest safety property, which is updated for MPST! in Definition 3.12.
 914

915 **Definition 3.12 (Safety).** φ is a !-safety property on type environment Γ iff:
 916

- 917 $\varphi\text{-S}$ $\varphi(\Gamma)$ implies Γ is safe;
- 918 $\varphi\text{-}\rho$ $\varphi(\Gamma)$ implies $\text{frv}(\Gamma) = \emptyset$;
- 919 $\varphi\text{-}\mu$ $\varphi(\Gamma \cdot s[\rho] : \mu t.S)$ implies $\varphi(\Gamma \cdot s[\rho] : S\{\mu t.S/t\})$;
- 920 $\varphi\text{-}\rightarrow$ $\varphi(\Gamma)$ and $\Gamma \rightarrow \Gamma'$ implies $\varphi(\Gamma')$.
- 921 $\text{safe}_!(\Gamma)$ is written iff $\varphi(\Gamma)$ and φ is a !-safety property.

922 The property requires contexts (along with their reductions and recursive unfoldings) to be safe
 923 (as defined in Table 1). Furthermore, the property also requires that contexts are void of unbound
 924 role variables; otherwise, role variable singletons in contexts may end up being shadowed by role
 925 variables defined in different sessions.
 926

927 **THEOREM 3.13 (SUBJECT REDUCTION).** If $\Gamma \vdash_D P$ with $\text{safe}_!(\Gamma)$ and $P \rightarrow_D P'$, then $\exists \Gamma'$ s.t. $\Gamma \rightarrow^* \Gamma'$
 928 and $\Gamma' \vdash_D P'$ with $\text{safe}_!(\Gamma')$.
 929

PROOF. By induction on the derivation of $P \rightarrow_D P'$. A sample case is given below.

The assumptions are: (A1) $\Gamma \vdash_D P$; (A2) $\text{safe}_!(\Gamma)$; and (A3) $P \rightarrow_D P'$.

Case R-!C₂:

Begin by inferring the shape of processes P and P' from the reduction rule R-!C₂. Then by assumptions (A1) and inversion of typing rules, the shape of context Γ can be inferred.

$$\text{Let } R = !s[\mathbf{q}][\delta] \ \&_{i \in I} m_i(\vec{b}_i) \cdot Q'_i \quad (\text{definition}) \quad (1)$$

$$\text{Let } P_{\oplus} = s[\mathbf{p}][\mathbf{q}] \oplus m_k(\vec{d}) \cdot Q \quad (\text{definition}) \quad (2)$$

$$\text{Let } n = |\vec{d}| \quad (\text{definition}) \quad (3)$$

$$P = P_{\oplus} \mid R \quad (\text{by R-!C}_2 \text{ and (A3)}) \quad (4)$$

$$P' = Q \mid R \mid Q'_k \{\vec{d}/\vec{b}_k\} \{\mathbf{p}/\delta\} \quad (\text{by R-!C}_2 \text{ and (A3)}) \quad (5)$$

Without loss of generality, consider $\vec{d} = \vec{r}, \vec{d}'$ where $\vec{d}'$ does not contain role values.

$$\Gamma = \Gamma^{\oplus} \cdot \Gamma^! \vdash_D P \quad (\text{by (4), (A1), and T-|}) \quad (6)$$

$$\Gamma^{\oplus} = \Gamma_0^{\oplus} \cdot \Gamma_c^{\oplus} \cdot \Gamma_r^{\oplus} (\cdot \Gamma_{d_i}^{\oplus})_{i \in 1..n} \vdash_D P_{\oplus} \quad (\text{by (6), (4), T-}\oplus) \quad (7)$$

$$\Gamma_c^{\oplus} \vdash s[\mathbf{p}] : \oplus \mathbf{q} : m_k(\vec{r}, \vec{T}) \cdot S \quad (\text{by (7), T-}\oplus) \quad (8)$$

$$\Gamma_r^{\oplus} \vdash \mathbf{q} : \mathbf{q} \quad (\text{by (7), T-}\oplus) \quad (9)$$

$$\forall i \in 1..|\vec{r}| : \Gamma_{d_i}^{\oplus} \vdash d_i : \mathbf{r}_i \text{ and } \forall j \in |\vec{r}|..|\vec{T}| : \Gamma_{d_j}^{\oplus} \vdash d_j : T_j \quad (\text{by (7), T-}\oplus) \quad (10)$$

$$\Gamma_0^{\oplus} + s[\mathbf{p}] : S \vdash_D Q \quad (\text{by (7), T-}\oplus) \quad (11)$$

$$\Gamma^! = \Gamma_0^! \cdot \Gamma_c^! \vdash_D R \quad (\text{by (6), (4), T-!}) \quad (12)$$

$$\Gamma_c^! \vdash s[\mathbf{q}] : !\&_{i \in I} \delta : m_i(\vec{\alpha}_i, \vec{T}_i) \cdot S'_i \quad (\text{by (12), T-!}) \quad (13)$$

$$\text{end}(\Gamma_0^!) \quad (\text{by (12), T-!}) \quad (14)$$

$$\forall i \in I : \Gamma_0^! + s[\mathbf{q}] : S'_i + \overrightarrow{\alpha_i : \alpha_i} + \overrightarrow{x_i : T_i} \leftrightarrow \delta \vdash_D Q'_i \quad (\text{by (12), T-!}) \quad (15)$$

Since Γ models sending and receiving actions, and since Γ is safe (Table 1) by assumption (A2), then the transitions enabled for the context can be inferred. Lemma B.1 (used below) states that if a context is safe, then its splits are also safe in isolation.

$$\Gamma_c^{\oplus} \xrightarrow{s:\mathbf{p}\oplus\mathbf{q}:m_k(\vec{r},\vec{T})} \Gamma_c^{\oplus'} \quad (\text{by (8), T-Wkn, T-Sub, Definition 2.3}) \quad (16)$$

$$\Gamma_c^! \xrightarrow{s:\mathbf{q}\&\delta:m'_j(\vec{\alpha}_j,\vec{T}'_j)} \Gamma_{c_j}^! \text{ for } j \in I \quad (\text{by (13), T-Wkn, T-Sub, Definition 2.3}) \quad (17)$$

$$\Gamma_c^{\oplus} \cdot \Gamma_c^! \xrightarrow{s:\mathbf{p},\mathbf{q}:m_k} \Gamma_c^{\oplus'} + \Gamma_{c_k}^! \{\vec{r}, \mathbf{p}/\vec{\alpha}, \delta\} \quad (\text{by (16), (17), (A2), Lemma B.1, } \Gamma\text{-Com}_1) \quad (18)$$

$$\Gamma_c^{\oplus'} \vdash s[\mathbf{p}] : S \quad (\text{by (8), (16), } \Gamma\text{-}\oplus, \text{T-}\oplus) \quad (19)$$

$$\Gamma_{c_k}^! = \Gamma_c^! \cdot \Gamma_k \text{ such that:} \quad (\text{by (13), (17), } \Gamma\text{-!}) \quad (20)$$

$$\Gamma_k \vdash s[\mathbf{q}] : S'_k \quad (\text{by (20), (13), (17), } \Gamma\text{-!, T-!}) \quad (21)$$

$$\Gamma_k \{\vec{r}/\vec{\alpha}_k\} \{\mathbf{p}/\delta\} \vdash s[\mathbf{q}] : S'_k \{\vec{r}/\vec{\alpha}_k\} \{\mathbf{p}/\delta\} \quad (\text{by (21), Lemma 3.10}) \quad (22)$$

$$\vec{T} \leq \vec{T}'_k \quad (\text{by (16), (17), (18), } \Gamma\text{-Com}_1) \quad (23)$$

Finally, contexts under which the reduced processes are typed can be identified from the typing rules, and it is subsequently shown that a valid context is obtained from the previously observed context reduction. Lemma A.7 states that singleton types can be removed from a context if they do not appear free in the process, and Lemma A.8 is the standard substitution lemma.

$$\Gamma_0^{\oplus} + \Gamma_c^{\oplus'} \vdash_D Q \quad (\text{by (11), (19)}) \quad (24)$$

$$\Gamma_0^! + \Gamma_c^! \vdash_D R \quad (\text{by (12)}) \quad (25)$$

$$\Gamma_0^! + \Gamma_k \{\vec{r}/\vec{\alpha}_k\} \{\mathbf{p}/\delta\} + \overrightarrow{\alpha_i : \alpha_i} + \overrightarrow{x_i : T_i} \vdash_D Q'_k \{\vec{r}/\vec{\alpha}_k\} \{\mathbf{p}/\delta\} \quad (\text{by (15), (22), Lemma 3.11}) \quad (26)$$

$$\Gamma_0^! + \Gamma_k\{\vec{r}/\vec{\alpha}_k\}\{p/\delta\} + \overrightarrow{x_i : T_i^!} \vdash_D Q'_k\{\vec{r}/\vec{\alpha}_k\}\{p/\delta\} \quad (\text{by (26), Lemma A.7}) \quad (27)$$

$$\Gamma_0^! + \Gamma_k\{\vec{r}/\vec{\alpha}_k\}\{p/\delta\} + (\Gamma_{d_i}^\oplus)_{i \in 1..n} \vdash_D Q'_k\{\vec{d}/\vec{b}_k\} \quad ((10), (15), (23), \text{Lemma A.8}) \quad (28)$$

$$\Gamma_0^\oplus + \Gamma_c^\oplus + \Gamma_0^! + \Gamma_c^! + \Gamma_0^! + \Gamma_k\{\vec{r}/\vec{\alpha}_k\}\{p/\delta\} + (\Gamma_{d_i}^\oplus)_{i \in 1..n} \vdash_D P' \quad (\text{by (24), (25), (28), T-|}) \quad (29)$$

By (12) and (14), $\Gamma_0^!$ is end-typed. Assume without loss of generality that $\Gamma_0^!$ does not contain any session typed entities, since any end-typed channels could be grouped with a different split of the context instead. Then, it holds that $\Gamma_0^! = \Gamma_0^! \cdot \Gamma_0^!$. Lemma A.1 states that addition is the left-inverse of context splitting.

$$\Gamma \rightarrow \Gamma' = (\Gamma_c^\oplus + (\Gamma_c^! \cdot \Gamma_k\{\vec{r}/\vec{\alpha}_k\}\{p/\delta\})) + (\Gamma_0^! \cdot \Gamma_0^! \cdot \Gamma_0^\oplus \cdot \Gamma_r^\oplus (\cdot \Gamma_{d_i}^\oplus)_{i \in 1..n}) \quad (\text{by (18), } \Gamma\text{-Cong}_1) \quad (30)$$

$$= \Gamma_0^\oplus + \Gamma_c^\oplus + \Gamma_0^! + \Gamma_c^! + \Gamma_0^! + \Gamma_k\{\vec{r}/\vec{\alpha}_k\}\{p/\delta\} + (\Gamma_{d_i}^\oplus)_{i \in 1..n} + \Gamma_r^\oplus \quad (\text{by (30), Lemma A.1}) \quad (31)$$

$$\text{end}(\Gamma_r^\oplus) \quad (\text{by (9), T-Wkn, T-q}) \quad (32)$$

Also assume, without loss of generality, that $\Gamma_r^\oplus$ is void of end-typed channels. Then, the proof can be concluded using Lemma A.2, stating that a context void of channel mappings can be added to a type environment with no effect on the typing judgement.

$$\Gamma' \vdash_D P' \quad (\text{by (29), (31), (32), Lemma A.2}) \quad (33)$$

$$\text{safe}_!(\Gamma') \quad (\text{by (A2), (30), } \varphi\text{-}\rightarrow) \quad (34)$$

$$\text{Case R-!C}_2 \text{ holds.} \quad (\text{by (30), (33), (34)}) \quad (35)$$

□

3.4 Session Fidelity

Informally, *session fidelity* states that if a context can take a step, then there should exist at least one transition on the context that can be matched by the processes. In other words, the process is faithful to the session type, and is guaranteed to follow one of the protocol's paths at runtime. This is a strong property—as is standard in the literature [12, 18, 19, 28, 36], assumptions on programs are necessary in order to guarantee session fidelity in MPST!. The assumptions are that processes (i) are productive (i.e., cannot infinitely reduce without communicating), (ii) do not incorrectly interleave multiple sessions, and (iii) do not play the wrong roles at the wrong times. Each of these issues may result in a program being deadlocked in spite of sessions being deadlock free. The following formally defines restrictions on the calculus that guarantee these assumptions.

Definition 3.14 (Guarded definitions). Process definitions D are said to be *guarded* iff for all mappings of the form $X \mapsto (\vec{x}, \vec{T}, P)$ in D , if $Y\langle\vec{V}\rangle$ is a subterm of P , then: $\exists i \in 1..|\vec{x}|$ s.t. $T_i \leq S \not\leq \text{end}$ and $x_i \in \vec{V}$ implies $Y\langle\vec{V}\rangle$ is guarded by a communication action on x_i in P .

To ensure processes do not infinitely reduce without communicating, Definition 3.14 requires all recursive calls involving channels as arguments to be guarded by at least one action for every channel. This guarantees processes will be productive.

Definition 3.15 (Plays one role). Assume programs to be constructed via the grammar in Figure 9. Then, given the judgement $\Gamma \vdash_D P$, it is said that P *only plays role* p by Γ iff: (i) D is guarded; (ii) $\text{fv}(P) = \emptyset$; (iii) $\Gamma = \Gamma'$, $s[p] : U$ with $U \not\leq \text{end}$ and $\text{end}(\Gamma')$.

Furthermore, it is said that:

- (1) P *only plays role* p iff $\exists \Gamma, D$ s.t. $\Gamma \vdash_D P$ and the above conditions hold; and
- (2) *one-role* $_!(P)$ iff $P \equiv \big|_{p \in I} P_p$ where each P_p is either 0 (up-to $\equiv$), or only plays role p .

Processes	$P, Q ::= \sum_{i \in I} P_i^\oplus \mid P^\& \mid !P^\& \mid X(\vec{V}) \mid 0$
Programs	$\mathcal{P} ::= ((vs : \Psi) P_1 \mid \cdots \mid P_n, D)$

Fig. 9. Session Fidelity Restrictions on MPST!

The next assumption is that processes cannot deadlock (for reasons other than a deadlocked protocol). To ensure this, the calculus is first restricted to the grammar in Figure 9, requiring programs to only contain an ensemble of parallel processes communicating via a single multiparty session. This prevents programs from getting stuck due to incorrect interleaving of multiple sessions. Furthermore, using the restricted grammar, processes could still deadlock by playing the wrong role at the wrong time. To circumvent this issue, Definition 3.15 describes processes that can only play a single role within the program's single session. This is formalised by item 1 above, where a process must be typed by a context containing only a single non-end channel entry. Then, one-role!(P) from item 2 is used to describe an ensemble of such processes in parallel, each playing their own role in the session. (Note, multiple processes may possibly play the same role at runtime, resulting from communication with replicated branches.) These restrictions suffice for session fidelity.

Different from the standard formalism [28], but similar to work more based in practice [12], session fidelity in MPST! requires a property stronger than safety. This is because the presented calculus makes use of undirected selection. (An example context which is safe but violates session fidelity is given in Example 3.17.) Instead, the theorem is proved parametric of a *fidelity* property; Definition 3.16 describes the largest fidelity property, capturing all fidelitous contexts.

Note. Generalised MPST systems typically tend to use directed choice (in which case a safety property suffices), or use undirected selection but require a stronger property than that presented in Definition 3.16. For instance, Harvey et al. [12] require a property akin to lock-freedom.

Definition 3.16 (Fidelity). φ is a *!-fidelity* property on type environment Γ iff:

- (i) $\varphi(\Gamma)$ implies Γ is *fidelitous*;
- (ii) $\varphi(\Gamma)$ implies $\text{frv}(\Gamma) = \emptyset$;
- (iii) $\varphi(\Gamma \cdot s[\mathbf{p}] : \mu t.S)$ implies $\varphi(\Gamma \cdot s[\mathbf{p}] : S\{\mu t.S/t\})$;
- (iv) $\varphi(\Gamma)$ and $\Gamma \rightarrow \Gamma'$ implies $\varphi(\Gamma')$.

$\text{fid}_!(\Gamma)$ is written iff $\varphi(\Gamma)$ and φ is a *!-fidelity* property.

The fidelity property requires contexts to be *fidelitous*—i.e., if communication is enabled on at least one message in a selection, then it should be eventually enabled for all messages—and all role variables in a context to be bound (just as in Definition 3.12). These conditions should hold after type unfolding and context reduction.

Example 3.17 (Safety vs. Fidelity). Recall the *safe* but not *fidelitous* context from Example 3.6.

$$\Gamma = \left\{ s[\mathbf{p}] : \oplus \left\{ \begin{array}{l} \mathbf{q}:\mathbf{m}.\mathbf{end} \\ \mathbf{r}:\mathbf{m}'.\mathbf{end} \end{array} \right. , \quad s[\mathbf{q}] : \&\mathbf{p}:\mathbf{m}.\mathbf{end}, \quad s[\mathbf{r}] : \mathbf{end} \right\}$$

This context can type a process violating session fidelity, i.e., the type context can reduce but the process is stuck.

$$\Gamma \vdash_\emptyset \quad s[\mathbf{p}][\mathbf{r}] \oplus \mathbf{m}' . 0 \quad \mid \quad s[\mathbf{q}][\mathbf{p}] \& \mathbf{m} . 0$$

By Rule T- $\oplus$, channel $s[\mathbf{p}]$ can implement one of the two choices and remain well-typed. If the implemented choice is the message towards role $\mathbf{r}$, then the resulting process is deadlocked even though the type context has an available communication action; violating session fidelity. Hence,

Theorem 3.20 relies on a *fidelity property* rather than *safety*. Note that, by the definition of a fidelitous context, both of the following environments are captured by the largest fidelity property.

$$\Gamma' = \left\{ s[\mathbf{p}] : \oplus \left\{ \begin{array}{l} \mathbf{q}:\mathbf{m}.\mathbf{end} \\ \mathbf{r}:\mathbf{m}'.\mathbf{end} \end{array} , \begin{array}{l} s[\mathbf{q}] : \&\mathbf{p}:\mathbf{m}.\mathbf{end}, \\ s[\mathbf{r}] : \&\mathbf{p}:\mathbf{m}'.\mathbf{end} \end{array} \right\} \quad \Gamma'' = \left\{ s[\mathbf{p}] : \oplus \left\{ \begin{array}{l} \mathbf{q}:\mathbf{m}.\mathbf{end} \\ \mathbf{r}:\mathbf{m}'.\mathbf{end} \end{array} , \begin{array}{l} s[\mathbf{q}] : \mathbf{end}, \\ s[\mathbf{r}] : \mathbf{end} \end{array} \right\} \right\}$$

This demonstrates the interesting qualities of the novel fidelity property, as it is strictly weaker than lock-freedom (and therefore liveness and termination), and can capture both deadlocked and deadlock-free protocols; all whilst being strictly stronger than safety.

The following now presents lemmas used together to infer information about processes from a type context governed by the session fidelity assumptions. Lemma 3.18 infers the shape of a type environment for an ensemble of processes that follow the session fidelity assumptions; whilst Lemma 3.19 (known as session inversion) determines the possible shapes of a process typed under contexts identified from the initial lemma.

LEMMA 3.18. Assume $\Gamma \vdash_D P$ with $\text{one-role}_!(P)$. Then $P \equiv \prod_{\mathbf{p} \in I} P_{\mathbf{p}}$ and $\Gamma = \Gamma_0(\cdot \Gamma'_{\mathbf{p}}, s[\mathbf{p}] : U_{\mathbf{p}})_{\mathbf{p} \in I'}$, where:

- (1) $\forall \mathbf{p} \in I' : \Gamma'_{\mathbf{p}}, s[\mathbf{p}] : U_{\mathbf{p}} \vdash_D P_{\mathbf{p}}$ and $c : U \notin \Gamma'_{\mathbf{p}}$;
- (2) $\forall \mathbf{q} \in I \setminus I' : P_{\mathbf{q}} \equiv \mathbf{0}$;
- (3) $c : U \notin \Gamma_0$.

PROOF. First observe that by Definition 3.15(2), $P \equiv \prod_{\mathbf{p} \in I} P_{\mathbf{p}}$ where $\forall \mathbf{p} \in I : P_{\mathbf{p}} \equiv \mathbf{0}$ or $P_{\mathbf{p}}$ only plays role $\mathbf{p}$. Therefore, by Rule T-, $\Gamma = (\cdot \Gamma_{\mathbf{p}})_{\mathbf{p} \in I}$ where $\forall \mathbf{p} \in I : \Gamma_{\mathbf{p}} \vdash_D P_{\mathbf{p}}$. For each parallel process, there are two cases to consider.

If $P_{\mathbf{p}} \equiv \mathbf{0}$, then by Rule T-0, $\text{end}(\Gamma_{\mathbf{p}})$.

Otherwise, $P_{\mathbf{p}}$ only plays role $\mathbf{p}$. Since $\Gamma_{\mathbf{p}} \vdash_D P_{\mathbf{p}}$, then $\Gamma_{\mathbf{p}}$ serves as witness for Definition 3.15(1). Hence, it can be concluded that $\Gamma_{\mathbf{p}} = \Gamma''_{\mathbf{p}}, s[\mathbf{p}] : U_{\mathbf{p}}$ with $U_{\mathbf{p}} \not\leq \mathbf{end}(\Gamma''_{\mathbf{p}})$.

Therefore, $\Gamma = (\cdot \Gamma''_{\mathbf{p}}, s[\mathbf{p}] : U_{\mathbf{p}})_{\mathbf{p} \in I'} (\cdot \Gamma_{\mathbf{q}})_{\mathbf{q} \in I \setminus I'}$ where $\forall \mathbf{p} \in I' : \Gamma''_{\mathbf{p}}, s[\mathbf{p}] : U_{\mathbf{p}} \vdash_D P_{\mathbf{p}}$ with $\text{end}(\Gamma''_{\mathbf{p}})$ and $\forall \mathbf{q} \in I \setminus I' : \Gamma_{\mathbf{q}} \vdash_D P_{\mathbf{q}} \equiv \mathbf{0}$ with $\text{end}(\Gamma_{\mathbf{q}})$.

Without loss of generality, assume that all end-typed channels are grouped in $I \setminus I''$. Hence, $\forall \mathbf{p} \in I'' : \Gamma''_{\mathbf{p}}, s[\mathbf{p}] : U_{\mathbf{p}} \vdash_D P_{\mathbf{p}}$ with $c : U \notin \Gamma''_{\mathbf{p}}$ and $\forall \mathbf{q} \in I \setminus I'' : \Gamma_{\mathbf{q}} \vdash_D P_{\mathbf{q}} \equiv \mathbf{0}$ with $\text{end}(\Gamma_{\mathbf{q}})$ and either $\Gamma_{\mathbf{q}} = \Gamma''_{\mathbf{q}}, s[\mathbf{q}] : \mathbf{end}$ or $c : U \notin \Gamma''_{\mathbf{q}}$.

Thus, $(\cdot \Gamma_{\mathbf{q}})_{\mathbf{q} \in I \setminus I''} = \Gamma_0(\cdot \Gamma_{\mathbf{q}})_{\mathbf{q} \in I'''} where $c : U \notin \Gamma_0$ and $I''' \subseteq I \setminus I''$.$

Let $I' = (I \setminus I'') \cup I'''$, then $\Gamma = \Gamma_0(\cdot \Gamma'_{\mathbf{p}}, s[\mathbf{p}] : U_{\mathbf{p}})_{\mathbf{p} \in I'}$. $\square$

The session inversion lemma (Lemma 3.19) can be used to infer further information about the contexts obtained from Lemma 3.18. Essentially, processes either directly mimic the action in the type, or they are a process call to a recursive definition that will then perform the action identified by the type. Different to a standard session inversion lemma, it is possible for the type to model a parallel composition of session types. This is shown to type a parallel composition of processes which obey the assumptions of the lemma, thus allowing the proposition to be inductively applied on the smaller processes.

LEMMA 3.19. Assume $\Gamma_0, s[\mathbf{p}] : U_{\mathbf{p}} \vdash_D P_{\mathbf{p}}$ with $c : U \notin \Gamma_0$ and either $P_{\mathbf{p}}$ only plays role $\mathbf{p}$ or $P_{\mathbf{p}} \equiv \mathbf{0}$. Then:

- (1) if $|S_1| \cdots |S_n| \leq U_{\mathbf{p}}$, then $P_{\mathbf{p}} \equiv P_1 | \cdots | P_n$ and $\forall i \in 1..n : \Gamma_0, s[\mathbf{p}] : S_i \vdash_D P_i$ and P_i only plays role $\mathbf{p}$;
- (2) if $!\&_{i \in I} \mathbf{p} : m_i(\vec{T}_i).S_i \leq U_{\mathbf{p}}$, then either:
 - (a) $P_{\mathbf{p}} \equiv !s[\mathbf{p}][\rho] \&_{i \in I'} m_i(\vec{b}_i).P'_i$ and $I \subseteq I'$; or
 - (b) $P_{\mathbf{p}} \equiv X\langle \vec{V} \rangle$ and $D(X) = (\vec{b}', \vec{T}', Q)$ and $Q\langle \vec{V}/\vec{b}' \rangle \equiv !s[\mathbf{p}][\rho] \&_{i \in I'} m_i(\vec{b}_i).P'_i$ and $I \subseteq I'$;

(3) if $\&_{i \in I} \mathbf{q} : m_i(\vec{T}_i).S_i \leq U_p$, then either:

(a) $P_p \equiv s[\mathbf{p}][\mathbf{q}] \&_{i \in I'} m_i(\vec{b}_i).P'_i$ and $I \subseteq I'$; or

(b) $P_p \equiv X\langle \vec{V} \rangle$ and $D(X) = (\vec{b}', \vec{T}', Q)$ and $Q\{\vec{V}/\vec{b}'\} \equiv s[\mathbf{p}][\mathbf{q}] \&_{i \in I'} m_i(\vec{b}_i).P'_i$ and $I \subseteq I'$;

(4) if $\oplus_{i \in I} \mathbf{q} : m_i(\vec{T}_i).S_i \leq U_p$, then either:

(a) $P_p \equiv \sum_{i \in I'} s[\mathbf{p}][\mathbf{q}_i] \oplus m_i(\vec{V}_i).P'_i$ and $I \supseteq I'$; or

(b) $P_p \equiv X\langle \vec{V} \rangle$ and $D(X) = (\vec{b}', \vec{T}', Q)$ and $Q\{\vec{V}/\vec{b}'\} \equiv \sum_{i \in I'} s[\mathbf{p}][\mathbf{q}_i] \oplus m_i(\vec{V}_i).P'_i$ and $I \supseteq I'$;

(5) if $\mathbf{end} \leq U_p$, then $P_p \equiv 0$.

PROOF. By case analysis on the 5 possible implications.

Case 1:

Follows from the definition of context splits (Figure 5), observing that since $c : U \notin \Gamma_0$ then $\Gamma_0, s[\mathbf{p}] : S_1 \cdots |S_n = \Gamma_0, s[\mathbf{p}] : S_1 \cdots \Gamma_0, s[\mathbf{p}] : S_n$.

Furthermore, by Rule T-, $\forall i \in 1..n : \Gamma_0, s[\mathbf{p}] : S_i \vdash_D P_i$.

Lastly, from the hypothesis it follows that D is guarded, $\text{fv}(P) = \emptyset$ and $U_p \not\leq \mathbf{end}$, thus $\forall i \in 1..n : \text{fv}(P_i) = \emptyset$ and $S_i \not\leq \mathbf{end}$. (Notice that any $\mathbf{end}$ in the parallel type can be removed via type congruence, and also reflected in the process.) Therefore, by Definition 3.15, $\forall i \in 1..n : P_i$ only plays role $\mathbf{p}$, by $\Gamma_0, s[\mathbf{p}] : S_i$.

Case 2:

From the hypothesis, and by subtyping and narrowing, $\Gamma_0, s[\mathbf{p}] : !\&_{i \in I} \mathbf{p} : m_i(\vec{T}_i).S_i \vdash_D P_p$. By inversion of typing rules, there are two possible rules that could be applied.

Rule T-!:

$\Gamma_0, s[\mathbf{p}] : !\&_{i \in I} \mathbf{p} : m_i(\vec{T}_i).S_i \vdash_D !s[\mathbf{p}][\mathbf{p}] \&_{i \in I} m_i(\vec{b}_i).P'_i$ (by T-!) (1)

$\Gamma_0, s[\mathbf{p}] : U_p \vdash_D !s[\mathbf{p}][\mathbf{p}] \&_{i \in I'} m_i(\vec{b}_i).P'_i$ where $I \subseteq I'$ (by (1) and Definition 2.3) (2)

$P_p \equiv !s[\mathbf{p}][\mathbf{p}] \&_{i \in I'} m_i(\vec{b}_i).P'_i$ (by (2) and subject congruence (Lemma A.12)) (3)

Rule T-X:

By inversion of T-X, $P_p \equiv X\langle \vec{V} \rangle$ and $\exists k \in 1..|\vec{V}| : V_k = s[\mathbf{p}]$ and $s[\mathbf{p}] : U_p \vdash V_k : T'_k$. Without loss of generality, assume $k = 1$, then $D(X) = ((x, \vec{b}), (T_k, \vec{T}'), Q)$.

Assuming that $\Gamma_0, s[\mathbf{p}] : U_p \vdash_D P_p$ is derived from a well-typed program, then it follows that $x : T'_k, \{\vec{b} : \vec{T}'\} \vdash_D Q$ by Rule T- $\mathcal{P}$. Furthermore, by Definition 3.14, it follows that Q cannot be typed by a further application of Rule T-X, and thus must instead be typed by Rule T-!. Following the same reasoning as outlined in the previous subcase, it must be that $Q \equiv !x[\mathbf{p}'] \&_{i \in I'} m_i(\vec{b}_i).P'_i$ and $I \subseteq I'$. Lastly, it follows that either $\mathbf{p}' = \mathbf{p}$, or $\exists k' \in 1..|\vec{V}| : V_{k'} = \mathbf{p}$ and $b_{k'} = \mathbf{p}'$. Either way, it results that $Q\{\vec{V}/\vec{b}'\} \equiv !s[\mathbf{p}][\mathbf{p}] \&_{i \in I'} m_i(\vec{b}_i).P'_i$.

Case 3, 4: Similar to Case 2.

Case 5:

Since $\mathbf{end} \leq U_p$ and $c : U \notin \Gamma_0$, then $\text{end}(\Gamma_0, s[\mathbf{p}] : U_p)$. By inversion of the typing rules, the only possible shape for P_p is $P_p \equiv 0$ via Rule T-0. $\square$

THEOREM 3.20 (SESSION FIDELITY). Assume $\Gamma \vdash_D P$ with $\text{fid}_!(\Gamma)$ and $\text{one-role}_!(P)$. Then, $\Gamma \rightarrow$ implies $\exists \Gamma', P'$ s.t.:

(i) $\Gamma \rightarrow \Gamma'$;

(ii) $P \rightarrow_D^* P'$;

(iii) $\Gamma' \vdash_D P'$ with $\text{fid}_!(\Gamma')$; and

(iv) $\text{one-role}(P')$.

PROOF. By induction on the derivation of $\Gamma \rightarrow$. There are two possible cases by which Γ can be enabled with a reduction: by Rule $\Gamma\text{-Com}_1$ or by Rule $\Gamma\text{-Com}_2$. The following elaborates on the latter case (the other case is similar).

The assumptions are: (A1) $\Gamma \vdash_D P$; (A2) $\text{fid}_!(\Gamma)$; (A3) $\text{one-role}_!(P)$; and (A4) $\Gamma \rightarrow$. The proof begins by inferring some structure on both the types and processes, utilizing Lemma 3.18.

$$P \equiv \big|_{p \in I} P_p \text{ and } \Gamma = \Gamma_0(\cdot \Gamma'_p, s[p] : U_p)_{p \in I'} \quad (\text{by (A1), (A3), and Lemma 3.18}) \quad (1)$$

$$\forall p \in I' : \Gamma'_p, s[p] : U_p \vdash_D P_p \text{ and } c : U \notin \Gamma'_p \quad (\text{by (A1), (A3), and Lemma 3.18}) \quad (2)$$

$$\forall q \in I \setminus I' : P_q \equiv 0 \quad (\text{by (A1), (A3), and Lemma 3.18}) \quad (3)$$

$$c : U \notin \Gamma_0 \quad (\text{by (A1), (A3), and Lemma 3.18}) \quad (4)$$

Since the context is known to be enabled with a reduction (by assumption), the actions emitted by the context can be inferred from Rule $\Gamma\text{-Com}_2$. Specifically, $\exists p, q \in I'$ such that:

$$\Gamma'_p, s[p] : U_p \xrightarrow{s:p \oplus q : m(\vec{r}, \vec{T}_1)} \Gamma'_p, s[p] : U'_p \quad (\text{by (1), (A4) and } \Gamma\text{-Com}_2) \quad (5)$$

$$\Gamma'_q, s[q] : U_q \xrightarrow{s:q \& \alpha' : m(\vec{\alpha}, \vec{T}_2)} \Gamma'_q, s[q] : U'_q \quad (\text{by (1), (A4) and } \Gamma\text{-Com}_2) \quad (6)$$

$$\vec{T}_1 \leq \vec{T}_2 \quad (\text{by (1), (A4) and } \Gamma\text{-Com}_2) \quad (7)$$

$$(\Gamma'_p, s[p] : U_p) \cdot (\Gamma'_q, s[q] : U_q) \xrightarrow{s:p, q : m} (\Gamma'_p, s[p] : U'_p) + (\Gamma'_q, s[q] : U'_q \{ \vec{r} / \vec{\alpha} \} \{ p / \alpha' \}) \quad (\text{by (5), (6), (7), and } \Gamma\text{-Com}_2) \quad (8)$$

From these actions, the shapes of the types can be outlined, following from the type semantics in Figure 7.

$$\oplus_{j \in J} q_j : m_j(\vec{T}_j).S_j \leq U_p \leq q \oplus m(\vec{r}, \vec{T}_1).S \quad (\text{by (5), } \Gamma\text{-}\oplus) \quad (9)$$

$$\exists l \in J : q_l = q \wedge m_l = m \wedge \vec{T}_l \geq \vec{T}_1 \quad (\text{by (9), Definition 2.3}) \quad (10)$$

$$! \&_{k \in K} \alpha' : m'_k(\vec{\delta}_k, \vec{T}'_k).S'_k \leq U_q \text{ and } \exists l \in K : m'_l = m \quad (\text{by (6), } \Gamma\text{-}!) \quad (11)$$

$$\vec{T}_l \leq \vec{T}_2 \quad (\text{by (11), (6), Definition 2.3}) \quad (12)$$

The shapes of processes can now be inferred from the types, using (2), (9), Lemma 3.19 for P_p , and (2), (11), Lemma 3.19 for P_q . Note that by the session inversion lemma, there are two possible shapes for each process; thus a total of four possible permutations to consider, resulting in 4 subcases.

Case $\oplus!$: Consider:

$$P_p \equiv \sum_{j \in J'} s[p][q_j] \oplus m_j(\vec{V}_j).P'_j \text{ and } J \supseteq J' \quad (\text{by case assumption}) \quad (13)$$

$$P_q \equiv !s[q][\alpha'] \&_{k \in K'} m'_k(\vec{b}_k).P''_k \text{ and } K \subseteq K' \quad (\text{by case assumption}) \quad (14)$$

This is where the novel fidelitous property comes into play. As it stands, there is no guarantee that $\exists x \in J' : q_x = q$, as the implementation is a subset of the type and could hence implement a part of the selection that does not include the role considered for communication in (8). However, by (8), it is guaranteed that at least one (type-level) communication is possible. Then by (A2) and Table 1, it follows that communication should be eventually possible for every path in J , and thus also J' (since $J' \subseteq J$ by (13)). Therefore, it can be assumed without loss of generality that the nondeterministic choice in P_p reduces to a path x in J' such that $q_x = q$.

$$P_p \rightarrow P'_p \equiv s[p][q] \oplus m_x(\vec{V}_x).P'_x \text{ and } x \in J' \quad (\text{explanation above}) \quad (15)$$

$$\exists y \in K : m_y = m_x \quad (\text{by (15), (A2), Definition 3.16}) \quad (16)$$

$$y \in K' \quad (\text{by (16) and (14)}) \quad (17)$$

$$P'_p \mid P_q \rightarrow P'_x \mid P_q \mid P''_y \{ p / \alpha' \} \{ \vec{V}_x / \vec{b}_y \} \quad (\text{by (14), (15), (17), R-!C}_2) \quad (18)$$

$$\Gamma'_p, s[p] : U_p \xrightarrow{s:p \oplus q : m_x(\vec{r}, \vec{T}_x)} \Gamma'_p, s[p] : U'_p \quad (\text{by (2), (15), } \Gamma\text{-}\oplus) \quad (19)$$

$$(\Gamma'_p, s[p] : U_p) \cdot (\Gamma'_q, s[q] : U_q) \xrightarrow{s:p,q:m_x} (\Gamma'_p, s[p] : U'_p) + (\Gamma'_q, s[q] : U'_q \{ \vec{r}' / \vec{\delta} \} \{ p / \alpha' \})$$

(by (19), (8), (A2)) (20)

$$\Gamma'_q, s[q] : U_q \xrightarrow{s:q \oplus \alpha' : m_x(\vec{\delta}, \vec{T}_x)} \Gamma'_q, s[p] : U'_q$$

(by (20), (19), Γ -Com₂) (21)

$$\vec{T}_x \leq \vec{T}'_x$$

(by (21), (20), (19), Γ -Com₂) (22)

The subcase is concluded by observing that the reduced contexts type the reduced processes, and that the reduced processes still adhere to the one-role_! requirements.

$$\Gamma'_p, s[p] : U'_p \vdash_D P'_x$$

(by (2), (19), T- $\oplus$) (23)

$$\Gamma'_q, s[p] : U'_q \{ p / \alpha' \} \{ \vec{r}' / \vec{\delta} \} \vdash_D P''_y \{ p / \alpha' \} \{ \vec{V}_x / \vec{b}_y \}$$

(by (2), (21), (22), (23), T-!, and both substitution lemmas) (24)

Note that for the above, all the types for payloads are present in Γ'_q since by the session fidelity assumptions, payloads can only contain non-session-typed entities; these are copied through context splits (Figure 5).

$$P \rightarrow^* P' \mid P'_x \mid P_q \mid P''_y \{ p / \alpha' \} \{ \vec{V}_x / \vec{b}_y \} \text{ with one-role}_!(P') \quad (\text{by (A3), (15), (18)}) \quad (25)$$

$$\text{one-role}_!(P'_x \mid P_q \mid P''_y \{ p / \alpha' \} \{ \vec{V}_x / \vec{b}_y \}) \quad (\text{by (1), (23), (24), Definition 3.15}) \quad (26)$$

$$\Gamma \rightarrow \Gamma' \quad (\text{by (20)}) \quad (27)$$

$$P \rightarrow^* P' \mid P'_x \mid P_q \mid P''_y \{ p / \alpha' \} \{ \vec{V}_x / \vec{b}_y \} \quad (\text{by (25)}) \quad (28)$$

$$\Gamma' \vdash_D P' \mid P'_x \mid P_q \mid P''_y \{ p / \alpha' \} \{ \vec{V}_x / \vec{b}_y \} \quad (\text{by (2), (20), (23), (24), T-|}) \quad (29)$$

$$\text{fid}_!(\Gamma') \quad (\text{by (A2), (27), } \varphi \rightarrow) \quad (30)$$

$$\text{one-role}_!(P' \mid P'_x \mid P_q \mid P''_y \{ p / \alpha' \} \{ \vec{V}_x / \vec{b}_y \}) \quad (\text{by (25), (26)}) \quad (31)$$

$$\text{Case } \oplus! \text{ holds.} \quad (\text{by (25), (27)–(31)}) \quad (32)$$

The remaining subcases all follow similar reasoning, except with the additional step of allowing for process reduction via Rule R-X before communication becomes available. $\square$

3.5 Behavioural Properties

A benefit of integrating replication and first-class roles into a generalised MPST calculus is to verify behavioural properties on client-server systems where finite clients may communicate with infinitely available servers. The definitions of certain properties for such systems need to be relaxed from what is considered standard. Definition 3.21 redefines behavioural properties of MPST! processes, allowing for the infinite nature of replication.

Definition 3.21 (Process properties). A process P is said to be:

deadlock-free (df) iff $P \rightarrow_D^* P' \not\rightarrow_D$ implies either

- (1) $P' \equiv 0$; or
- (2) $P' \equiv \prod_{i \in I} !P''_i$;

terminating (term) iff $P \rightarrow_D^* P'$ implies $\exists n$ finite s.t. $P' \rightarrow_D^n P''$ and either

- (1) $P'' \equiv 0$; or
- (2) $P'' \equiv \prod_{i \in I} !P'''_i$;

never-terminating (n-term) iff $P \rightarrow_D^* P'$ implies $P' \rightarrow_D$;

lock-free (lf) iff $P \rightarrow_D^* P'$ implies:

- (1) if $P' \equiv c[q] \oplus m(\vec{V}).Q \mid Q'$, then $P' \rightarrow_D P'' \equiv Q \mid Q''$; and
- (2) if $P' \equiv c[q] \&_{i \in I} m_i(\vec{b}_i).Q_i \mid Q'$, then $\exists k \in I, \vec{V} : P' \rightarrow_D Q_k \{ \vec{V} / b_k \} \mid Q''$;

live (live) iff $P \rightarrow_D^* P'$ implies $\exists n$ finite s.t.:

- (1) if $P' \equiv c[q] \oplus m(\vec{V}).Q \mid Q'$ and $P' \rightarrow_D^n P'_n$, then $\exists j \leq n : P'_j \rightarrow_D P'' \equiv Q \mid Q''$; and

(2) if $P' \equiv c[\mathbf{q}] \ \&_{i \in I} m_i(\vec{b}_i) \cdot Q_i \mid Q'$ and $P' \rightarrow_D^n P'_n$, then $\exists j \leq n, k \in I, \vec{V} : P'_j \rightarrow_D P'' \equiv Q_k\{\vec{V}/\vec{b}_k\} \mid Q''$.

A *deadlock-free* process is one which only stops reducing when it reaches $\mathbf{0}$, or when it only contains replicated servers (since they are designed to remain infinitely available). *Terminating* processes reach $\mathbf{0}$, or a state in which only replicated processes remain, within a finite number of reduction steps. *Never-terminating* processes can always reduce, *ad infinitum*. *Lock-free* processes can always reduce to the continuations of their outputs/non-replicated inputs; and *live* processes will always reduce to the continuations of their outputs/non-replicated inputs, within a finite number of steps. It would not make sense to impose the eventual communication requirement on replicated servers, as it is intentional that they may expect messages that may possibly never arrive (e.g. after servicing all client requests).

THEOREM 3.22 (PROPERTY VERIFICATION). *If $\Gamma \vdash_D P$ with $\text{fid}_l(\Gamma)$ and $\text{one-role}_l(P)$, then $\phi(\Gamma) \implies \phi(P)$ for $\phi \in \{\text{deadlock-free, terminating, never-terminating, lock-free, live}\}$.*

PROOF. Observe that by Theorem 3.20, Γ simulates the *at most* behaviour of process P . Therefore, the trace observed at runtime for a program is guaranteed to be simulated by a path on the type context. Hence, if the sequences of actions for all paths of a type context adhere to some property, then that property holds for processes typed under that context. Thus, to prove this theorem, what is required is to show that the definitions for type-level properties soundly capture the process-level equivalents. This is demonstrated for deadlock-free processes below (other properties follow similar reasoning).

Case df: RTP (under the theorem's assumptions) $\text{df}(\Gamma)$ implies $\text{df}(P)$.

Note that if $\Gamma \rightarrow$, then $P \rightarrow$, hence P would be trivially deadlock-free. Thus, consider $\Gamma \not\rightarrow$. In this case, by Table 1, it can be concluded that $\text{end}!(\Gamma)$. Then, by Definition 3.7 and session inversion (Lemma 3.19), either $P \equiv \mathbf{0}$, or $P \equiv \mid_{i \in I} !P'_i$, or $P \rightarrow^* P'' \equiv \mid_{i \in I} !P''_i$ via a finite series of applications of **R-X**. (It is guaranteed that the number of reductions via **R-X** is finite by Definition 3.14 and since $\text{one-role}_l(P)$ by assumption.) $\square$

4 Expressivity and Decidability

This section discusses, and shows by example, the benefits and limitations of MPST!. Section 4.1 focuses on demonstrating the expressivity gained by using replication and first-class roles in MPST, whilst Section 4.2 discusses decidability.

4.1 Expressivity

To begin, the following demonstrates a common design pattern—a *service*—used for describing protocols. A number of services are built to showcase language features, and to describe protocols which were previously untypable in any MPST theory. Specifically, using the increased expressiveness of replication and first-class roles, protocols are defined for communicating serialised *binary trees*, the *dining philosophers problem*, and an *auction*. Importantly, all examples shown adhere to the decidability requirements discussed later (cf. Section 4.2).

4.1.1 Services. A *service* is a building-block of a protocol, involving some universal receive, with the aim of *offering* a specific interaction. A *client* interacts with a service to achieve the communication pattern it offers. Importantly, services may be clients of other services, promoting a modular design of protocols in MPST! and adhering to the client-server paradigm [29] mentioned in Section 1.

Example 4.1 (Ping). The *ping service* simply responds to a ping with a pong.

$P : !\alpha \& \text{ping} . \alpha \oplus \text{pong} . \text{end}$

A basic yet useful service is given in Example 4.1, where role P offers a *ping* service. As a convention, capitals are used for naming services. It is key to note the use of both replication and free role names in types to be able to design modular components of a protocol—both are integral to designing a sub-protocol agnostic of the larger scope in which it is used. The following showcases how a service can be used as a building-block for larger protocols, and also serves as witness for the increased expressiveness gained in MPST!.

4.1.2 Context-Free MPST. Context-free session types [1, 22, 31] are a formalism that replace prefix-style session types with individual actions, along with a sequencing operator $;$ with neutral element skip. The goal of this line of work is to express communication protocols that are not possible under tail-recursive session types, given their restriction to regular languages. The classic example is that of communicating a serialised binary tree.

Example 4.2 (Binary tree in standard context-free STs). Consider a binary tree data type described by the following context-free grammar.

$$\text{tree} ::= (\text{node}, \text{tree}, \text{tree}) \mid \text{leaf}$$

An attempt at representing this data type in a session protocol follows:

$$\mu t. \oplus \{ \text{leaf} : \text{skip}, \text{node} : t \}$$

However, this type is not precise enough—it does not guarantee that the correct structure of a binary tree is maintained. Work on context-free session types solves this by proposing type systems allowing the following specification:

$$\mu t. \oplus \{ \text{leaf} : \text{skip}, \text{node} : \text{skip} ; t ; t \}$$

Selecting the *node* label instead guarantees that the type requires the program to send *two* sub-trees.

The parallel types presented in Section 2.3, although not exposed directly to users, lift expressiveness of types in MPST!. In fact, since replicated branches are *permanently available* (by composing continuation types in parallel), the sequencing operator $;$ can be simulated using type-level parallel composition.

Example 4.3 (Binary Tree Service). Recall the ping service P from Example 4.1. A *binary tree service* T is built using P as shown below:

$$T : !\beta \& \text{tree}. P \oplus \text{ping}. !P \& \text{pong}. \beta \& \{ \text{leaf}. \text{end}, \text{node}. P \oplus \text{ping}. P \oplus \text{ping}. \text{end} \}$$

The service begins by receiving a request for a *tree* from a client. It then sends a *ping* to the ping service, exposing a replicated branch waiting to receive the *pong* reply. The client is now free to build the binary tree. It is key to note that any *node* sent to the service will subsequently forward *two* *ping* requests to P . In turn, this communication will pull out two copies of the type continuation $\beta \& \{ \text{leaf}. \text{end}, \text{node}. P \oplus \text{ping}. P \oplus \text{ping}. \text{end} \}$, forcing the client to maintain the appropriate binary tree structure. For example, if a client t wishes to build a tree consisting of one root node and two leaf nodes, its type would be defined as:

$$t : T \oplus \text{tree}. T \oplus \text{node}. T \oplus \text{leaf}. T \oplus \text{leaf}. \text{end}$$

The metatheoretic framework can be used to determine that any protocol failing to abide by the binary tree structure will result in a *deadlock*; whilst any faithful protocol that obeys the correct structure is *terminating*, e.g. the protocol $\{t, T, P\}$.

An obscure limitation of the tree service in Example 4.3 is that it can only be used by a single client. Consider, for example, two separate clients sending a *node* message to T . Since both tree service types communicate with P to unroll the replicated branch $!P \& \text{pong}$, the protocol becomes

non-deterministic in a non-confluent manner and can result in deadlocked behaviour. To resolve this issue, the tree service is amended to accept a payload role acting as a personal ping service for the client; this guarantees that the tree type is only unrolled by the client that made the initial request.

Example 4.4 (Multi-Client Binary Tree). The binary tree service is now redesigned, this time capable of concurrently building multiple trees for different clients. The key difference being that the new service, M , accepts a role as a payload on the initial request to which it will issue its pings.

$$M : !\alpha \& \text{tree}(\beta) . \beta \oplus \text{ping} . !\beta \& \text{pong} . \alpha \& \{ \text{leaf} . \text{end}, \text{node} . \beta \oplus \text{ping} . \beta \oplus \text{ping} . \text{end} \}$$

$$S_p = !M \& \text{ping} . M \oplus \text{pong} . \text{end}$$

Multiple clients can now issue concurrent requests to the tree service whilst maintaining safety and fidelity. A sample (terminating) protocol is that of $\{t_1, t_2, p_1, p_2, M\}$, where $p_1, p_2 : S_p$, and the types for t_1, t_2 are given by:

$$t_1 : M \oplus \text{tree}(p_1) . M \oplus \text{node} . M \oplus \text{leaf} . M \oplus \text{leaf} . \text{end}$$

$$t_2 : M \oplus \text{tree}(p_2) . M \oplus \text{node} . M \oplus \text{leaf} . M \oplus \text{node} . M \oplus \text{leaf} . M \oplus \text{leaf} . \text{end}$$

4.1.3 Replication vs. Recursion. As demonstrated, replication and parallel composition increase the expressive power of MPST beyond that of tail-recursion. Naturally, one might ask, “*is recursion still needed?*” Interestingly, this study finds replication and recursion in MPST to be mutually non-inclusive—i.e., both can produce protocols which *cannot* be typed under the other construct. This has already been demonstrated in one direction with the binary tree examples; the following showcases how recursion cannot be replaced by replication.

Example 4.5 (Lock Service). The lock service provides clients with a *mutex lock*.

$$L : !\theta \& \text{lock} . \mu t . \theta \& \{ \text{acquire} . \theta \& \text{release} . t, \text{done} . \text{end} \}$$

When a client requests a lock from L , a copy of the recursive continuation is exposed. The recursive definition allows sequences of *acquire* and *release* messages to be received. It is key to note that, whilst replication maintains a top-level branch that is permanently available to receive a message, the top-level action in a recursive definition is *not* fixed.

Copies of the continuation type of a replicated receive are executed concurrently. Example 4.5 provides a service for roles to enter race-sensitive portions of a protocol, as if it were an atomic action. This can be used to type the dining philosophers problem.

Example 4.6 (Dining Philosophers). A number of philosophers gather to eat on a round table. Each plate is separated by a single chopstick, and a philosopher needs two chopsticks to eat. The dining philosophers problem requires the philosophers to employ an algorithm to ensure the table does not get deadlocked. It is possible to view *chopsticks as services* and *philosophers as clients*. The types for n chopsticks are given by:

$$(C_i)_{i \in 1..n} : L \oplus \text{lock} . !\eta \& \left\{ \begin{array}{l} \text{take?} . L \oplus \text{acquire} . \eta \oplus \text{ok} . \eta \& \text{give} . L \oplus \text{release} . \text{end} \\ \text{done} . L \oplus \text{done} . \text{end} \end{array} \right\}$$

Before offering its service, a chopstick requests a lock from L . This ensures that every chopstick has its own lock that it may *acquire* and *release*. The lock is used to guarantee that a chopstick is only ever taken by a single philosopher at a time. A chopstick then waits for a *take?* request from a philosopher; receiving one will result in it attempting to *acquire* the lock. This acquisition is only successful if the same role has not already requested it in some other parallel composition. If the lock was already acquired, then the $L \oplus \text{acquire}$ will block until the lock is released. Acquiring the

lock sends an **ok** back to the philosopher, symbolising that they have successfully obtained the chopstick. When done from eating, the philosopher may then send back a **give** message, which in turn releases the lock, as the chopstick is now available to be taken by a different philosopher. The naïve algorithm for n philosophers may be described as follows:

$$(p_i)_{i \in 1..n} : C_i \oplus \text{take?}. C_{i+1} \oplus \text{take?}. C_i \& \text{ok}. C_{i+1} \& \text{ok}. C_i \oplus \text{give}. C_{i+1} \oplus \text{give}. q \oplus \text{fin}. \text{end}$$

$$q : p_1 \& \text{fin}. \dots p_n \& \text{fin}. C_1 \oplus \text{done}. \dots C_n \oplus \text{done}. \text{end}$$

Every philosopher p_i has a similar type. They begin by requesting to take the chopsticks to their left and right—note that this results in every chopstick receiving two **take?** requests. Receiving both **ok** messages means the philosopher can eat, and subsequently **give** back the chopsticks. Finally, when finished, philosophers send a **fin** to role q , acting as a clean-up for the protocol. The protocol $\{p_i, q, C_i, L\}_{i \in 1..n}$ is *fidelitous*, but fails typechecking for $\varphi = \text{terminating}$. In fact, the naïve protocol allows for the scenarios in which all philosophers take a single chopstick, resulting in a deadlock. This problem has many solutions; the following presents the simplest, in which philosophers take turns to eat. (Key changes are underlined.)

$$S_1 = C_1 \oplus \text{take?}. C_2 \oplus \text{take?}. C_1 \& \text{ok}. C_2 \& \text{ok}. C_1 \oplus \text{give}. C_2 \oplus \text{give}. \underline{p_2 \oplus \text{fin}. \text{end}}$$

$$S_2 = \underline{p_{i-1} \& \text{fin}. C_i \oplus \text{take?}. C_{i+1} \oplus \text{take?}. C_i \& \text{ok}. C_{i+1} \& \text{ok}. C_i \oplus \text{give}. C_{i+1} \oplus \text{give}. \underline{p_{i+1} \oplus \text{fin}. \text{end}}}$$

$$S_3 = \underline{p_{n-1} \& \text{fin}. C_n \oplus \text{take?}. C_1 \oplus \text{take?}. C_n \& \text{ok}. C_1 \& \text{ok}. C_n \oplus \text{give}. C_1 \oplus \text{give}. q \oplus \text{fin}. \text{end}}$$

$$q' : p_n \& \text{fin}. C_1 \oplus \text{done}. \dots C_n \oplus \text{done}. \text{end}$$

Here, all philosophers other than the first must wait for the previous to **finish** eating before they can request to take their chopsticks. The updated protocol $\{p_1 : S_1, p_i : S_2, p_n : S_3, q', C_j, L\}_{j \in 2..n-1}^{i \in 2..n-1}$ now typechecks for $\varphi = \text{terminating}$.

The previous examples demonstrate how recursion hidden by a universal receive can be used to mimic changes in state. The final example does the inverse, i.e., a universal receive hidden by a recursive binder can be used to model resources which eventually reach some permanent state. In addition, the example demonstrates how universal receives model *fair races*, since they do not impose an order on how communication is handled.

Example 4.7 (Auction). A merchant m sets up an auction A to accept bids from some buyers b . A merchant can employ different mechanisms for choosing who to sell to (e.g., first come first served, highest bid, biased selling, etc.); but must always respond to buyers with either a **yes**, **no**, or **not-avail** message.

$$A : !\alpha \& \text{bid}(\text{Int}). m \oplus \text{bid}(\text{Int}, \alpha). \text{end}$$

$$m : \mu t. A \& \text{bid}(\text{Int}, \beta). \beta \oplus \left\{ \begin{array}{l} \text{yes}. !A \& \text{bid}(\text{Int}, \kappa). \kappa \oplus \text{not-avail}. \text{end} \\ \text{no}. t \end{array} \right\}$$

$$(b_i)_{i \in 1..n} : \mu t. A \oplus \text{bid}(\text{Int}). m \& \{ \text{yes}. \text{end}, \text{no}. t, \text{not-avail}. \text{end} \}$$

Buyers b_i race to send **bids** to the auction service. In turn, the auction forwards bids and buyer role identifiers to the merchant, who processes bids sequentially (but still in an arbitrary order). If the merchant declines a bid, then the client is offered another chance; if the merchant accepts a bid, then it exposes a replicated receive which informs any further buyers that the product is no longer available. It is key to note that, unlike in Example 4.6 where locks are used to avoid race conditions, races here are not only allowed but are integral to the protocol. Additionally, by uncovering a replicated receive, the merchant enters a *permanent* state. These two characteristics guarantee that, no matter the selling algorithm employed by the merchant: (i) bids always arrive in a fair arbitrary order; and (ii) the product can only be sold once.

4.1.4 *Discussion.* Replication and recursion have been shown to be mutually non-inclusive in MPST, and that the studied extension increases the expressiveness of the type system. It is important to understand the causal dependencies between added features and added expressiveness.

Replication alone is enough to increase the expressiveness of MPSTs w.r.t. the Chomsky hierarchy. Note that, e.g., Example 4.3 could be easily re-written without the universal receive, especially since it should not be used by multiple clients to uphold deadlock-freedom—thus, replication in MPSTs increases their expressiveness to (at least) that of context-free languages.

“First-class roles” in the formalism refers to: (i) *universal receives* acting as binders on role variables; and (ii) the ability to *pass roles* as payloads in messages. Universal receives allow protocols to be designed agnostic of the client pool (consider Examples 1.3, 4.1, 4.4 and 4.5); and also act as a fair way of introducing races—e.g. Example 1.3 describes a load balancer that responds to requests in no particular order. Role passing allows for safe distributed choice. In a load balancer (in general), it is impossible for a client to know which worker will service its request. In Example 1.3, role passing allows the server to inform clients of its choice, and also informs the worker of the identity of the client. Role passing increases expressiveness by introducing dependencies into a protocol. For instance, Example 4.7 uses role passing to ensure the merchant correctly services the right buyers; without it, a merchant could not respond to requests without bias.

As a final note, first-class roles are different to, e.g., *delegation* or *multiple sessions* (both supported in MPST!) since they act *inside* a session. Therefore these constructs can be used whilst abiding by the session fidelity assumptions, and thus the generalised framework can still be used to verify runtime properties. This not possible with interleaved sessions without other mechanisms such as an interaction typing system [2] or priorities [6].

4.2 Decidability

As may be expected, the added expressiveness of replication and first-class roles into types does not come without a cost. In fact, the type semantics for MPST! cannot be simulated by a finite state machine, and so decidability results are not obvious. The remainder of this section discusses decidability of typechecking in detail. Typechecking is shown to be *only as decidable as the safety property*; examples of problematic types are given; decidable sublanguages of MPST! are identified; and strategies for restricting protocols to these decidable sublanguages are given.

THEOREM 4.8 (DECIDABILITY OF TYPE CHECKING). *If φ is decidable, then typechecking is decidable.*

PROOF. Since typing rules in Figure 6 can be deterministically applied based on the structure of a process P , and a typing context need only be split a finite number of times to separate all linear types, there are a finite number of contexts that can be tried for each rule that requires a context split. Lastly, subtyping is decidable [10] (decidability of subtyping replicated types is equivalent to regular branching types, and of parallel types is equivalent to checking multiple session types); and φ is decidable by assumption. $\square$

Theorem 4.8 states that decidability of type checking is only as decidable as property φ . Naturally, the next step is thus to determine the decidability of φ . Example 4.11, will demonstrate why φ may not necessarily be decidable in general for the type semantics presented in Figure 7. To do this, the following first defines *behavioural sets* of type contexts (as in [27, Appendix K]).

Definition 4.9 (Behavioural set). The behavioural set of a type context, written $\text{beh}(\Gamma)$, is given by $\text{beh}(\Gamma) = \text{unf}^*(\{\Gamma' \mid \Gamma \rightarrow^* \Gamma'\})$; where unf^* is the closure of unf —a function that unfolds all top-level recursive binders in a set of contexts.

Informally, the behavioural set of a context Γ is the set of (i) its reductions; and (ii) its reductions’ unfoldings. The benefit of beh is that it mechanically abides by conditions $\varphi\text{-}\mu$ and $\varphi\text{-}\rightarrow$ from

Definition 3.12. Condition $\varphi\text{-}\rho$ can be checked by traversing through the syntax of the (finite) type context, as per Figure 8. Therefore, to determine whether $\text{beh}(\Gamma)$ is a safety property, all that remains is to exhaustively check the contexts that inhabit $\text{beh}(\Gamma)$ against the remaining safety conditions.

Example 4.10. Consider a context $\Gamma = \{s[p] : \mu t.q \oplus m.t, s[q] : \mu t'.p \& m.t'\}$. The behavioural set of Γ is given by:

$$\text{beh}(\Gamma) = \left\{ \left\{ \begin{array}{l} s[p] : \mu t.q \oplus m.t, \\ s[q] : \mu t'.p \& m.t' \end{array} \right\}, \left\{ \begin{array}{l} s[p] : q \oplus m.\mu t.q \oplus m.t, \\ s[q] : p \& m.\mu t'.p \& m.t' \end{array} \right\} \right\}$$

The left element is the original context after 0 reduction steps, whereas the right element is the unfolding of Γ . Moreover, any further reductions only yield contexts (and unfoldings) already captured by these two elements.

The next example context, however, is problematic for typechecking.

Example 4.11. Consider a context $\Gamma = \{s[p] : \mu t.q \oplus m.t, s[q] : !p \& m.r \oplus m\}$. The behavioural set of Γ is given by:

$$\text{beh}(\Gamma) = \left\{ \left\{ \begin{array}{l} s[p] : \mu t.q \oplus m.t, \\ s[q] : !p \& m.r \oplus m \end{array} \right\}, \left\{ \begin{array}{l} s[p] : q \oplus m.\mu t.q \oplus m.t, \\ s[q] : !p \& m.r \oplus m \end{array} \right\}, \right. \\ \left. \left\{ \begin{array}{l} s[p] : \mu t.q \oplus m.t, \\ s[q] : !p \& m.r \oplus m \mid r \oplus m \end{array} \right\}, \left\{ \begin{array}{l} s[p] : q \oplus m.\mu t.q \oplus m.t, \\ s[q] : !p \& m.r \oplus m \mid r \oplus m \end{array} \right\}, \right. \\ \left. \left\{ \begin{array}{l} s[p] : \mu t.q \oplus m.t, \\ s[q] : !p \& m.r \oplus m \mid r \oplus m \mid r \oplus m \end{array} \right\}, \dots \right\}$$

Indeed, $\text{beh}(\Gamma)$ is *infinite*. This is a result of how replication and parallel composition are modelled in Figure 7. In fact, the type semantics for replicated communication allows for context reduction to yield *larger* types. Note how in this example, the contexts that inhabit $\text{beh}(\Gamma)$ get infinitely larger by pulling out infinitely many copies of type $r \oplus m$.

Furthermore, infinite behavioural sets are not only a result of recursive communication with replicated branches. Consider, e.g.:

$$\Gamma' = \{s[p] : !\alpha \& m.\alpha \oplus m'.r \oplus m, s[q] : p \oplus m.!\beta \& m'.\beta \oplus m\}$$

Such a context will also pull out infinitely many copies of type $r \oplus m$, because the replicated communication forms an infinite loop. Lastly, it is key to note that $\text{beh}(\Gamma')$ is finite for any Γ'' that does not contain replicated branches, since there is no other way for a context reduction to yield a larger type.

Knowing whether $\text{beh}(\Gamma)$ is (in-)finite is key for the main decidability result.

THEOREM 4.12 (DECIDABILITY OF BEH). *Let $\varphi = \text{beh}(\Gamma)$. If $\text{beh}(\Gamma)$ is finite, then φ is decidable.*

PROOF. Since $\text{beh}(\Gamma)$ contains all possible reducts and unfoldings of Γ , then conditions $\varphi \rightarrow$ and $\varphi\text{-}\mu$ are satisfied immediately. Therefore, to determine whether $\text{beh}(\Gamma)$ is a safety property, all inhabitants of $\text{beh}(\Gamma)$ may be exhaustively checked against conditions $\varphi\text{-}S$ and $\varphi\text{-}\rho$, which is decidable since $\text{beh}(\Gamma)$ is finite (by assumption); and since subtyping and frv are decidable. $\square$

Theorem 4.12 states that φ is *decidable* for any $\varphi = \text{beh}(\Gamma)$ where $\text{beh}(\Gamma)$ is a finite set. In other words, if a protocol can be shown to have a finite behavioural set, then typechecking for that protocol is *decidable*. This could be done manually for each protocol; however, to further increase the practicality of the type system, two strategies for restricting protocols into a subset of MPST! with finite behavioural sets are presented.

4.2.1 Decidability Strategies. By theorems 4.8 and 4.12, typechecking is decidable for the subset of MPST! typed under a context Γ for which $\text{beh}(\Gamma)$ is finite. Hence, the strategies presented determine sound (but not complete) properties guaranteed to restrict types to this decidable subset.

Definition 4.13 prevents types like those in Example 4.11 using a naïve approach; $\text{tf}(\Gamma)$ captures protocols where all clients of a replicated server are non-recursive and non-replicated.

Definition 4.13 (Trivially finite). A context Γ is trivially finite, $\text{tf}(\Gamma)$, iff:

- (1) no type in the body of a recursive binder sends to a replicated branch; and
- (2) continuations of replicated branches do not send to other replicated branches.

Example 4.14. The protocols modelling the dining philosophers problem in Example 4.6 are *trivially finite*. Note how the chopstick services make the initial request to the lock service *before* they offer their replicated branch.

The trivially finite property is restrictive. Indeed it nullifies the benefit of the using the client-server paradigm to build modular systems in which servers can act as clients to other servers. For such protocols a more nuanced strategy is required. Definition 4.15 formalises “loops” in a protocol which may result from replicated servers infinitely bouncing messages amongst each other (such as Γ' in Example 4.11).

Definition 4.15 (Loop free). Given a protocol Ψ , and a context Γ derived from Ψ (possibly after a number of reductions), a **cycle** in the LTS of Γ is defined as the series of transitions s.t., for $\Gamma = \Gamma' \cdot s[\mathbf{p}] : !\rho \&_{i \in I} m_i(\vec{T}_i).S_i$

$$\Gamma \xrightarrow{s:q,p:m_k} \left(\xrightarrow{s:p'_j,q'_j:m'_j} \right)_{j \in 1..n} \xrightarrow{s:q,p:m_k} \Gamma''$$

where $k \in I$, and for any $\mathbf{p}', \mathbf{q}', m', n, \Gamma''$. A cyclic replicated communication path (**CRCP**) is defined as a cycle with these added conditions:

- (1) $\Gamma(s[\mathbf{q}]) = S_q$ s.t. either $S_q \equiv !S^\&|U$ or S_q appears after a recursive binder in $\Psi(\mathbf{q})$, for any $S^\&, U$; and
- (2) $\forall x \in 1..n : \Gamma \xrightarrow{s:q,p:m_k} \left(\xrightarrow{s:p'_l,q'_l:m'_l} \right)_{l \in 1..x-1} \Gamma''' \xrightarrow{s:p'_x,q'_x:m'_x} \Gamma'''$, it must be that:

$$\Gamma'''(s[\mathbf{q}'_x]) = S_{q'_x} \text{ s.t. either } S_{q'_x} \equiv !S^\&|U \text{ or } S_{q'_x} \text{ appears after a recursive binder in } \Psi(\mathbf{q}'_x).$$

Γ is said to be **loop free**, written $\text{lf}(\Gamma)$, iff the LTS of Γ does not contain a CRCP.

A *cycle* in the LTS of a context Γ : (i) starts with an incoming communication action into a replicated type; (ii) performs some intermediary transitions; and (iii) ends with the transition that began the cycle. A CRCP is a special case of a cycle, where all intermediary transitions must also be between roles that have a replicated type; or form part of the body of a recursive type. Finally, a context is *loop free* iff its LTS does not produce any CRCPs.

Example 4.16. Contexts Γ and Γ' from Example 4.11 contain CRCPs: Γ contains a CRCP at $\mathbf{q}$ with 0 intermediary transitions; and Γ' contains two CRCPs, at $\mathbf{q}$ and $\mathbf{p}$, both with 1 intermediary transition forming part of a replicated type.

Example 4.17. The protocols in Examples 1.3, 4.3 and 4.4 are *loop free*: Example 1.3 because there are no cycles; Examples 4.3 and 4.4 because the cycle between the pong branch on $\mathbf{T}$ (resp. $\mathbf{M}$) and the ping branch on $\mathbf{P}$ (resp. $\mathbf{p}_1, \mathbf{p}_2$) includes communication with the (non-replicated/-recursive) client; breaking the CRCP.

PROPOSITION 4.18. *If $\text{beh}(\Gamma)$ is infinite, then Γ contains a CRCP.*

PROOF. From the type semantics (Figure 7), the only reductions that can yield *larger* types are communications with replicated branches. Therefore, it follows that for $\text{beh}(\Gamma)$ to be infinite, there must be some reoccurring transitions in the LTS of Γ that repeatedly communicates with a replicated branch. That is, there is a sequence of transitions, repeated infinitely, that results in communication with a replicated type. This is the definition of a CRCP (Definition 4.15). $\square$

THEOREM 4.19 (STRATEGY SOUNDNESS). *Given a context Γ , $\Phi(\Gamma)$ implies $\text{beh}(\Gamma)$ is finite, for $\Phi \in \{\text{tf}, \text{lf}\}$.*

PROOF. *Case tf.* By contradiction: assume $\text{beh}(\Gamma)$ is infinite, then, by Proposition 4.18, Γ contains a CRCP; but, by Definition 4.15, a CRCP will violate at least one of the conditions for *tf* in Definition 4.13—contradiction.

Case lf. By contradiction: assume $\text{beh}(\Gamma)$ is infinite, then, by Proposition 4.18, Γ contains a CRCP—contradiction; therefore $\text{beh}(\Gamma)$ is *finite*. $\square$

Approximations. Properties *tf* and *lf* are both *decidable* for all protocols *without* first-class roles: *tf* can be determined via a linear traversal of a type context; and *lf* can be checked by constructing a directed graph of visited replicated branches in a context, then checking that the graph is acyclic (which is decidable). An approximation is only required for protocols using role variables, since their values can only be known at runtime. One possible approximation is to treat any role variable in a selection type to have the capability of reaching *any other role*.

Example 4.20 (*Approximation false negative*). Consider the following protocol:

$$p : !\alpha \& m. \alpha \oplus m' \quad q : p \oplus m. p \& m'. r \oplus m'. r \& m \quad r : !\beta \& m'. \beta \oplus m$$

Although the above is *trivially finite* (p and r do not communicate), it would be *falsely* flagged as *not tf* because α is over-approximated to include r .

It is key to note that false negatives of the approximation are avoidable by *requiring unique branching labels on replicated types*. Furthermore, even with this approximation, all examples presented in this chapter (except Example 4.7) are captured by either *lf* or *tf*. With respect to Example 4.7, the presented protocol still yields a finite behavioural set, and thus by Theorem 4.12 and Theorem 4.8, it lives in the subset of MPST! for which typechecking is decidable.

5 Related Work

Toninho and Yoshida [32] assess the relative expressiveness of a multiparty session calculus and a process calculus inspired by classical linear logic, showing that MPST calculi allow strictly more expressive process networks (i.e., those that can include cycles). As part of this investigation they explore a limited form of type-level replication that permits a liveness property. However, their system does not consider first-class roles and pre-dates generalised MPST so is guided primarily by global types, and is therefore less expressive than MPST!.

Replicated session types have been used to a limited extent in a wide variety of works on binary session types (e.g., [3, 4, 6, 35]), primarily in works pertaining to Curry-Howard interpretations of propositions as session types, where the exponential modality from linear logic $!A$ is typically linked to replication from the π -calculus. Several further lines of work investigate client-server communication following this correspondence.

Kokke et al. [17] investigate an extension of the logically-inspired HCP calculus [16] with two dual modalities $!_n A$ and $?_n A$ to type a pool of n clients and a replicated server that can service n requests respectively, and show that their calculus allows nondeterministic behaviour while still preventing deadlocks and ensuring termination. Qian et al. [23] develop CSLL (client-server

linear logic) that uses the dual *coexponential* modalities $\downarrow A$ and $\uparrow A$ to type servers and client pools respectively, along with rules to merge client pools. The subtle difference between the $\downarrow A$ modality and the exponential $!A$ being that the former serves type A *only as many times as required according to client requests*. This is similar to how replication in the MPST! type system operates, given that replicated receives only pull out copies of continuations upon communication. Multiple requests induce non-determinism into further reductions; in MPST! this is observed through parallel types, which in the work of Qian et al. [23] is observed through hyperenvironments [9, 16].

Unlike all the aforementioned works, the calculus developed in this chapter focuses on *multiparty* session types, where interacting with a replicated channel spawns a process that *remains in the same session*. This results in the key novelty of the type system, i.e., the semantic interpretation of type-level replication through the use of parallel types.

Marshall and Orchard [21] investigate the effects of adding a *semiring graded necessity* modality (a generalisation of the $!$ modality) to a session-typed λ -calculus, showing interesting consequences such as replicated servers and multicast communication. However, the type system was developed to target a different class of protocols than to those of MPST!. Specifically, their type system allows for describing protocols that rely on multicasts; a construct that cannot be straightforwardly encoded in MPST!. Meanwhile, it is difficult to see how their approach would scale to the examples described in section 4.

Rocha and Caires [24] introduce CLASS, a process calculus with a correspondence to Differential Linear Logic [8]. CLASS integrates session-typed communication, reference cells with mutual exclusion, and replication. Their calculus guarantees preservation and progress, the proof of the latter property requiring a logical relation. CLASS can encode the dining philosophers problem, making essential use of shared state; in contrast, MPST! does not rely on sharing memory, but rather leverages the interplay between replication and recursion to model locks.

Deniérou et al. [7] introduce parameterised MPST as a means of designing protocols for parallel algorithms. Their formalism allows for parameterisation of participants in the form of $c[i]$, representing the i^{th} client from some group of n clients, for a bound n . The key difference between their formalism and MPST! is that the latter preserves, and allows for the fair handling of, *races*. Parameterised MPST enforce a predetermined prioritisation on the order of communication (thus, Example 4.7 cannot be expressed in that system).

6 Conclusion

This work presented MPST!, a multiparty session π -calculus introducing for the first time *replication* and *first-class roles*, and proved its metatheory. It has been shown that the interplay between replication and recursion allows for describing interesting and previously inexpressible MPST protocols such as those that rely on races and mutual exclusion, as well as giving a method by which one can express context-free protocols. Although replication can have implications for decidability of typechecking, (decidable) strategies for determining whether protocols live within a decidable subset of MPST! have been identified.

For future work, it would be interesting to investigate an extension of MPST! with polymorphism, as this would continue to improve on the modular design of protocols already promoted by universal receives. Furthermore, a deeper investigation of decidability of type checking is required to determine the merit of studying and implementing the decidable strategies of Section 4.2.1. It is conjectured that type checking is indeed decidable (at least without first-class roles). Proving this will open a new path of study: investigating how to implement MPST! without the need for restrictions to protocols.

References

- [1] Bernardo Almeida, Andreia Mordido, Peter Thiemann, and Vasco T. Vasconcelos. 2022. Polymorphic lambda calculus with context-free session types. *Inf. Comput.* 289, Part (2022), 104948. doi:10.1016/j.ic.2022.104948
- [2] Lorenzo Bettini, Mario Coppo, Loris D'Antoni, Marco De Luca, Mariangiola Dezani-Ciancaglini, and Nobuko Yoshida. 2008. Global Progress in Dynamically Interleaved Multiparty Sessions. In *CONCUR 2008 - Concurrency Theory, 19th International Conference, CONCUR 2008, Toronto, Canada, August 19-22, 2008. Proceedings (Lecture Notes in Computer Science, Vol. 5201)*, Franck van Breugel and Marsha Chechik (Eds.). Springer, 418–433. doi:10.1007/978-3-540-85361-9_33
- [3] Luís Caires and Jorge A. Pérez. 2017. Linearity, Control Effects, and Behavioral Types. In *Programming Languages and Systems - 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings (Lecture Notes in Computer Science, Vol. 10201)*, Hongseok Yang (Ed.). Springer, 229–259. doi:10.1007/978-3-662-54434-1_9
- [4] Luís Caires, Frank Pfenning, and Bernardo Toninho. 2016. Linear logic propositions as session types. *Math. Struct. Comput. Sci.* 26, 3 (2016), 367–423. doi:10.1017/S0960129514000218
- [5] Mario Coppo, Mariangiola Dezani-Ciancaglini, Luca Padovani, and Nobuko Yoshida. 2015. A Gentle Introduction to Multiparty Asynchronous Session Types. In *Formal Methods for Multicore Programming - 15th International School on Formal Methods for the Design of Computer, Communication, and Software Systems, SFM 2015, Bertinoro, Italy, June 15-19, 2015, Advanced Lectures (Lecture Notes in Computer Science, Vol. 9104)*, Marco Bernardo and Einar Broch Johnsen (Eds.). Springer, 146–178. doi:10.1007/978-3-319-18941-3_4
- [6] Ornela Dardha and Simon J. Gay. 2018. A New Linear Logic for Deadlock-Free Session-Typed Processes. In *Foundations of Software Science and Computation Structures - 21st International Conference, FOSSACS 2018, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2018, Thessaloniki, Greece, April 14-20, 2018, Proceedings (Lecture Notes in Computer Science, Vol. 10803)*, Christel Baier and Ugo Dal Lago (Eds.). Springer, 91–109. doi:10.1007/978-3-319-89366-2_5
- [7] Pierre-Malo Deniérou, Nobuko Yoshida, Andi Bejleri, and Raymond Hu. 2012. Parameterised Multiparty Session Types. *Log. Methods Comput. Sci.* 8, 4 (2012). doi:10.2168/LMCS-8(4:6)2012
- [8] Thomas Ehrhard. 2018. An introduction to differential linear logic: proof-nets, models and antiderivatives. *Math. Struct. Comput. Sci.* 28, 7 (2018), 995–1060.
- [9] Simon Fowler, Wen Kokke, Ornela Dardha, Sam Lindley, and J. Garrett Morris. 2023. Separating Sessions Smoothly. *Log. Methods Comput. Sci.* 19, 3 (2023).
- [10] Simon J. Gay and Malcolm Hole. 2005. Subtyping for session types in the pi calculus. *Acta Informatica* 42, 2-3 (2005), 191–225. doi:10.1007/S00236-005-0177-Z
- [11] Simon J. Gay and Vasco Thudichum Vasconcelos. 2025. *Session Types*. Cambridge University Press.
- [12] Paul Harvey, Simon Fowler, Ornela Dardha, and Simon J. Gay. 2021. Multiparty Session Types for Safe Runtime Adaptation in an Actor Language. In *35th European Conference on Object-Oriented Programming, ECOOP 2021, July 11-17, 2021, Aarhus, Denmark (Virtual Conference) (LIPIcs, Vol. 194)*, Anders Møller and Manu Sridharan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 10:1–10:30. doi:10.4230/LIPICS.ECOOP.2021.10
- [13] Kohei Honda. 1993. Types for Dyadic Interaction. In *CONCUR '93, 4th International Conference on Concurrency Theory, Hildesheim, Germany, August 23-26, 1993, Proceedings (Lecture Notes in Computer Science, Vol. 715)*, Eike Best (Ed.). Springer, 509–523. doi:10.1007/3-540-57208-2_35
- [14] Kohei Honda, Vasco Thudichum Vasconcelos, and Makoto Kubo. 1998. Language Primitives and Type Discipline for Structured Communication-Based Programming. In *Programming Languages and Systems - ESOP'98, 7th European Symposium on Programming, Held as Part of the European Joint Conferences on the Theory and Practice of Software, ETAPS'98, Lisbon, Portugal, March 28 - April 4, 1998, Proceedings (Lecture Notes in Computer Science, Vol. 1381)*, Chris Hankin (Ed.). Springer, 122–138. doi:10.1007/BFb0053567
- [15] Kohei Honda, Nobuko Yoshida, and Marco Carbone. 2016. Multiparty Asynchronous Session Types. *J. ACM* 63, 1 (2016), 9:1–9:67. doi:10.1145/2827695
- [16] Wen Kokke, Fabrizio Montesi, and Marco Peressotti. 2019. Better late than never: a fully-abstract semantics for classical processes. *Proc. ACM Program. Lang.* 3, POPL (2019), 24:1–24:29.
- [17] Wen Kokke, J. Garrett Morris, and Philip Wadler. 2020. Towards Races in Linear Logic. *Log. Methods Comput. Sci.* 16, 4 (2020).
- [18] Matthew Alan Le Brun and Ornela Dardha. 2023. $\text{MAG}\pi$: Types for Failure-Prone Communication. In *Programming Languages and Systems - 32nd European Symposium on Programming, ESOP 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2023, Paris, France, April 22-27, 2023, Proceedings (Lecture Notes in Computer Science, Vol. 13990)*, Thomas Wies (Ed.). Springer, 363–391. doi:10.1007/978-3-031-30044-8_14
- [19] Matthew Alan Le Brun and Ornela Dardha. 2024. $\text{MAG}\pi$!: The Role of Replication in Typing Failure-Prone Communication. In *Formal Techniques for Distributed Objects, Components, and Systems - 44th IFIP WG 6.1 International Conference, FORTE 2024, Held as Part of the 19th International Federated Conference on Distributed Computing Techniques,*

- DisCoTec 2024, Groningen, The Netherlands, June 17-21, 2024, *Proceedings (Lecture Notes in Computer Science, Vol. 14678)*, Valentina Castiglioni and Adrian Francalanza (Eds.). Springer, 99–117. doi:10.1007/978-3-031-62645-6_6
- [20] Matthew Alan Le Brun, Simon Fowler, and Ornela Dardha. 2025. Multiparty Session Types with a Bang!. In *Programming Languages and Systems - 34th European Symposium on Programming, ESOP 2025, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2025, Hamilton, ON, Canada, May 3-8, 2025, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 15695)*, Viktor Vafeiadis (Ed.). Springer, 125–153. doi:10.1007/978-3-031-91121-7_6
- [21] Danielle Marshall and Dominic Orchard. 2022. Replicate, Reuse, Repeat: Capturing Non-Linear Communication via Session Types and Graded Modal Types. In *Proceedings of the 13th International Workshop on Programming Language Approaches to Concurrency and Communication-cEntric Software, PLACES@ETAPS 2022, Munich, Germany, 3rd April 2022 (EPTCS, Vol. 356)*, Marco Carbone and Rumyana Neykova (Eds.). 1–11. doi:10.4204/EPTCS.356.1
- [22] Diogo Poças, Diana Costa, Andreia Mordido, and Vasco T. Vasconcelos. 2023. System F_{ω}^H with Context-free Session Types. In *Programming Languages and Systems - 32nd European Symposium on Programming, ESOP 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2023, Paris, France, April 22-27, 2023, Proceedings (Lecture Notes in Computer Science, Vol. 13990)*, Thomas Wies (Ed.). Springer, 392–420. doi:10.1007/978-3-031-30044-8_15
- [23] Zesen Qian, G. A. Kavvos, and Lars Birkedal. 2021. Client-server sessions in linear logic. *Proc. ACM Program. Lang.* 5, ICFP (2021), 1–31. doi:10.1145/3473567
- [24] Pedro Rocha and Luis Caires. 2023. Safe Session-Based Concurrency with Shared Linear State. In *ESOP (Lecture Notes in Computer Science, Vol. 13990)*. Springer, 421–450.
- [25] Alceste Scalas, Ornela Dardha, Raymond Hu, and Nobuko Yoshida. 2017. A Linear Decomposition of Multiparty Sessions for Safe Distributed Programming. In *31st European Conference on Object-Oriented Programming, ECOOP 2017, June 19-23, 2017, Barcelona, Spain (LIPIcs, Vol. 74)*, Peter Müller (Ed.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 24:1–24:31. doi:10.4230/LIPIcs.ECOOP.2017.24
- [26] Alceste Scalas, Ornela Dardha, Raymond Hu, and Nobuko Yoshida. 2017. A Linear Decomposition of Multiparty Sessions for Safe Distributed Programming (Artifact). *Dagstuhl Artifacts Ser.* 3, 2 (2017), 03:1–03:2. doi:10.4230/DARTS.3.2.3
- [27] Alceste Scalas and Nobuko Yoshida. 2018. *Less is more: multiparty session types revisited*. Technical Report 6. Imperial College London. <https://www.doc.ic.ac.uk/research/technicalreports/2018/6>
- [28] Alceste Scalas and Nobuko Yoshida. 2019. Less is more: multiparty session types revisited. *Proc. ACM Program. Lang.* 3, POPL (2019), 30:1–30:29. doi:10.1145/3290343
- [29] Alok Sinha. 1992. Client-server computing. *Commun. ACM* 35, 7 (July 1992), 77–98. doi:10.1145/129902.129908
- [30] Felix Stutz. 2024. *Implementability of Asynchronous Communication Protocols - The Power of Choice*. Ph. D. Dissertation. Kaiserslautern University of Technology, Germany. <https://kluedo.ub.rptu.de/frontdoor/index/index/docId/8077>
- [31] Peter Thiemann and Vasco T. Vasconcelos. 2016. Context-free session types. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016, Nara, Japan, September 18-22, 2016*, Jacques Garrigue, Gabriele Keller, and Eijiro Sumii (Eds.). ACM, 462–475. doi:10.1145/2951913.2951926
- [32] Bernardo Toninho and Nobuko Yoshida. 2018. Interconnectability of Session-Based Logical Processes. *ACM Trans. Program. Lang. Syst.* 40, 4 (2018), 17:1–17:42.
- [33] Amjad Umar and CW Fraser. 1993. *Distributed computing: a practical synthesis of networks, client-server systems, distributed applications, and open systems*. Prentice-Hall, Inc.
- [34] Vasco T. Vasconcelos. 2012. Fundamentals of session types. *Inf. Comput.* 217 (2012), 52–70. doi:10.1016/J.IC.2012.05.002
- [35] Philip Wadler. 2014. Propositions as sessions. *J. Funct. Program.* 24, 2-3 (2014), 384–418. doi:10.1017/S095679681400001X
- [36] Nobuko Yoshida and Ping Hou. 2024. Less is More Revisited: Association with Global Multiparty Session Types. In *The Practice of Formal Methods: Essays in Honour of Cliff Jones, Part II*, Ana Cavalcanti and James Baxter (Eds.). Lecture Notes in Computer Science, Vol. 14781. Springer, 268–291. doi:10.1007/978-3-031-66673-5_14

A Lemmata

A.1 Context Operations

Lemma A.1 verifies that context addition in MPST! is the left inverse of context splitting; i.e., the pieces of a split context can be added back together to form the original context.

LEMMA A.1. *If $\Gamma = \Gamma_1 \cdot \Gamma_2$, then $\Gamma_1 + \Gamma_2 = \Gamma$.*

PROOF. By induction on the derivation of $\Gamma = \Gamma_1 \cdot \Gamma_2$.

Case 1:
$$\frac{}{\Gamma = \emptyset = \emptyset \cdot \emptyset}$$

$\emptyset + \emptyset = \emptyset$

(by fig. 5) (1)

Case 1 holds.

(by (1)) (2)

Case 2:
$$\frac{\Gamma' = \Gamma_1 \cdot \Gamma_2}{\Gamma = \Gamma', c : U = (\Gamma_1, c : U) \cdot \Gamma_2}$$

$c \notin \text{dom}(\Gamma')$

(from the hyp. and def. of context composition) (1)

$c \notin \text{dom}(\Gamma_2)$

(by (1) and the hyp.) (2)

$\Gamma_1 + \Gamma_2 = \Gamma'$

(the ind. hyp.) (3)

$(\Gamma_1, c : U) + \Gamma_2 = \Gamma', c : U$

(by (2), (3), and fig. 5) (4)

Case 2 holds.

(by (4)) (5)

Case 3:
$$\frac{\Gamma' = \Gamma_1 \cdot \Gamma_2}{\Gamma = \Gamma', c : U = \Gamma_1 \cdot (\Gamma_2, c : U)}$$

$c \notin \text{dom}(\Gamma')$

(from the hyp. and def. of context composition) (1)

$c \notin \text{dom}(\Gamma_1)$

(by (1) and the hyp.) (2)

$\Gamma_1 + \Gamma_2 = \Gamma'$

(the ind. hyp.) (3)

$\Gamma_1 + (\Gamma_2, c : U) = \Gamma', c : U$

(by (2), (3), and fig. 5) (4)

Case 3 holds.

(by (4)) (5)

Case 4:
$$\frac{\Gamma' = \Gamma_1 \cdot \Gamma_2}{\Gamma = \Gamma', x : B = \Gamma_1, x : B \cdot \Gamma_2, x : B}$$

$\Gamma_1 + \Gamma_2 = \Gamma'$

(the ind. hyp.) (1)

$\Gamma_1, x : B + \Gamma_2, x : B = \Gamma', x : B$

(by (1) and fig. 5) (2)

Case 4 holds.

(by (2)) (3)

Case 5:
$$\frac{\Gamma' = \Gamma_1 \cdot \Gamma_2}{\Gamma = \Gamma', \alpha : \alpha = \Gamma_1, \alpha : \alpha \cdot \Gamma_2, \alpha : \alpha}$$

$\Gamma_1 + \Gamma_2 = \Gamma'$

(the ind. hyp.) (1)

$\Gamma_1, \alpha : \alpha + \Gamma_2, \alpha : \alpha = \Gamma', \alpha : \alpha$

(by (1) and fig. 5) (2)

Case 5 holds.

(by (2)) (3)

Case 6:
$$\frac{\Gamma' = \Gamma_1 \cdot \Gamma_2}{\Gamma = \Gamma', c : U_1 | U_2 = \Gamma_1, c : U_1 \cdot \Gamma_2, c : U_2}$$

$\Gamma_1 + \Gamma_2 = \Gamma'$

(the ind. hyp.) (1)

$\Gamma_1, c : U_1 + \Gamma_2, c : U_2 = \Gamma', c : U_1 | U_2$

(by (1) and fig. 5) (2)

Case 6 holds.

(by (2)) (3)

□

Lemma A.2 states that contexts not containing session types can be added into the type environment with no effect on typing a process.

LEMMA A.2. *If $\Gamma \vdash_D P$ and $\Gamma' = \{\widetilde{b} : \vec{T}\}$ such that $\forall b \in \text{dom}(\Gamma') : \Gamma'(b) \not\leq S$, then $\Gamma + \Gamma' \vdash_D P$*

PROOF. Firstly observe that by the hypothesis and definition 2.7, $\text{end}(\Gamma')$. Furthermore, since Γ' only contains non-linear types, by fig. 5, addition of Γ' distributes over context splits. (Since all non-linear types are merged when added.) Now proceed by rule induction on $\Gamma \vdash_D P$.

Case T-0: $\frac{\text{end}(\Gamma)}{\Gamma \vdash_D 0}$

$\text{end}(\Gamma)$ (by T-0) (1)
 $\text{end}(\Gamma + \Gamma')$ (by definition 2.7, fig. 5, and since $\text{end}(\Gamma')$) (2)
 $\Gamma + \Gamma' \vdash_D 0$ (by (2), T-0) (3)
 Case T-0 holds. (by (3)) (4)

Case T- $\oplus$: $\frac{\Gamma_{\oplus} \vdash c : \oplus \rho : m(\vec{T}).S \quad \Gamma_r \vdash \rho : \rho}{(\Gamma_i \vdash V_i : \vec{T}_i)_{i \in 1..n} \quad \Gamma_0 + c : S \vdash_D P'}$
 $\Gamma = \Gamma_0 \cdot \Gamma_{\oplus} \cdot \Gamma_r \cdot (\Gamma_i)_{i \in 1..n} \vdash_D P = c[\rho] \oplus m\langle (V_i)_{i \in 1..n} \rangle \cdot P'$

$\Gamma + \Gamma' = (\Gamma_0 + \Gamma') \cdot (\Gamma_{\oplus} + \Gamma') \cdot (\Gamma_r + \Gamma') \cdot (\Gamma_i + \Gamma')_{i \in 1..n}$ (by fig. 5) (1)
 $(\Gamma_0 + \Gamma') + c : S \vdash_D P'$ (by ind. hyp) (2)
 $\Gamma_{\oplus} + \Gamma' \vdash c : \oplus \rho : m(\vec{T}).S$ (by T-Wkn, $\text{end}(\Gamma')$) (3)
 $\Gamma_r + \Gamma' \vdash \rho : \rho$ (by T-Wkn, $\text{end}(\Gamma')$) (4)
 $\forall i \in 1..n : \Gamma_i + \Gamma' \vdash V_i : \vec{T}_i$ (by T-Wkn, $\text{end}(\Gamma')$) (5)
 $\Gamma + \Gamma' \vdash_D P$ (by (2), (3), (4), (5), T- $\oplus$) (6)
 Case T- $\oplus$ holds. (by (6)) (7)

Case T- $\&$, T- $!$: Similar to Case T- $\oplus$.

Case T- $|$: $\frac{\Gamma_1 \vdash_D P_1 \quad \Gamma_2 \vdash_D P_2}{\Gamma = \Gamma_1 \cdot \Gamma_2 \vdash_D P = P_1 | P_2}$

$\Gamma = (\Gamma_1 + \Gamma') \cdot (\Gamma_2 + \Gamma')$ (by fig. 5) (1)
 Case T- $|$ holds. (by (1), the ind. hyp. and T- $|$) (2)

Case T-X: $\frac{D(X) = (\vec{x}, \vec{T}, P) \quad \forall i \in 1..n : \Gamma_i \vdash V_i : \vec{T}_i \quad \text{end}(\Gamma_0)}{\Gamma = \Gamma_0 \cdot (\Gamma_i)_{i \in 1..n} \vdash_D P = X\langle (V_i)_{i \in 1..n} \rangle}$

$\Gamma + \Gamma' = \Gamma_0 + \Gamma' \cdot (\Gamma_i + \Gamma')_{i \in 1..n}$ (by fig. 5) (1)
 $\text{end}(\Gamma_0 + \Gamma')$ (since $\text{end}(\Gamma_0)$ and $\text{end}(\Gamma')$) (2)
 $\forall i \in 1..n : \Gamma_i + \Gamma' \vdash V_i : \vec{T}_i$ (by T-Wkn) (3)
 Case T-X holds. (by (2), (3), T-X) (4)

Case T- $+$, T- ν : $\frac{(\Gamma \vdash_D P_i^{\oplus})_{i \in I}}{\Gamma \vdash_D \sum_{i \in I} P_i^{\oplus}}$ and $\frac{\varphi(\text{assoc}_s(\Psi)) \quad \varphi \text{ is a } !\text{-safety property}}{s \notin \Gamma \quad \Gamma + \text{assoc}_s(\Psi) \vdash_D P}$
 $\Gamma \vdash_D (\nu s : \Psi) P$

Both hold directly from the ind. hyp. $\square$

A.2 Substitution

LEMMA A.3. *If $\Gamma_1 + \Gamma_2 = \Gamma$, then $\Gamma_1\{q/\alpha\} + \Gamma_2\{q/\alpha\} = \Gamma\{q/\alpha\}$.*

PROOF. By induction on the derivation of $\Gamma_1 + \Gamma_2 = \Gamma$.

Case 1: $\frac{}{\Gamma + \emptyset = \Gamma}$. Since $\emptyset\{q/\alpha\} = \emptyset$, then $\Gamma\{q/\alpha\} + \emptyset = \Gamma\{q/\alpha\}$.

Case 2 and 3: $\frac{\Gamma_1 + \Gamma_2 = \Gamma}{\Gamma_1, \alpha : \alpha + \Gamma_2, \alpha : \alpha = \Gamma, \alpha : \alpha}$ and $\frac{\Gamma_1 + \Gamma_2 = \Gamma}{\Gamma_1, x : B + \Gamma_2, x : B = \Gamma, x : B}$

Hold by the ind. hyp. and since $\{\alpha : \alpha\}\{q/\alpha\} = \{\alpha : \alpha\}$ and $\{x : B\}\{q/\alpha\} = \{x : B\}$ (by fig. 8).

Case 4: $\frac{\Gamma_1 + \Gamma_2 = \Gamma \quad a \notin \text{dom}(\Gamma_2)}{\Gamma_1, a : T + \Gamma_2 = \Gamma, a : T}$

First, observe that by fig. 8, substitution does not affect context domains.

$(\Gamma_1, a : T)\{q/\alpha\} = (\Gamma_1\{q/\alpha\}), (a : T\{q/\alpha\})$ (by fig. 8) (1)

$\Gamma_1\{q/\alpha\} + \Gamma_2\{q/\alpha\} = \Gamma\{q/\alpha\}$ (ind. hyp.) (2)

$a \notin \text{dom}(\Gamma_2\{q/\alpha\})$ (since $a \notin \text{dom}(\Gamma_2)$) (3)

$(\Gamma_1, a : T)\{q/\alpha\} + \Gamma_2\{q/\alpha\} = (\Gamma\{q/\alpha\}), (a : T\{q/\alpha\})$ (by (1), (2), (3), +) (4)

Case 4 holds. (by (4), fig. 8) (5)

Case 5: $\frac{\Gamma_1 + \Gamma_2 = \Gamma \quad a \notin \text{dom}(\Gamma_1)}{\Gamma_1 + \Gamma_2, a : T = \Gamma, a : T}$. Similar to Case 4.

Case 6: $\frac{\Gamma_1 + \Gamma_2 = \Gamma}{\Gamma_1, c : U_1 + \Gamma_2, c : U_2 = \Gamma, c : U_1|U_2}$. Holds directly from the ind. hyp.

□

LEMMA A.4. If $\Gamma \vdash V : T$, then $\Gamma\{q/\alpha\} \vdash V\{q/\alpha\} : T\{q/\alpha\}$.

PROOF. By rule induction on $\Gamma \vdash V : T$.

Case T-Sub: $\frac{T \leq T'}{\Gamma = c : T \vdash (V = c) : T'}$. RTP: $(c : T)\{q/\alpha\} \vdash c\{q/\alpha\} : T'\{q/\alpha\}$

Note that if c is a variable and T is not a session type, then role substitution has no effect and thus the case would hold trivially. Consider now $T = S$ and $T' = S'$.

$S \leq S'$ (by assumption) (1)

$S\{q/\alpha\} \leq S'\{q/\alpha\}$ (by (1) and lemma A.9) (2)

$(c : S)\{q/\alpha\} \vdash c : S'\{q/\alpha\}$ (by (2) and rule T-Sub) (3)

$c\{q/\alpha\} = c$ (since $c = x$ or $c = s[p]$, for any x, s, p) (4)

Case T-Sub holds. (by (3), (4), and since $S = T$ and $S' = T'$) (5)

Case T-Wkn: $\frac{\Gamma_1 + \Gamma_2 = \Gamma \quad \Gamma_1 \vdash V : T \quad \text{end}(\Gamma_2)}{\Gamma \vdash V : T}$. RTP: $\Gamma\{q/\alpha\} \vdash V\{q/\alpha\} : T\{q/\alpha\}$

$\Gamma_1\{q/\alpha\} \vdash V\{q/\alpha\} : T\{q/\alpha\}$ (ind. hyp.) (1)

$\text{end}(\Gamma_2)$ (by assumption) (2)

$\Gamma_2 = \Gamma_2\{q/\alpha\}$ (by fig. 8) (3)

$\Gamma_1\{q/\alpha\} + \Gamma_2 = \Gamma\{q/\alpha\}$ (by hyp., (3), and lemma A.3) (4)

$\Gamma\{q/\alpha\} \vdash V\{q/\alpha\} : T\{q/\alpha\}$ (by (1), (2), (4)) (5)

Case T-Wkn holds. (by (5)) (6)

Case T- α : $\frac{}{\alpha : \alpha \vdash \alpha : \alpha}$. RTP: $(\alpha : \alpha)\{q/\alpha\} \vdash (\alpha\{q/\alpha\}) : (\alpha\{q/\alpha\})$

It is key to note that the substitution has no effect on role singletons in the type context, but does indeed substitute the role variables in the value and value type positions. Therefore, it must be shown that: $\alpha : \alpha \vdash q : q$.

$\emptyset + \alpha : \alpha = \alpha : \alpha$ (by fig. 5) (1)

$\emptyset \vdash q : q$ (by rule T-q) (2)

$\text{end}(\alpha : \alpha)$ (by definition 2.7) (3)

$\alpha : \alpha \vdash q : q$ (by (1),(2),(3), and rule T-Wkn) (4)

Case T- α holds. (by (4)) (5)

Case T-B, T-q: These cases hold trivially.

□

LEMMA A.5. If $\Gamma \vdash_D P$, then $\Gamma\{q/\alpha\} \vdash_D P\{q/\alpha\}$.

PROOF. By rule induction on $\Gamma \vdash_D P$.

Case T-0: $\frac{\text{end}(\Gamma)}{\Gamma \vdash_D 0}$. Since $\text{end}(\Gamma)$, then $\Gamma\{q/\alpha\} = \Gamma$ (fig. 8). Also $0\{q/\alpha\} = 0$.

Case T- $\oplus$:
$$\frac{\Gamma_{\oplus} \vdash c : \oplus \rho : m(\vec{T}).S \quad \Gamma_r \vdash \rho : \rho}{(\Gamma_i \vdash V_i : T_i)_{i \in 1..n} \quad \Gamma_0 + c : S \vdash_D P'}{\Gamma = \Gamma_0 \cdot \Gamma_{\oplus} \cdot \Gamma_r(\cdot \Gamma_i)_{i \in 1..n} \vdash_D P = c[\rho] \oplus m\langle (V_i)_{i \in 1..n} \rangle \cdot P'}$$

First observe that by the definition of role substitution on contexts fig. 8, substitution distributes over context splits. Then observe that by the ind. hyp.:

$$(\Gamma_0 + c : S)\{q/\alpha\} \vdash_D P'\{q/\alpha\}$$

Furthermore, by lemma A.4:

$$\begin{aligned} \Gamma_{\oplus}\{q/\alpha\} &\vdash (c\{q/\alpha\}) : (\oplus \rho : m(\vec{T}).S\{q/\alpha\}) \\ \Gamma_r\{q/\alpha\} &\vdash (\rho\{q/\alpha\}) : (\rho\{q/\alpha\}) \\ \forall i \in 1..n : \Gamma_i\{q/\alpha\} &\vdash (V_i\{q/\alpha\}) : (T_i\{q/\alpha\}) \end{aligned}$$

Then, using T- $\oplus$, the case holds.

All other cases follow the same steps, relying on the inductive hypothesis and lemma A.4. □

LEMMA A.6. $T \leq T'$ implies $T\{q/\alpha\} \leq T'\{q/\alpha\}$.

PROOF. By coinduction on $T \leq T'$ (definition 2.3).

Case 1: $\frac{}{T \leq T}$

The reflexive case immediately holds since the substitution occurs on the same type on both sides.

Case 2:
$$\frac{(\vec{T}_i \leq \vec{T}'_i)_{i \in I} \quad (S_i \leq S'_i)_{i \in I}}{\&_{i \in I} \rho : m_i(\vec{T}_i).S_i \leq \&_{i \in I \cup J} \rho : m_i(\vec{T}'_i).S'_i}$$

From the coinductive hypothesis, it follows that $\forall i \in I : \vec{T}_i\{q/\alpha\} \leq \vec{T}'_i\{q/\alpha\}$ and $S_i\{q/\alpha\} \leq S'_i\{q/\alpha\}$. Furthermore, if $\rho = \alpha$, then the substitution occurs on both sides.

Other cases follow the same reasoning and hold directly from the coinductive hypothesis and since any role substitution made on the type subject affects both sides equally. □

LEMMA A.7. If $\Gamma + \alpha : \alpha \vdash_D P$ and $\alpha \notin \text{frv}(P)$, then $\Gamma \vdash_D P$

PROOF. By rule induction on $\Gamma + \alpha : \alpha \vdash_D P$, observing that the role is never used if it does not appear free. $\square$

LEMMA A.8. If $\Gamma + x : T \vdash_D P$ and $\Gamma' \vdash V : T'$ where $T' \leq T$, then $\Gamma + \Gamma' \vdash_D P\{V/x\}$.

PROOF. By rule induction on the typing derivation. Proof is standard, similar to that of, e.g., [28]. $\square$

A.3 Subtyping

LEMMA A.9. $T \leq T'$ implies $T\{q/\alpha\} \leq T'\{q/\alpha\}$.

PROOF. By coinduction on $T \leq T'$ (definition 2.3).

Case 1: $\frac{}{T \leq T}$

The reflexive case immediately holds since the substitution occurs on the same type on both sides.

Case 2: $\frac{(\vec{T}_i \leq \vec{T}'_i)_{i \in I} \quad (S_i \leq S'_i)_{i \in I}}{\&_{i \in I} \rho : m_i(\vec{T}_i).S_i \leq \&_{i \in I \cup J} \rho : m_i(\vec{T}'_i).S'_i}$

From the coinductive hypothesis, it follows that $\forall i \in I : \vec{T}_i\{q/\alpha\} \leq \vec{T}'_i\{q/\alpha\}$ and $S_i\{q/\alpha\} \leq S'_i\{q/\alpha\}$. Furthermore, if $\rho = \alpha$, then the substitution occurs on both sides.

Other cases follow the same reasoning and hold directly from the coinductive hypothesis and since any role substitution made on the type subject affects both sides equally. $\square$

LEMMA A.10. Assume $\Gamma \vdash V : T$ and $\Gamma' \leq \Gamma$. Then, $\Gamma' \vdash V : T$.

PROOF. By rule induction on the typing derivation. $\square$

LEMMA A.11. Assume $\Gamma \vdash_D P$ and $\Gamma' \leq \Gamma$. Then, $\Gamma' \vdash_D P$.

PROOF. By rule induction on the typing derivation, and using lemma A.10 for values. $\square$

A.4 Congruence

LEMMA A.12. Assume $\Gamma \vdash_D P$ and $P \equiv P'$. Then, $\exists \Gamma'$ s.t. $\Gamma \equiv \Gamma'$ and $\Gamma' \vdash P'$.

PROOF. By rule induction on $P \equiv P'$, observing that for each rule, either the process remains well typed under the same context, or the context has a matching congruence rule. $\square$

B Subject Reduction

LEMMA B.1. For all contexts Γ_1, Γ_2 :

- (1) if $\text{safe}_1(\Gamma_1 \cdot \Gamma_2)$, then $\text{safe}_1(\Gamma_1)$;
- (2) if $\text{safe}_1(\Gamma_1)$ and $\Gamma_1 \leq \Gamma_2$, then $\text{safe}_1(\Gamma_2)$;
- (3) if $\text{safe}_1(\Gamma_1)$ and $\exists \Gamma'_2 : \Gamma_1 \leq \Gamma_2 \rightarrow \Gamma'_2$, then $\exists \Gamma'_1 : \Gamma_1 \rightarrow \Gamma'_1 \leq \Gamma'_2$.

PROOF. (1) First observe that by the definition of frv (Figure 8), if $\text{frv}(\Gamma_1 \cdot \Gamma_2) = \emptyset$ then $\text{frv}(\Gamma_1) = \emptyset$. Hence condition $\varphi\text{-}\rho$ is preserved through context splits. The preservation of the remaining conditions is proved by contradiction. Assume Γ_1 is *not* safe, then there must exist some Γ' such that $\Gamma_1 \rightarrow^* \Gamma''$ and $\Gamma' = \text{unf}^*(\Gamma'')$ and Γ' violates $\varphi\text{-S}$. But by rule $\Gamma\text{-Cong}_2$, $\Gamma_2 \cdot \Gamma_1 \rightarrow^* \Gamma_2 + \Gamma''$ and after some possible applications of rule $\Gamma\text{-}\mu$, $\Gamma_2 + \Gamma'$ violates $\varphi\text{-S}$.

Hence, $\Gamma_2 + \Gamma'$ is not safe. By Table 1, the proof splits into two similar cases; the first case is demonstrated below.

$$\exists s, \mathbf{p}, \mathbf{q}, m, m', \vec{T}, \vec{T}' : \Gamma' \xrightarrow{s:\mathbf{p} \oplus \mathbf{q}:m(\vec{T})} \wedge \Gamma' \xrightarrow{s:\mathbf{q} \& \mathbf{p}:m'(\vec{T}')} \wedge m, m' \in L \wedge \neg \left(\text{all} \left(\Gamma' \xrightarrow{s:\mathbf{p}, \mathbf{q}:m} \right) \right).$$

By the definition of $+$ (Figure 5), context addition only (at-most) increases the number of transitions and does not *decrease* or *replace* transitions.

$$\therefore \Gamma_2 + \Gamma' \xrightarrow{s:\mathbf{p} \oplus \mathbf{q}:m(\vec{T})} \wedge \Gamma_2 + \Gamma' \xrightarrow{s:\mathbf{q} \& \mathbf{p}:m'(\vec{T}')} \wedge m, m' \in L \wedge \neg \left(\text{all} \left(\Gamma_2 + \Gamma' \xrightarrow{s:\mathbf{p}, \mathbf{q}:m} \right) \right).$$

This implies that $\Gamma_2 + \Gamma'$ is not safe, meaning $\Gamma_2 + \Gamma''$ is not safe via $\varphi\text{-}\mu$, and thus $\Gamma_2 + \Gamma_1$ is not safe by $\varphi\text{-}\rightarrow$. By lemma A.1 and commutativity of $\cdot$, $\Gamma_1 \cdot \Gamma_2$ is not safe. This contradicts the thesis.

The other case is when the safety violation occurs through a replicated label, i.e., $m, m' \in R$. The case follows similar reasoning to the above. $\square$

(2) By coinduction on $\text{safe}_!(\Gamma_1)$. Observe that by $\text{safe}_!(\Gamma_1)$, Γ_1 is safe (as in Table 1). Thus, if $\Gamma_1 \xrightarrow{s:\mathbf{p} \oplus \mathbf{q}:m(\vec{T})}$ and $\Gamma_1 \xrightarrow{s:\mathbf{q} \& \mathbf{p}:m'(\vec{T}')} \rightarrow$ then $\Gamma_1 \xrightarrow{s:\mathbf{p}, \mathbf{q}:m}$. Assume communication to be enabled on Γ_1 , and without loss of generality, assume the above output and input actions to also be enabled. By rule $\Gamma\text{-}\oplus$ and rule $\Gamma\text{-}\&$, the shape of Γ_1 can be inferred to be:

$$\begin{aligned} \Gamma_1 &= \Gamma'_1 \cdot s[\mathbf{p}] : S^\oplus \cdot s[\mathbf{q}] : S^\& \\ S^\oplus &= \oplus_{i \in I} \mathbf{r}_i : m_i(\vec{T}_i).S_i \quad \text{for some } I \\ \exists I' \subseteq I \text{ such that } \forall i \in I' : \mathbf{r}_i &= \mathbf{q} \\ S^\& &= \&_{j \in J} \mathbf{p} : m'_j(\vec{T}'_j).S'_j \text{ or } S^\& = !\&_{j \in J} \mathbf{p} : m'_j(\vec{T}'_j).S'_j \text{ or } S^\& = !\&_{j \in J} \alpha : m'_j(\vec{T}'_j).S'_j \end{aligned}$$

By $\varphi\text{-}S$ there are no unexpected messages in $S^\oplus$. Consider now a $S^{\oplus'} \geq S^\oplus$ and $S^{\&' } \geq S^\&$. By the definition of $\leq$ (Definition 2.3), $S^{\oplus'}$ has (at most) less internal choices, and $S^{\&'}$ has (at most) more external choices. Therefore, there are still no unexpected messages in $S^{\oplus'}$. Hence, given $\text{safe}_!(\Gamma_1)$ and a $\Gamma_2 \geq \Gamma_1$, it follows that $\varphi\text{-}S$ holds for Γ_2 . It also holds that $\text{frv}(S^{\oplus'}) = \text{frv}(S^\oplus) = \emptyset$, by $\text{safe}_!(\Gamma_1)$ and since $S^{\oplus'}$ has (at most) less choices than $S^\oplus$; and $\text{frv}(S^{\&'}) = \text{frv}(S^\&) = \emptyset$, by $\text{safe}_!(\Gamma_1)$ and since role variables in any new branches are in binding position (by Figure 8). Furthermore, by Rule $\Gamma\text{-}\mu$, no new transitions are introduced via recursive binders, hence $\varphi\text{-}\mu$ must also hold for Γ_2 . Lastly, by the coinductive hypothesis, $\text{safe}_!$ holds for contexts containing the continuation types of elements in Γ_1 and Γ_2 —these contexts are uniquely contained within the context reduction relation; therefore $\varphi\text{-}\rightarrow$ holds for Γ_2 . $\square$

(3) By case analysis. Assume $\Gamma_2 \rightarrow$ by rule $\Gamma\text{-Com}_1$, then:

$$\begin{aligned} \Gamma_2 &= \Gamma''_2 \cdot s[\mathbf{p}] : S^{\oplus'} \cdot s[\mathbf{q}] : S^{\&' } \\ S^{\oplus'} &= \oplus_{i \in I'} \mathbf{r}_i : m''_i(\vec{T}''_i).S''_i \\ S^{\&' } &= \&_{j \in J'} \mathbf{p} : m'''_j(\vec{T}'''_j).S'''_j \text{ or } S^{\&' } = !\&_{j \in J'} \mathbf{p} : m'''_j(\vec{T}'''_j).S'''_j \\ \exists K' \subseteq I' \text{ such that } \forall i \in K' : \mathbf{r}_i &= \mathbf{q} \\ K' \subseteq J' \text{ and } \forall i \in K' : \vec{T}''_i &\leq \vec{T}'''_i \end{aligned}$$

Via subtyping, the shape of Γ_1 can be inferred (since $\Gamma_1 \leq \Gamma_2$).

$$\Gamma_1 = \Gamma_1'' \cdot s[\mathbf{p}] : S^\oplus \cdot s[\mathbf{q}] : S^\&$$

$$S^\oplus = \oplus_{i \in I} \mathbf{r}_i : \mathbf{m}_i(\vec{T}_i) . S_i$$

$$S^\& = \&_{j \in J} \mathbf{p} : \mathbf{m}'_j(\vec{T}'_j) . S'_j \text{ or } S^\& = !\&_{j \in J} \mathbf{p} : \mathbf{m}'_j(\vec{T}'_j) . S'_j$$

$$\exists K \subseteq I \text{ such that } \forall i \in K : \mathbf{r}_i = \mathbf{q}$$

$$K \subseteq J \text{ and } \forall i \in K : \vec{T}_i \leq \vec{T}'_i$$

By the definition of $\leq$, it follows that $K \supseteq K'$ and therefore $K' \subseteq J$. Hence, since $\text{safe}_!(\Gamma_1)$, $\forall i \in K' : \vec{T}_i'' \leq \vec{T}'_i$; therefore Γ_1 can reduce, at the very least, by the transitions enabled on the supertype. That is, $\exists \Gamma'_1 : \Gamma_1 \rightarrow \Gamma'_1$. Furthermore, from Definition 2.3, type continuations preserve subtyping. Therefore $\Gamma'_1 \leq \Gamma'_2$.

The second case is when $\Gamma_2 \rightarrow$ by rule $\Gamma\text{-Com}_2$. The proof case is similar to the above, just with $S^\&$ and $S^{\&'}$ being universal receives (observe that by Definition 2.3, this has no impact on the set inclusions obtained from subtyping). $\square$

THEOREM B.2 (SUBJECT REDUCTION). *If $\Gamma \vdash_D P$ with $\text{safe}_!(\Gamma)$ and $P \rightarrow_D P'$, then $\exists \Gamma'$ s.t. $\Gamma \rightarrow^* \Gamma'$ and $\Gamma' \vdash_D P'$ with $\text{safe}_!(\Gamma')$.*

PROOF. By induction on the derivation of $P \rightarrow_D P'$.

The assumptions are: **(A1)** $\Gamma \vdash_D P$; **(A2)** $\text{safe}_!(\Gamma)$; and **(A3)** $P \rightarrow_D P'$.

Case R-C:

$$\text{Let } P_\oplus = s[\mathbf{p}][\mathbf{q}] \oplus \mathbf{m}_k(\vec{d}) \cdot Q \quad (\text{definition}) \quad (1)$$

$$\text{Let } P_\& = s[\mathbf{q}][\mathbf{p}] \&_{i \in I} \mathbf{m}_i(\vec{b}_i) \cdot Q'_i \quad (\text{definition}) \quad (2)$$

$$P = P_\oplus \mid P_\& \quad (\text{by R-C, (A3)}) \quad (3)$$

$$P' = Q \mid Q'_k \{\vec{d}/\vec{b}_k\} \quad (\text{by R-C, (A3)}) \quad (4)$$

$$\text{Let } n = |\vec{d}| \quad (\text{definition}) \quad (5)$$

Without loss of generality, consider $\vec{d} = \vec{r}, \vec{d}'$ where $\vec{d}'$ does not contain role values.

$$\Gamma = \Gamma^\oplus \cdot \Gamma^\& \vdash_D P \quad (\text{by (3), (A1), T-}) \quad (6)$$

$$\Gamma^\oplus = \Gamma_0^\oplus \cdot \Gamma_c^\oplus \cdot \Gamma_r^\oplus (\cdot \Gamma_{d_i}^\oplus)_{i \in 1..n} \vdash_D P_\oplus \quad (\text{by (6), (3), T-}\oplus) \quad (7)$$

$$\Gamma_c^\oplus \vdash s[\mathbf{p}] : \oplus \mathbf{q} : \mathbf{m}_k(\vec{r}, \vec{T}) . S \quad (\text{by (7), T-}\oplus) \quad (8)$$

$$\Gamma_r^\oplus \vdash \mathbf{q} : \mathbf{q} \quad (\text{by (7), T-}\oplus) \quad (9)$$

$$\forall i \in 1..|\vec{r}| : \Gamma_{d_i}^\oplus \vdash d_i : \mathbf{r}_i \text{ and } \forall j \in |\vec{r}|..|\vec{T}| : \Gamma_{d_j}^\oplus \vdash d_j : T_j \quad (\text{by (7), T-}\oplus) \quad (10)$$

$$\Gamma_0^\oplus + s[\mathbf{p}] : S \vdash_D Q \quad (\text{by (7), T-}\oplus) \quad (11)$$

$$\Gamma^\& = \Gamma_0^\& \cdot \Gamma_c^\& \cdot \Gamma_r^\& \vdash_D P_\& \quad (\text{by (6), (3), T-}\&) \quad (12)$$

$$\Gamma_c^\& \vdash s[\mathbf{q}] : \&_{i \in I} \mathbf{p} : \mathbf{m}'_i(\vec{\alpha}_i, \vec{T}'_i) . S'_i \quad (\text{by (12), T-}\&) \quad (13)$$

$$\Gamma_r^\& \vdash \mathbf{p} : \mathbf{p} \quad (\text{by (12), T-}\&) \quad (14)$$

$$\forall i \in I : \Gamma_0^\& + s[\mathbf{q}] : S'_i + \overrightarrow{\alpha_i} : \overrightarrow{\alpha_i} + x_i : \vec{T}'_i \vdash_D Q'_i \quad (\text{by (12), T-}\&) \quad (15)$$

$$\Gamma_c^\oplus \xrightarrow{s:\mathbf{p}\oplus\mathbf{q}:\mathbf{m}_k(\vec{r},\vec{T})} \Gamma_c^{\oplus'} \quad (\text{by (8), T-Wkn, T-Sub, definition 2.3}) \quad (16)$$

$$\Gamma_c^\& \xrightarrow{s:\mathbf{q}\&\mathbf{p}:\mathbf{m}'_j(\vec{\alpha}_j,\vec{T}'_j)} \Gamma_{c_j}^{\&' } \text{ for } j \in I \quad (\text{by (13), T-Wkn, T-Sub, definition 2.3}) \quad (17)$$

$$\Gamma_c^\oplus \cdot \Gamma_c^\& \xrightarrow{s:\mathbf{p},\mathbf{q}:\mathbf{m}_k} \Gamma_c^{\oplus'} + \Gamma_{c_k}^{\&' } \{\vec{r}/\vec{\alpha}_k\} \quad (\text{by (16), (17), (A2), lemma B.1, } \Gamma\text{-Com}_1) \quad (18)$$

2157	$\Gamma_c^{\oplus'} \vdash s[p] : S$	(by (8), (16), $\Gamma\text{-}\oplus$, $T\text{-}\oplus$)	(19)
2158	$\Gamma_{c_k}^{\&'} \vdash s[q] : S'_k$	(by (13), (17), $\Gamma\text{-}\&$, $T\text{-}\&$)	(20)
2159	$\Gamma_{c_k}^{\&'}(\vec{r}/\vec{\alpha}_k) \vdash s[q] : S'_k\{\vec{r}/\vec{\alpha}_k\}$	(by (20), lemma A.4)	(21)
2160	$\vec{T} \leq \vec{T}'_k$	(by (16), (17), (18), $\Gamma\text{-Com}_1$)	(22)
2161			
2162	$\Gamma_0^{\&} + \Gamma_{c_k}^{\&'} + \overrightarrow{\alpha_k : \alpha_k} + x_k : \vec{T}'_k \vdash_D Q'_k$	(by (15), (20), $T\text{-Wkn}$, $T\text{-Sub}$)	(23)
2163	$\Gamma_0^{\&} + \Gamma_{c_k}^{\&'}\{\vec{r}/\vec{\alpha}_k\} + \overrightarrow{\alpha_k : \alpha_k} + x_k : \vec{T}'_k \vdash_D Q'_k\{\vec{r}/\vec{\alpha}_k\}$	(by (21), (23), lemma A.5)	(24)
2164	$\Gamma_0^{\&} + \Gamma_{c_k}^{\&'}\{\vec{r}/\vec{\alpha}_k\} + x_k : \vec{T}'_k \vdash_D Q'_k\{\vec{r}/\vec{\alpha}_k\}$	(by (24), lemma A.7)	(25)
2165	$\Gamma_0^{\&} + \Gamma_{c_k}^{\&'}\{\vec{r}/\vec{\alpha}_k\} + (\Gamma_{d_i}^{\oplus})_{i \in 1..n} \vdash_D Q'_k\{\vec{d}/\vec{b}_k\}$	(by (10), (25), (22), lemma A.8)	(26)
2166	$\Gamma_0^{\oplus} + \Gamma_c^{\oplus'} \vdash_D Q$	(by (11), (19), $T\text{-Wkn}$, $T\text{-Sub}$)	(27)
2167			
2168	$(\Gamma_0^{\&} + \Gamma_{c_k}^{\&'}\{\vec{r}/\vec{\alpha}_k\} + (\Gamma_{d_i}^{\oplus})_{i \in 1..n}) \cdot (\Gamma_0^{\oplus} + \Gamma_c^{\oplus'}) \vdash_D P'$	(by (26), (27), $T\text{- }$)	(28)
2169	$\Gamma_0^{\&} + \Gamma_{c_k}^{\&'}\{\vec{r}/\vec{\alpha}_k\} + (\Gamma_{d_i}^{\oplus})_{i \in 1..n} + \Gamma_0^{\oplus} + \Gamma_c^{\oplus'} \vdash_D P'$	(by (28), lemma A.1)	(29)
2170			
2171	$\Gamma \rightarrow \Gamma'$		
2172	$= (\Gamma_c^{\oplus'} + \Gamma_{c_k}^{\&'}\{\vec{r}/\vec{\alpha}_k\}) + (\Gamma_0^{\&} \cdot \Gamma_r^{\&} \cdot \Gamma_0^{\oplus} \cdot \Gamma_r^{\oplus} (\cdot \Gamma_{d_i}^{\oplus})_{i \in 1..n})$	(by (6), (18), $\Gamma\text{-Cong}_1$)	(30)
2173	$= \Gamma_c^{\oplus'} + \Gamma_{c_k}^{\&'}\{\vec{r}/\vec{\alpha}_k\} + \Gamma_0^{\&} + \Gamma_r^{\&} + \Gamma_0^{\oplus} + \Gamma_r^{\oplus} + (\Gamma_{d_i}^{\oplus})_{i \in 1..n}$	(by (30), lemma A.1)	(31)
2174	$\text{end}(\Gamma_r^{\oplus} + \Gamma_r^{\&})$	(by (9), (14), $T\text{-Wkn}$, $T\text{-q}$)	(32)
2175			
2176	$\Gamma' \vdash_D P'$	(by (29), (31), (32), lemma A.2)	(33)
2177	$\text{safe}_l(\Gamma')$	(by (A2), (30), $\varphi \rightarrow$)	(34)
2178	Case R-C holds.	(by (30), (33), (34))	(35)
2179	Case R-!C₁:		
2180	Let $R = !s[q][p] \&_{i \in I} m_i(\vec{b}_i) \cdot Q'_i$	(definition)	(1)
2181	Let $P_{\oplus} = s[p][q] \oplus m_k(\vec{d}) \cdot Q$	(definition)	(2)
2182	Let $n = \vec{d} $	(definition)	(3)
2183	$P = P_{\oplus} \mid R$	(by R-!C ₁ and (A3))	(4)
2184	$P' = Q \mid R \mid Q'_k\{\vec{d}/\vec{b}_k\}$	(by R-!C ₁ and (A3))	(5)
2185			
2186	Without loss of generality, consider $\vec{d} = \vec{r}, \vec{d}'$ where $\vec{d}'$ does not contain role values.		
2187	$\Gamma = \Gamma^{\oplus} \cdot \Gamma^! \vdash_D P$	(by (4), (A1), and $T\text{- }$)	(6)
2188			
2189	$\Gamma^{\oplus} = \Gamma_0^{\oplus} \cdot \Gamma_c^{\oplus} \cdot \Gamma_r^{\oplus} (\cdot \Gamma_{d_i}^{\oplus})_{i \in 1..n} \vdash_D P_{\oplus}$	(by (6), (4), $T\text{-}\oplus$)	(7)
2190	$\Gamma_c^{\oplus} \vdash s[p] : \oplus q : m_k(\vec{r}, \vec{T}) \cdot S$	(by (7), $T\text{-}\oplus$)	(8)
2191	$\Gamma_r^{\oplus} \vdash q : q$	(by (7), $T\text{-}\oplus$)	(9)
2192	$\forall i \in 1.. \vec{r} : \Gamma_{d_i}^{\oplus} \vdash d_i : r_i \text{ and } \forall j \in \vec{r} .. \vec{T} : \Gamma_{d_j}^{\oplus} \vdash d_j : T_j$	(by (7), $T\text{-}\oplus$)	(10)
2193	$\Gamma_0^{\oplus} + s[p] : S \vdash_D Q$	(by (7), $T\text{-}\oplus$)	(11)
2194			
2195	$\Gamma^! = \Gamma_0^! \cdot \Gamma_c^! \vdash_D R$	(by (6), (4), $T\text{-}!$)	(12)
2196	$\Gamma_c^! \vdash s[q] : !\&_{i \in I} p : m_i(\vec{\alpha}_i, \vec{T}_i) \cdot S'_i$	(by (12), $T\text{-}!$)	(13)
2197	$\text{end}(\Gamma_0^!)$	(by (12), $T\text{-}!$)	(14)
2198	$\forall i \in I : \Gamma_0^! + s[q] : S'_i + \overrightarrow{\alpha_i : \alpha_i} + x_i : \vec{T}_i' \leftrightarrow p \vdash_D Q'_i$	(by (12), $T\text{-}!$)	(15)
2199			
2200	$\Gamma_c^{\oplus} \xrightarrow{s:p \oplus q : m_k(\vec{r}, \vec{T})} \Gamma_c^{\oplus'}$	(by (8), $T\text{-Wkn}$, $T\text{-Sub}$, definition 2.3)	(16)
2201			
2202	$\Gamma_c^! \xrightarrow{s:q \& p : m'_j(\vec{\alpha}_j, \vec{T}'_j)} \Gamma_{c_j}^! \text{ for } j \in I$	(by (13), $T\text{-Wkn}$, $T\text{-Sub}$, definition 2.3)	(17)
2203			
2204	$\Gamma_c^{\oplus} \cdot \Gamma_c^! \xrightarrow{s:p, q : m_k} \Gamma_c^{\oplus'} + \Gamma_{c_k}^! \{\vec{r}/\vec{\alpha}\}$	(by (16), (17), (A2), lemma B.1, $\Gamma\text{-Com}_1$)	(18)
2205			

$$\begin{array}{ll}
2206 & \Gamma_c^{\oplus'} \vdash s[\mathbf{p}] : S \quad (\text{by (8), (16), } \Gamma\text{-}\oplus, \text{T-}\oplus) \quad (19) \\
2207 & \Gamma_{c_k}^{\dagger} = \Gamma_c^{\dagger} \cdot \Gamma_k \text{ such that:} \quad (\text{by (13), (17), } \Gamma\text{-!}) \quad (20) \\
2208 & \Gamma_k \vdash s[\mathbf{q}] : S'_k \quad (\text{by (20), (13), (17), } \Gamma\text{-!}, \text{T-!}) \quad (21) \\
2209 & \Gamma_k\{\vec{r}/\vec{\alpha}_k\} \vdash s[\mathbf{q}] : S'_k\{\vec{r}/\vec{\alpha}_k\} \quad (\text{by (21), lemma A.4}) \quad (22) \\
2210 & \vec{T} \leq \vec{T}'_k \quad (\text{by (16), (17), (18), } \Gamma\text{-Com}_1) \quad (23) \\
2211 & \Gamma_0^{\oplus} + \Gamma_c^{\oplus'} \vdash_D Q \quad (\text{by (11), (19)}) \quad (24) \\
2212 & \Gamma_0^{\dagger} + \Gamma_c^{\dagger} \vdash_D R \quad (\text{by (12)}) \quad (25) \\
2213 & \Gamma_0^{\dagger} + \Gamma_k\{\vec{r}/\vec{\alpha}_k\} + \overrightarrow{\alpha_i : \alpha_i} + x_i : T'_i \vdash_D Q'_k\{\vec{r}/\vec{\alpha}_k\} \quad (\text{by (15), (22), lemma A.5}) \quad (26) \\
2214 & \Gamma_0^{\dagger} + \Gamma_k\{\vec{r}/\vec{\alpha}_k\} + \overrightarrow{\alpha_i : \alpha_i} \vdash_D Q'_k\{\vec{r}/\vec{\alpha}_k\} \quad (\text{by (26), lemma A.7}) \quad (27) \\
2215 & \Gamma_0^{\dagger} + \Gamma_k\{\vec{r}/\vec{\alpha}_k\} (+\Gamma_{d_i}^{\oplus})_{i \in 1..n} \vdash_D Q'_k\{\vec{d}/\vec{b}_k\} \quad (\text{by (10), (15), (23), lemma A.8}) \quad (28) \\
2216 & \Gamma_0^{\dagger} + \Gamma_c^{\oplus'} + \Gamma_0^{\dagger} + \Gamma_c^{\dagger} + \Gamma_0^{\dagger} + \Gamma_k\{\vec{r}/\vec{\alpha}_k\} (+\Gamma_{d_i}^{\oplus})_{i \in 1..n} \vdash_D P' \quad (\text{by (24), (25), (28), T-|}) \quad (29) \\
2217 & \Gamma_0^{\dagger} + \Gamma_c^{\oplus'} + \Gamma_0^{\dagger} + \Gamma_c^{\dagger} + \Gamma_0^{\dagger} + \Gamma_k\{\vec{r}/\vec{\alpha}_k\} (+\Gamma_{d_i}^{\oplus})_{i \in 1..n} \vdash_D P' \quad (\text{by (24), (25), (28), T-|}) \quad (29) \\
2218 & \\
2219 & \text{By (12) and (14), } \Gamma_0^{\dagger} \text{ is end-typed. Assume without loss of generality that } \Gamma_0^{\dagger} \text{ does not contain any} \\
2220 & \text{session typed entities, since any end-typed channels could be grouped with a different split of the} \\
2221 & \text{context instead. Then, it holds that } \Gamma_0^{\dagger} = \Gamma_0^{\dagger} \cdot \Gamma_0^{\dagger} \\
2222 & \Gamma \rightarrow \Gamma' \\
2223 & = (\Gamma_c^{\oplus'} + (\Gamma_c^{\dagger} \cdot \Gamma_k\{\vec{r}/\vec{\alpha}_k\})) + (\Gamma_0^{\dagger} \cdot \Gamma_0^{\dagger} \cdot \Gamma_0^{\oplus} \cdot \Gamma_r^{\oplus} (\cdot \Gamma_{d_i}^{\oplus})_{i \in 1..n}) \quad (\text{by (18), } \Gamma\text{-Cong}_1) \quad (30) \\
2224 & = \Gamma_0^{\oplus} + \Gamma_c^{\oplus'} + \Gamma_0^{\dagger} + \Gamma_c^{\dagger} + \Gamma_0^{\dagger} + \Gamma_k\{\vec{r}/\vec{\alpha}_k\} (+\Gamma_{d_i}^{\oplus})_{i \in 1..n} + \Gamma_r^{\oplus} \quad (\text{by (30), lemma A.1}) \quad (31) \\
2225 & \text{end}(\Gamma_r^{\oplus}) \quad (\text{by (9), T-Wkn, T-q}) \quad (32) \\
2226 & \Gamma' \vdash_D P' \quad (\text{by (29), (31), (32), lemma A.2}) \quad (33) \\
2227 & \text{safe}_1(\Gamma') \quad (\text{by (A2), (30), } \varphi \rightarrow) \quad (34) \\
2228 & \text{Case R-!C}_1 \text{ holds.} \quad (\text{by (30), (33), (34)}) \quad (35)
\end{array}$$

Case R-!C₂:

$$\begin{array}{ll}
2234 & \text{Let } R = !s[\mathbf{q}][\delta] \&_{i \in I} m_i(\vec{b}_i) \cdot Q'_i \quad (\text{definition}) \quad (1) \\
2235 & \text{Let } P_{\oplus} = s[\mathbf{p}][\mathbf{q}] \oplus m_k(\vec{d}) \cdot Q \quad (\text{definition}) \quad (2) \\
2236 & \text{Let } n = |\vec{d}| \quad (\text{definition}) \quad (3) \\
2237 & P = P_{\oplus} \mid R \quad (\text{by R-!C}_2 \text{ and (A3)}) \quad (4) \\
2238 & P' = Q \mid R \mid Q'_k\{\vec{d}/\vec{b}_k\}\{\mathbf{p}/\delta\} \quad (\text{by R-!C}_2 \text{ and (A3)}) \quad (5) \\
2239 &
\end{array}$$

Without loss of generality, consider $\vec{d} = \vec{r}, \vec{d}'$ where $\vec{d}'$ does not contain role values.

$$\begin{array}{ll}
2241 & \Gamma = \Gamma^{\oplus} \cdot \Gamma^{\dagger} \vdash_D P \quad (\text{by (4), (A1), and T-|}) \quad (6) \\
2242 & \\
2243 & \Gamma^{\oplus} = \Gamma_0^{\oplus} \cdot \Gamma_c^{\oplus} \cdot \Gamma_r^{\oplus} (\cdot \Gamma_{d_i}^{\oplus})_{i \in 1..n} \vdash_D P_{\oplus} \quad (\text{by (6), (4), T-}\oplus) \quad (7) \\
2244 & \Gamma_c^{\oplus} \vdash s[\mathbf{p}] : \oplus \mathbf{q} : m_k(\vec{r}, \vec{T}) \cdot S \quad (\text{by (7), T-}\oplus) \quad (8) \\
2245 & \Gamma_r^{\oplus} \vdash \mathbf{q} : \mathbf{q} \quad (\text{by (7), T-}\oplus) \quad (9) \\
2246 & \forall i \in 1..|\vec{r}| : \Gamma_{d_i}^{\oplus} \vdash d_i : \mathbf{r}_i \text{ and } \forall j \in |\vec{r}|..|\vec{T}| : \Gamma_{d_j}^{\oplus} \vdash d_j : T_j \quad (\text{by (7), T-}\oplus) \quad (10) \\
2247 & \Gamma_0^{\oplus} + s[\mathbf{p}] : S \vdash_D Q \quad (\text{by (7), T-}\oplus) \quad (11) \\
2248 & \\
2249 & \Gamma^{\dagger} = \Gamma_0^{\dagger} \cdot \Gamma_c^{\dagger} \vdash_D R \quad (\text{by (6), (4), T-!}) \quad (12) \\
2250 & \Gamma_c^{\dagger} \vdash s[\mathbf{q}] : !\&_{i \in I} \delta : m_i(\vec{\alpha}_i, \vec{T}_i) \cdot S'_i \quad (\text{by (12), T-!}) \quad (13) \\
2251 & \text{end}(\Gamma_0^{\dagger}) \quad (\text{by (12), T-!}) \quad (14) \\
2252 & \forall i \in I : \Gamma_0^{\dagger} + s[\mathbf{q}] : S'_i + \overrightarrow{\alpha_i : \alpha_i} + x_i : T'_i \leftrightarrow \delta \vdash_D Q'_i \quad (\text{by (12), T-!}) \quad (15) \\
2253 & \\
2254 &
\end{array}$$

$$\Gamma_c^\oplus \xrightarrow{s:p^\oplus q:m_k(\vec{r}, \vec{T})} \Gamma_c^{\oplus'} \quad (\text{by (8), T-Wkn, T-Sub, definition 2.3}) \quad (16)$$

$$\Gamma_c^! \xrightarrow{s:q\&\delta:m_j'(\vec{\alpha}_j, \vec{T}_j')} \Gamma_{c_j}^{!'} \text{ for } j \in I \quad (\text{by (13), T-Wkn, T-Sub, definition 2.3}) \quad (17)$$

$$\Gamma_c^\oplus \cdot \Gamma_c^! \xrightarrow{s:p,q:m_k} \Gamma_c^{\oplus'} + \Gamma_{c_k}^{!'} \{\vec{r}, p/\vec{\alpha}, \delta\} \quad (\text{by (16), (17), (A2), lemma B.1, } \Gamma\text{-Com}_1) \quad (18)$$

$$\Gamma_c^{\oplus'} \vdash s[p] : S \quad (\text{by (8), (16), } \Gamma\text{-}\oplus, \text{T-}\oplus) \quad (19)$$

$$\Gamma_{c_k}^{!'} = \Gamma_c^! \cdot \Gamma_k \text{ such that:} \quad (\text{by (13), (17), } \Gamma\text{-!}) \quad (20)$$

$$\Gamma_k \vdash s[q] : S'_k \quad (\text{by (20), (13), (17), } \Gamma\text{-!}, \text{T-!}) \quad (21)$$

$$\Gamma_k \{\vec{r}/\vec{\alpha}_k\} \{p/\delta\} \vdash s[q] : S'_k \{\vec{r}/\vec{\alpha}_k\} \{p/\delta\} \quad (\text{by (21), lemma A.4}) \quad (22)$$

$$\vec{T} \leq \vec{T}'_k \quad (\text{by (16), (17), (18), } \Gamma\text{-Com}_1) \quad (23)$$

$$\Gamma_0^\oplus + \Gamma_c^{\oplus'} \vdash_D Q \quad (\text{by (11), (19)}) \quad (24)$$

$$\Gamma_0^! + \Gamma_c^! \vdash_D R \quad (\text{by (12)}) \quad (25)$$

$$\Gamma_0^! + \Gamma_k \{\vec{r}/\vec{\alpha}_k\} \{p/\delta\} + \overline{\alpha_i} : \vec{\alpha}_i + x_i : T'_i \vdash_D Q'_k \{\vec{r}/\vec{\alpha}_k\} \{p/\delta\} \quad (\text{by (15), (22), lemma A.5}) \quad (26)$$

$$\Gamma_0^! + \Gamma_k \{\vec{r}/\vec{\alpha}_k\} \{p/\delta\} + \overline{x_i} : T'_i \vdash_D Q'_k \{\vec{r}/\vec{\alpha}_k\} \{p/\delta\} \quad (\text{by (26), lemma A.7}) \quad (27)$$

$$\Gamma_0^! + \Gamma_k \{\vec{r}/\vec{\alpha}_k\} \{p/\delta\} (+\Gamma_{d_i}^\oplus)_{i \in 1..n} \vdash_D Q'_k \{\vec{d}/\vec{b}_k\} \quad ((10), (15), (23), \text{lemma A.8}) \quad (28)$$

$$\Gamma_0^\oplus + \Gamma_c^{\oplus'} + \Gamma_0^! + \Gamma_c^! + \Gamma_0^! + \Gamma_k \{\vec{r}/\vec{\alpha}_k\} \{p/\delta\} (+\Gamma_{d_i}^\oplus)_{i \in 1..n} \vdash_D P' \quad (\text{by (24), (25), (28), T-|}) \quad (29)$$

By (12) and (14), $\Gamma_0^!$ is end-typed. Assume without loss of generality that $\Gamma_0^!$ does not contain any session typed entities, since any end-typed channels could be grouped with a different split of the context instead. Then, it holds that $\Gamma_0^! = \Gamma_0^! \cdot \Gamma_0^!$

$$\Gamma \rightarrow \Gamma' = (\Gamma_c^{\oplus'} + (\Gamma_c^! \cdot \Gamma_k \{\vec{r}/\vec{\alpha}_k\} \{p/\delta\})) + (\Gamma_0^! \cdot \Gamma_0^! \cdot \Gamma_0^\oplus \cdot \Gamma_r^\oplus (\cdot \Gamma_{d_i}^\oplus)_{i \in 1..n}) \quad (\text{by (18), } \Gamma\text{-Cong}_1) \quad (30)$$

$$= \Gamma_0^\oplus + \Gamma_c^{\oplus'} + \Gamma_0^! + \Gamma_c^! + \Gamma_0^! + \Gamma_k \{\vec{r}/\vec{\alpha}_k\} \{p/\delta\} (+\Gamma_{d_i}^\oplus)_{i \in 1..n} + \Gamma_r^\oplus \quad (\text{by (30), lemma A.1}) \quad (31)$$

$$\text{end}(\Gamma_r^\oplus) \quad (\text{by (9), T-Wkn, T-q}) \quad (32)$$

$$\Gamma' \vdash_D P' \quad (\text{by (29), (31), (32), lemma A.2}) \quad (33)$$

$$\text{safe}_l(\Gamma') \quad (\text{by (A2), (30), } \varphi\text{-}\rightarrow) \quad (34)$$

$$\text{Case R-!C}_2 \text{ holds.} \quad (\text{by (30), (33), (34)}) \quad (35)$$

Remaining cases are straight forward. □

C Session Fidelity

LEMMA C.1. Assume $\Gamma \vdash_D P$ with one-role_l(P). Then $P \equiv \prod_{p \in I} P_p$ and $\Gamma = \Gamma_0(\cdot \Gamma'_p, s[p] : U_p)_{p \in I'}$, where:

- (1) $\forall p \in I' : \Gamma'_p, s[p] : U_p \vdash_D P_p$ and $c : U \notin \Gamma'_p$;
- (2) $\forall q \in I \setminus I' : P_q \equiv 0$;
- (3) $c : U \notin \Gamma_0$.

PROOF. First observe that by definition 3.15(2), $P \equiv \prod_{p \in I} P_p$ where $\forall p \in I : P_p \equiv 0$ or P_p only plays role p . Therefore, by rule T-|, $\Gamma = (\cdot \Gamma_p)_{p \in I}$ where $\forall p \in I : \Gamma_p \vdash_D P_p$. For each split of the context, there are two cases to consider.

If $P_p \equiv 0$, then by **rule T-0**, $\text{end}(\Gamma_p)$.

Otherwise, P_p only plays role p . Since $\Gamma_p \vdash_D P_p$, then Γ_p serves as witness for definition 3.15(1). Hence, it can be concluded that $\Gamma_p = \Gamma''_p, s[p] : U_p$ with $U_p \not\leq \text{end}$ and $\text{end}(\Gamma''_p)$.

Therefore, $\Gamma = (\cdot \Gamma''_p, s[p] : U_p)_{p \in I''} (\cdot \Gamma_q)_{q \in I \setminus I''}$ where $\forall p \in I'' : \Gamma''_p, s[p] : U_p \vdash_D P_p$ with $\text{end}(\Gamma''_p)$ and $\forall q \in I \setminus I'' : \Gamma_q \vdash_D P_q \equiv 0$ with $\text{end}(\Gamma_q)$.

Without loss of generality, assume that all end-typed channels are grouped in $I \setminus I''$. Hence, $\forall p \in I'' : \Gamma''_p, s[p] : U_p \vdash_D P_p$ with $c : U \notin \Gamma''_p$ and $\forall q \in I \setminus I'' : \Gamma_q \vdash_D P_q \equiv 0$ with $\text{end}(\Gamma_q)$ and either $\Gamma_q = \Gamma''_q, s[q] : \text{end}$ or $c : U \notin \Gamma''_q$.

Thus, $(\cdot \Gamma_q)_{q \in I \setminus I''} = \Gamma_0(\cdot \Gamma_q)_{q \in I'''} where $c : U \notin \Gamma_0$ and $I''' \subseteq I \setminus I''$.$

Let $I' = (I \setminus I'') \cup I'''$, then $\Gamma = \Gamma_0(\cdot \Gamma'_p, s[p] : U_p)_{p \in I'}$. □

LEMMA C.2. Assume $\Gamma_0, s[p] : U_p \vdash_D P_p$ with $c : U \notin \Gamma_0$ and either P_p only plays role p or $P_p \equiv 0$. Then:

- (1) if $S_1 | \dots | S_n \leq U_p$, then $P_p \equiv P_1 | \dots | P_n$ and $\forall i \in 1..n : \Gamma_0, s[p] : S_i \vdash_D P_i$ and P_i only plays role p ;
- (2) if $!\&_{i \in I} p : m_i(\vec{T}_i).S_i \leq U_p$, then either:
 - (a) $P_p \equiv !s[p][\rho] \&_{i \in I'} m_i(\vec{b}_i).P'_i$ and $I \subseteq I'$; or
 - (b) $P_p \equiv X\langle \vec{V} \rangle$ and $D(X) = (\vec{b}', \vec{T}', Q)$ and $Q\{\vec{V}/\vec{b}'\} \equiv !s[p][\rho] \&_{i \in I'} m_i(\vec{b}_i).P'_i$ and $I \subseteq I'$;
- (3) if $\&_{i \in I} q : m_i(\vec{T}_i).S_i \leq U_p$, then either:
 - (a) $P_p \equiv s[p][q] \&_{i \in I'} m_i(\vec{b}_i).P'_i$ and $I \subseteq I'$; or
 - (b) $P_p \equiv X\langle \vec{V} \rangle$ and $D(X) = (\vec{b}', \vec{T}', Q)$ and $Q\{\vec{V}/\vec{b}'\} \equiv s[p][q] \&_{i \in I'} m_i(\vec{b}_i).P'_i$ and $I \subseteq I'$;
- (4) if $\oplus_{i \in I} q_i : m_i(\vec{T}_i).S_i \leq U_p$, then either:
 - (a) $P_p \equiv \sum_{i \in I'} s[p][q_i] \oplus m_i(\vec{V}_i).P'_i$ and $I \supseteq I'$; or
 - (b) $P_p \equiv X\langle \vec{V} \rangle$ and $D(X) = (\vec{b}', \vec{T}', Q)$ and $Q\{\vec{V}/\vec{b}'\} \equiv \sum_{i \in I'} s[p][q_i] \oplus m_i(\vec{V}_i).P'_i$ and $I \supseteq I'$;
- (5) if $\text{end} \leq U_p$, then $P_p \equiv 0$.

PROOF. By case analysis on the 5 possible implications.

Case 1:

Follows from the definition of context splits (fig. 5), observing that since $c : U \notin \Gamma_0$ then $\Gamma_0, s[p] : S_1 | \dots | S_n = \Gamma_0, s[p] : S_1 \cdot \dots \cdot \Gamma_0, s[p] : S_n$.

Furthermore, by **rule T-|**, $\forall i \in 1..n : \Gamma_0, s[p] : S_i \vdash_D P_i$.

Lastly, from the hypothesis it follows that D is guarded, $\text{fv}(P) = \emptyset$ and $U_p \not\leq \text{end}$, thus $\forall i \in 1..n : \text{fv}(P_i) = \emptyset$ and $S_i \not\leq \text{end}$. (Notice that any **end** in the parallel type can be removed via type congruence, and also reflected in the process.) Therefore, by definition 3.15, $\forall i \in 1..n : P_i$ only plays role p , by $\Gamma_0, s[p] : S_i$.

Case 2:

From the hypothesis, and by subtyping and narrowing, $\Gamma_0, s[p] : !\&_{i \in I} p : m_i(\vec{T}_i).S_i \vdash_D P_p$. By inversion of typing rules, there are two possible rules that could be applied.

Rule T-!:

$$\Gamma_0, s[p] : !\&_{i \in I} p : m_i(\vec{T}_i).S_i \vdash_D !s[p][\rho] \&_{i \in I} m_i(\vec{b}_i).P'_i \quad (\text{by T-!}) \quad (1)$$

$$\Gamma_0, s[p] : U_p \vdash_D !s[p][\rho] \&_{i \in I} m_i(\vec{b}_i).P'_i \text{ where } I \subseteq I' \quad (\text{by (1) and definition 2.3}) \quad (2)$$

$$P_p \equiv !s[p][p] \&_{i \in I'} m_i(\vec{b}_i).P'_i \quad (\text{by (2) and subject congruence (lemma A.12)}) \quad (3)$$

Rule T-X:

By inversion of **T-X**, $P_p \equiv X(\vec{V})$ and $\exists k \in 1..|\vec{V}| : V_k = s[p]$ and $s[p] : U_p \vdash V_k : T'_k$. Without loss of generality, assume $k = 1$, then $D(X) = ((x, \vec{b}), (T_k, \vec{T}'), Q)$.

Assuming that $\Gamma_0, s[p] : U_p \vdash_D P_p$ is derived from a well-typed program, then it follows that $x : T'_k, \{\vec{b} : \vec{T}'\} \vdash_D Q$ by **rule T-P**. Furthermore, by definition 3.14, it follows that Q cannot be typed by a further application of **rule T-X**, and thus must instead be typed by **rule T-!**. Following the same reasoning as outlined in the previous subcase, it must be that $Q \equiv !x[p] \&_{i \in I'} m_i(\vec{b}_i).P'_i$ and $I \subseteq I'$

Case 3, 4: Similar to **Case 2**.

Case 5:

Since $\text{end} \leq U_p$ and $c : U \notin \Gamma_0$, then $\text{end}(\Gamma_0, s[p] : U_p)$. By inversion of the typing rules, the only possible shape for P_p is $P_p \equiv 0$ via **rule T-0**. $\square$

THEOREM C.3 (SESSION FIDELITY). Assume $\Gamma \vdash_D P$ with $\text{fid}_!(\Gamma)$ and $\text{one-role}_!(P)$. Then, $\Gamma \rightarrow$ implies $\exists \Gamma', P' \text{ s.t.:$

- (i) $\Gamma \rightarrow \Gamma'$;
- (ii) $P \rightarrow_D^* P'$;
- (iii) $\Gamma' \vdash_D P'$ with $\text{fid}_!(\Gamma')$; and
- (iv) $\text{one-role}(P')$.

PROOF. By induction on the derivation of $\Gamma \rightarrow$. There are two possible cases by which Γ can be enabled with a reduction: by **rule $\Gamma\text{-Com}_1$** or by **rule $\Gamma\text{-Com}_2$** . The following elaborates on the latter case (the other case is similar).

The assumptions are: **(A1)** $\Gamma \vdash_D P$; **(A2)** $\text{fid}_!(\Gamma)$; **(A3)** $\text{one-role}_!(P)$; and **(A4)** $\Gamma \rightarrow$.

The proof begins by inferring some structure on both the types and processes, utilizing lemma C.1.

$$P \equiv \prod_{p \in I} P_p \text{ and } \Gamma = \Gamma_0(\cdot \Gamma'_p, s[p] : U_p)_{p \in I'} \quad (\text{by (A1), (A3), and lemma C.1}) \quad (1)$$

$$\forall p \in I' : \Gamma'_p, s[p] : U_p \vdash_D P_p \text{ and } c : U \notin \Gamma'_p \quad (\text{by (A1), (A3), and lemma C.1}) \quad (2)$$

$$\forall q \in I \setminus I' : P_q \equiv 0 \quad (\text{by (A1), (A3), and lemma C.1}) \quad (3)$$

$$c : U \notin \Gamma_0 \quad (\text{by (A1), (A3), and lemma C.1}) \quad (4)$$

Since the context is known to be enabled with a reduction (by assumption), the actions emitted by the context can be inferred from **rule $\Gamma\text{-Com}_2$** . Specifically, $\exists p, q \in I'$ such that:

$$\Gamma'_p, s[p] : U_p \xrightarrow{s:p \oplus q : m(\vec{r}, \vec{T}_1)} \Gamma'_p, s[p] : U'_p \quad (\text{by (1), (A4) and } \Gamma\text{-Com}_2) \quad (5)$$

$$\Gamma'_q, s[q] : U_q \xrightarrow{s:q \& \alpha' : m(\vec{\alpha}, \vec{T}_2)} \Gamma'_q, s[q] : U'_q \quad (\text{by (1), (A4) and } \Gamma\text{-Com}_2) \quad (6)$$

$$\vec{T}_1 \leq \vec{T}_2 \quad (\text{by (1), (A4) and } \Gamma\text{-Com}_2) \quad (7)$$

$$(\Gamma'_p, s[p] : U_p) \cdot (\Gamma'_q, s[q] : U_q) \xrightarrow{s:p, q : m} (\Gamma'_p, s[p] : U'_p) + (\Gamma'_q, s[q] : U'_q \{ \vec{r} / \vec{\alpha} \} \{ p / \alpha' \})$$

(by (5), (6), (7), and $\Gamma\text{-Com}_2$) (8)

From these actions, the shapes of the types can be outlined, following from the type semantics in fig. 7.

$$\oplus_{j \in J} \mathbf{q}_j : \mathbf{m}_j(\vec{T}_j).S_j \leq U_p \leq \mathbf{q} \oplus \mathbf{m}(\vec{r}, \vec{T}_1).S \quad (\text{by (5), } \Gamma\text{-}\oplus) \quad (9)$$

$$\exists l \in J : \mathbf{q}_l = \mathbf{q} \wedge \mathbf{m}_l = \mathbf{m} \wedge \vec{T}_l \geq \vec{T}_1 \quad (\text{by (9), definition 2.3}) \quad (10)$$

$$! \&_{k \in K} \alpha' : \mathbf{m}'_k(\vec{\delta}_k, \vec{T}'_k).S'_k \leq U_q \text{ and } \exists l \in K : \mathbf{m}'_l = \mathbf{m} \quad (\text{by (6), } \Gamma\text{-}!) \quad (11)$$

$$\vec{T}'_1 \leq \vec{T}_2 \quad (\text{by (11), (6), definition 2.3}) \quad (12)$$

The shapes of processes can now be inferred from the types, using (2), (9), lemma C.2 for P_p , and (2), (11), lemma C.2 for P_q . Note that by the session inversion lemma, there are two possible shapes for each process; thus a total of four possible permutations to consider, resulting in 4 subcases.

Case $\oplus!$: Consider:

$$P_p \equiv \sum_{j \in J'} s[\mathbf{p}][\mathbf{q}_j] \oplus \mathbf{m}_j(\vec{V}_j).P'_j \text{ and } J \supseteq J' \quad (\text{by case assumption}) \quad (13)$$

$$P_q \equiv !s[\mathbf{q}][\alpha'] \&_{k \in K'} \mathbf{m}'_k(\vec{b}_k).P''_k \text{ and } K \subseteq K' \quad (\text{by case assumption}) \quad (14)$$

This is where the novel fidelitous property comes into play. As it stands, there is no guarantee that $\exists x \in J' : \mathbf{q}_x = \mathbf{q}$, as the implementation is a subset of the type and could hence implement a part of the selection that does not include the role considered for communication in (8). However, by (8), it is guaranteed that at least one (type-level) communication is possible. Then by (A2) and table 1, it follows that communication should be eventually possible for every path in J , and thus also J' (since $J' \subseteq J$ by (13)). Therefore, it can be assumed without loss of generality that the nondeterministic choice in P_p reduces to a path x in J' such that $\mathbf{q}_x = \mathbf{q}$.

$$P_p \rightarrow P'_p \equiv s[\mathbf{p}][\mathbf{q}] \oplus \mathbf{m}_x(\vec{V}_x).P'_x \text{ and } x \in J' \quad (\text{explanation above}) \quad (15)$$

$$\exists y \in K : \mathbf{m}_y = \mathbf{m}_x \quad (\text{by (15), (A2), definition 3.16}) \quad (16)$$

$$y \in K' \quad (\text{by (16) and (14)}) \quad (17)$$

$$P'_p \mid P_q \rightarrow P'_x \mid P_q \mid P''_y\{\mathbf{p}/\alpha'\}\{\vec{V}_x/\vec{b}_y\} \quad (\text{by (14), (15), (17), R-!C}_2) \quad (18)$$

$$\Gamma'_{p'}, s[\mathbf{p}] : U_p \xrightarrow{s:\mathbf{p} \oplus \mathbf{q} : \mathbf{m}_x(\vec{r}', \vec{T}_x)} \Gamma'_{p'}, s[\mathbf{p}] : U'_p \quad (\text{by (2), (15), } \Gamma\text{-}\oplus) \quad (19)$$

$$(\Gamma'_{p'}, s[\mathbf{p}] : U_p) \cdot (\Gamma'_{q'}, s[\mathbf{q}] : U_q) \xrightarrow{s:\mathbf{p}, \mathbf{q} : \mathbf{m}_x} (\Gamma'_{p'}, s[\mathbf{p}] : U'_p) + (\Gamma'_{q'}, s[\mathbf{q}] : U'_q\{\vec{r}'/\vec{\delta}\}\{\mathbf{p}/\alpha'\}) \quad (\text{by (19), (8), (A2)}) \quad (20)$$

$$\Gamma'_{q'}, s[\mathbf{q}] : U_q \xrightarrow{s:\mathbf{q} \oplus \alpha' : \mathbf{m}_x(\vec{\delta}, \vec{T}'_x)} \Gamma'_{q'}, s[\mathbf{p}] : U'_q \quad (\text{by (20), (19), } \Gamma\text{-Com}_2) \quad (21)$$

$$\vec{T}_x \leq \vec{T}'_x \quad (\text{by (21), (20), (19), } \Gamma\text{-Com}_2) \quad (22)$$

The subcase is concluded by observing that the reduced contexts type the reduced processes, and that the reduced processes still adhere to the one-role_! requirements.

$$\Gamma'_{p'}, s[\mathbf{p}] : U'_p \vdash_D P'_x \quad (\text{by (2), (19), T-}\oplus) \quad (23)$$

$$\Gamma'_{q'}, s[\mathbf{p}] : U'_q\{\mathbf{p}/\alpha'\}\{\vec{r}'/\vec{\delta}\} \vdash_D P''_y\{\mathbf{p}/\alpha'\}\{\vec{V}_x/\vec{b}_y\} \quad (\text{by (2), (21), (22), (23), T-!, and both substitution lemmas}) \quad (24)$$

Note that for the above, all the types for payloads are present in $\Gamma'_{q'}$ since by the session fidelity assumptions, payloads can only contain non-session-typed entities; these are copied through context splits (fig. 5).

$$P \rightarrow^* P' \mid P'_x \mid P_q \mid P''_y\{\mathbf{p}/\alpha'\}\{\vec{V}_x/\vec{b}_y\} \text{ with one-role}_!(P') \quad (\text{by (A3), (15), (18)}) \quad (25)$$

2451	$\text{one-role}_!(P'_x \mid P_q \mid P''_y\{\mathbf{p}/\alpha'\}\{\vec{V}_x/\vec{b}_y\})$	(by (1), (23), (24), definition 3.15)	(26)
2452	$\Gamma \rightarrow \Gamma'$	(by (20))	(27)
2453	$P \rightarrow^* P' \mid P'_x \mid P_q \mid P''_y\{\mathbf{p}/\alpha'\}\{\vec{V}_x/\vec{b}_y\}$	(by (25))	(28)
2454	$\Gamma' \vdash_D P' \mid P'_x \mid P_q \mid P''_y\{\mathbf{p}/\alpha'\}\{\vec{V}_x/\vec{b}_y\}$	(by (2), (20), (23), (24), T-)	(29)
2455	$\text{fid}_!(\Gamma')$	(by (A2), (27), $\varphi \rightarrow$)	(30)
2456	$\text{one-role}_!(P' \mid P'_x \mid P_q \mid P''_y\{\mathbf{p}/\alpha'\}\{\vec{V}_x/\vec{b}_y\})$	(by (25), (26))	(31)
2458	Case $\oplus!$ holds.	(by (25), (27)–(31))	(32)

2459

2460

2461

2462

2463

2464

2465

2466

2467

2468

2469

2470

2471

2472

2473

2474

2475

2476

2477

2478

2479

2480

2481

2482

2483

2484

2485

2486

2487

2488

2489

2490

2491

2492

2493

2494

2495

2496

2497

2498

2499

The remaining subcases all follow similar reasoning, except with the additional step of allowing for process reduction via rule R-X before communication becomes available.

□

Received X November 2025